OPERATOR LAB · ORACLE RAC 19C ON KUBEADM

The operator lab: Phases 0–11

A second, separate lab next to the manual one, built the same way up to Multus, then handed to Oracle's database operator for the RAC database. Own VM names, IPs, ports and disks; the same golden image.

`op-controlplane · op-worker1 · op-worker2` `10.0.1.20–22` `SSH 2220–2222` `D:\VMs\rac-op`

`Oracle Database Operator v2.2.0`

26 Oct 2026 · Companion to the manual lab guide; Phases 7 onward follow oracle/oracle-database-operator.

`PowerShell / Git Bash`

on your PC

`golden VM · controlplane · worker`

root shell on a VM

`racnode1-0 · racnode2-0`

root shell in a RAC pod

How to use this guide

This guide builds a **second lab, the operator lab**, next to the manual lab you already have. It follows the manual guide's Phases 0 and 3–6 step by step, with its own names, addresses and ports, so the two labs never overwrite each other. The RAC database itself will come from Oracle's database operator in the later phases, instead of the hand-run installs of the manual lab's Phases 8–14.

Copy every block exactly as it is. Each block's label says where it runs, and the **connect** line shows how to get there. Blocks labelled `# PowerShell` run in a PowerShell window on your PC, never in Git Bash.

This lab's output blocks after a **Check** show what this lab's own run printed, so you can compare your screen line by line. Long logs are cut to the lines that matter (`…` marks a cut); where a step first went wrong, that output is shown too, labelled, next to the corrected run. IDs, UUIDs, image IDs and times will differ on your build. Steps without such a block weren't captured during this run (most of Phases 4 and 5).

	MANUAL LAB (UNCHANGED)	OPERATOR LAB (THIS GUIDE)
VM names and host names	`controlplane`, `worker1`, `worker2`	`op-controlplane`, `op-worker1`, `op-worker2`
IPs on `k8s-public` 10.0.1.0/24	.10, .11, .12	**.20, .21, .22**

	MANUAL LAB (UNCHANGED)	OPERATOR LAB (THIS GUIDE)
SSH from Windows	2210, 2211, 2212	**2220, 2221, 2222**
VM and disk folder	`D:\VMs\rac-poc`	`D:\VMs\rac-op`
Shared ASM disks	`rac-poc\shared\asm1.vdi`, `asm2.vdi`: one disk group `DATA`	`rac-op\shared\vote1.vdi` **(10 GB)**, `asm1.vdi`, `asm2.vdi`: disk groups `VOTEDG`, `DATA`, `FRA`
Private networks (VirtualBox internal)	`rac-priv1`, `rac-priv2`	`op-priv1`, `op-priv2`, same subnets 10.0.11/12.0/24
Kubernetes cluster name	`rac-poc`	`rac-op`
Golden VM `ol98-golden` (2205, 10.0.1.5)	shared	shared: only the clone source; nothing changes in it
NAT Network `k8s-public`	shared	shared: the operator lab only adds its own SSH forwards

Skipped phases: Phases 1 and 2 built the golden VM, which both labs share. Phase 7 (ASM disks as Kubernetes volumes) and everything after it depend on what the operator expects, so they come after reading Oracle's operator documentation.

One lab at a time. Each lab needs about 48 GB of the PC's 64 GB, and both use the same NAT Network. Before you start the operator lab, the manual lab must be powered off, and the other way round:

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
```

Check: before working on the operator lab, nothing is listed, or only `op-*` VMs.

Switching labs later (after both are built):

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
# stop the manual lab (stop Grid in its pods first, as in "how to shut down")
foreach ($n in "worker2","worker1","controlplane") { & $VB controlvm $n acpipowerbutton 2>$null }
# start the operator lab
foreach ($n in "op-controlplane","op-worker1","op-worker2") { & $VB startvm $n --type headless }
```

Before stopping the operator lab, stop Clusterware in its pods first, as in the runbook in 14.6. To go back, swap the two lists: stop `op-worker2`, `op-worker1`, `op-controlplane`, then start `controlplane`, `worker1`, `worker2`.

PHASE 0

Windows host and the operator lab's port forwards

Result of this phase: a clean starting point for the operator lab, the PC checked for room, and SSH port forwards 2220–2222 on the shared `k8s-public` network. The manual lab and the golden VM are not touched.

0.0 Start clean (only if you built the operator lab before)

What: removes an earlier operator lab: the VMs `op-controlplane`, `op-worker1`, `op-worker2`, the folder `D:\VMs\rac-op` with all its disks, the operator lab's port-forward rules, and the SSH host keys remembered for ports 2220–2222. On a first build it just reports that there's nothing to remove.

What it never touches: `ol98-golden`, `controlplane`, `worker1`, `worker2`, `D:\VMs\rac-poc` and the `k8s-public` network itself. Every name in the block starts with `op-` or `rac-op`.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
$existing = (& $VB list vms) -join "`n"
foreach ($v in "op-worker1","op-worker2","op-controlplane") {
  if ($existing -match ('"' + $v + '"')) { & $VB controlvm $v poweroff 2>$null; Start-Sleep 3; & $VB
unregistervm $v --delete; "removed VM $v" }
}
$left = (& $VB list hdds) | Select-String '^Location:\s+(.*)$' | ForEach-Object {
$_.Matches[0].Groups[1].Value } | Where-Object { $_ -like "$Base\*" }
foreach ($d in $left) { & $VB closemedium disk $d --delete; "removed disk $d" }
if (Test-Path $Base) { Remove-Item $Base -Recurse -Force; "removed folder $Base" }
foreach ($r in "ssh-opcp","ssh-opw1","ssh-opw2","k8sapi-op") { & $VB natnetwork modify --netname k8s-
public --port-forward-4 delete $r 2>$null }
foreach ($f in "$env:USERPROFILE\.ssh\known_hosts", "$env:USERPROFILE\.ssh\known_hosts_rac_lab") {
  if (Test-Path $f) { foreach ($p in 2220, 2221, 2222) { ssh-keygen -R "[127.0.0.1]:$p" -f $f 2>$null |
Out-Null } }
}
"--- VMs (the manual lab and ol98-golden must still be listed; no op-*) ---"
& $VB list vms
```

Check: `list vms` still shows `ol98-golden`, `controlplane`, `worker1` and `worker2`, and no `op-*` VM.

This lab's output:

Output — what this lab printed; nothing to run

```
PS> & $VB list vms        (first build: nothing to remove, so no "removed …" lines before this)
--- VMs (the manual lab and ol98-golden must still be listed; no op-*) ---
…                                  (other VMs on this PC)
"ol98-golden" {9c69f3bd-2c8a-420a-914b-ad3922917b45}
"controlplane" {779954cf-d37f-45c4-86b3-ee95bc0ee6dc}
"worker1" {4a3c7ad5-0e74-4f94-a556-81aa2795b42e}
"worker2" {5f7040de-3e38-4246-b6b7-f26966b40cd7}
```

0.1 Check the host

Why: the operator lab needs the same memory and CPU as the manual lab, about 48 GB, so only one lab can run at a time. On disk it needs its own room: three fixed ASM disks (110 GB together) are written in full straight away, and the workers' disks grow as they fill.

PowerShell

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
Get-CimInstance Win32_Processor | Select-Object NumberOfCores, NumberOfLogicalProcessors
[Math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory / 1GB)
Get-PSDrive D | Select-Object @{n='FreeGB';e={[math]::Round($_.Free/1GB)}}
```

Check:

- `list runningvms` prints **nothing**: the manual lab and the golden VM are off;
- 8 or more logical processors and about 64 GB of RAM;
- **300 GB or more free on D:**. Less than about 150 GB stops you at 3.3, when the ASM disks are created.

This lab's output:

Output what this lab printed; nothing to run

```
PS> & $VB list runningvms
                                    (nothing: no VM running)
NumberOfCores NumberOfLogicalProcessors
------------- -------------------------
            8                         8
64


FreeGB
------
  1166
```

0.2 SSH key and lab SSH settings (already done)

The manual lab's Phase 0.2 set these up, and they cover every port on `127.0.0.1`, including 2220–2222. This block only checks them; it changes nothing that is already there.

PowerShell

```powershell
if (-not (Test-Path "$env:USERPROFILE\.ssh\id_ed25519")) { ssh-keygen -t ed25519 }   # press Enter at every prompt
Select-String -Path "$env:USERPROFILE\.ssh\config" -Pattern '^Host 127\.0\.0\.1$' -Context 0,5
```

Check: the `Host 127.0.0.1` section with its five settings (`User root`, `StrictHostKeyChecking no`, `UserKnownHostsFile ~/.ssh/known_hosts_rac_lab`, `LogLevel ERROR`, `ServerAliveInterval 30`). If nothing prints, run the manual guide's 0.2 block once.

This lab's output:

Output what this lab printed; nothing to run

```
> C:\Users\alpoo\.ssh\config:2:Host 127.0.0.1
  C:\Users\alpoo\.ssh\config:3:    User root
  C:\Users\alpoo\.ssh\config:4:    StrictHostKeyChecking no
  C:\Users\alpoo\.ssh\config:5:    UserKnownHostsFile ~/.ssh/known_hosts_rac_lab
  C:\Users\alpoo\.ssh\config:6:    LogLevel ERROR
  C:\Users\alpoo\.ssh\config:7:    ServerAliveInterval 30
```

0.3 Port forwards for the operator lab

What: three SSH forwards on the existing `k8s-public` network: 2220 → op-controlplane, 2221 → op-worker1, 2222 → op-worker2. The golden VM's forward, 2205 → 10.0.1.5, is already there and is reused for personalising the clones in Phase 4.

Why on the same network: the two labs never run at the same time, so they can share `k8s-public` and its NAT. Different IPs (.20–.22) and different ports keep them apart; the rule names start with `ssh-op` so 0.0 can remove exactly these.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opcp:tcp:[]:2220:[10.0.1.20]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opw1:tcp:[]:2221:[10.0.1.21]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-opw2:tcp:[]:2222:[10.0.1.22]:22"
& $VB natnetwork list
```

Check: `natnetwork list` shows `k8s-public` with the manual lab's rules (`ssh-golden` , `ssh-cp` , `ssh-w1` , `ssh-w2` , and any NodePort rules) **plus** `ssh-opcp` , `ssh-opw1` and `ssh-opw2` . If a line says the rule already exists, it was added before; that's fine.

This lab's output:

Output what this lab printed; nothing to run

```
Name:         k8s-public
Enabled:      Yes
Network:      10.0.1.0/24
Gateway:      10.0.1.1
DHCP Server:  No
…
Port-forwarding (ipv4)
        k8sapi:tcp:[]:6443:[10.0.1.10]:6443
        np30200:tcp:[]:30200:[10.0.1.11]:30200
        np30203:tcp:[]:30203:[10.0.1.11]:30203
        np30204:tcp:[]:30204:[10.0.1.11]:30204
        ssh-cp:tcp:[]:2210:[10.0.1.10]:22
        ssh-golden:tcp:[]:2205:[10.0.1.5]:22
        ssh-opcp:tcp:[]:2220:[10.0.1.20]:22
        ssh-opw1:tcp:[]:2221:[10.0.1.21]:22
        ssh-opw2:tcp:[]:2222:[10.0.1.22]:22
        ssh-w1:tcp:[]:2211:[10.0.1.11]:22
        ssh-w2:tcp:[]:2212:[10.0.1.12]:22
```

PHASE 3

Clone the three operator-lab nodes

Result of this phase: three VMs (`op-controlplane` , `op-worker1` , `op-worker2`) cloned from the same golden image as the manual lab, the workers with two data disks each and their private NICs on `op-priv1` / `op-priv2` , and three new disks that **both** workers share for ASM: 10 GB for `VOTEDG` , 50 GB each for `DATA` and `FRA` .

3.1 Check that the golden VM is still sealed

Why: clones must come from the **sealed** golden image: no machine ID, no SSH host keys. You sealed it before cloning the manual lab. If the golden VM has been started since then, it has made itself a new machine ID and host keys, and every clone would share them. Kubernetes refuses to join nodes with the same machine ID.

How to tell without starting it: VirtualBox records when a VM last changed state. If the golden VM's last change (its power-off after sealing) is **older** than the manual lab's clones, nobody has started it since.

PowerShell

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $g = "ol98-golden"
$info = & $VB showvminfo $g --machinereadable
$state = ($info | Select-String '^VMState="(.*)"').Matches[0].Groups[1].Value
$raw = ($info | Select-String '^VMStateChangeTime="(.*)"').Matches[0].Groups[1].Value
$changed = [datetime]::SpecifyKind([datetime]::ParseExact($raw.Substring(0,19), 'yyyy-MM-ddTHH:mm:ss',
$null), 'Utc')   # VirtualBox writes UTC
$cpCfg = (& $VB showvminfo controlplane --machinereadable | Select-String '^CfgFile="
(.*)"').Matches[0].Groups[1].Value
$cloned = (Get-Item (Split-Path $cpCfg)).CreationTimeUtc   # when the manual lab's controlplane was
cloned
"manual lab folder:    $(Split-Path $cpCfg)"
"golden state:         $state"
"golden last changed:  $($changed.ToString('u'))"
"manual lab cloned:    $($cloned.ToString('u'))"
if ($state -eq 'poweroff' -and $changed -lt $cloned) { "OK: still sealed, not started since the manual
lab was cloned" } else { "STOP: golden VM was started after the seal; reseal it (below)" }
& $VB snapshot $g list
```

Check: the line says `OK: still sealed…`, and the snapshot list shows the `pre-seal` snapshot. Then go straight to 3.2. The block finds the manual lab's folder from VirtualBox itself, wherever it is on your disk.

This lab's output:

Output what this lab printed; nothing to run

```
manual lab folder:    D:\\VMs\\rac-poc\\k8s-rac-manual\\controlplane\
golden state:         poweroff
golden last changed:  2026-10-25 12:08:16Z
manual lab cloned:    2026-10-25 12:08:31Z
OK: still sealed, not started since the manual lab was cloned
   Name: pre-seal (UUID: 50fb527d-13b9-4e6b-94bb-49208269b5ff) *
   Description: Prepared golden image before sealing
```

This lab's output (the first version of the check, before it was fixed: it assumed the manual lab was in D:\VMs\rac-poc\controlplane, got no clone time and gave a false STOP. The current block asks VirtualBox for the folder instead):

Output what this lab printed; nothing to run

```
$cloned = (Get-Item "D:\VMs\rac-poc\controlplane").CreationTimeUtc
Get-Item : Cannot find path 'D:\VMs\rac-poc\controlplane' because it does not exist.
golden state:         poweroff
golden last changed:  2026-10-25 12:08:16Z
manual lab cloned:
STOP: golden VM was started after the seal; reseal it (below)
```

Only if it says STOP: seal it again, in place. Don't restore the `pre-seal` snapshot: fixes made to the golden image after that snapshot would be lost. Starting the golden VM gives it a new machine ID and host keys for a

moment; the seal block removes them again and powers it off.

PowerShell (only after a STOP in the check above)

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm ol98-golden --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
```

golden VM

(the last command powers it off; don't start it again before cloning)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf clean all
rm -f /etc/ssh/ssh_host_*
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
rm -rf /tmp/* /var/tmp/*
journalctl --rotate; journalctl --vacuum-time=1s
rm -f /root/.bash_history; history -c
poweroff
```

If SSH warns that the host key for port 2205 has changed, run `ssh-keygen -R "[127.0.0.1]:2205"` in PowerShell and connect again.

3.2 Clone and size the nodes

Why these sizes: the same as the manual lab. op-controlplane gets 2 vCPU and 4 GB with no private NICs; each worker gets 4 vCPU and 22 GB, an 80 GB disk for `/scratch` and a 100 GB disk for `/var/lib/containers`.

What's different: the workers' second and third NICs move from the manual lab's internal networks `rac-priv1` / `rac-priv2` to the operator lab's `op-priv1` / `op-priv2`. The clones inherit the golden VM's NIC settings, including the "Allow All" promiscuous mode Multus needs; only the network names change.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"; $g = "ol98-golden"
while ("$(& $VB showvminfo $g --machinereadable | Select-String '^VMState=')" -notmatch 'poweroff') {
Start-Sleep 3 }
foreach ($n in "op-controlplane","op-worker1","op-worker2") { & $VB clonevm $g --name $n --basefolder
$Base --mode machine --register }
& $VB modifyvm op-controlplane --cpus 2 --memory 4096 --nic2 none --nic3 none
foreach ($w in "op-worker1","op-worker2") {
  & $VB modifyvm $w --cpus 4 --memory 22528 --intnet2 op-priv1 --intnet3 op-priv2
  & $VB createmedium disk --filename "$Base\$w\$w-scratch.vdi" --size 81920 --variant Standard
  & $VB createmedium disk --filename "$Base\$w\$w-containers.vdi" --size 102400 --variant Standard
  & $VB storageattach $w --storagectl SATA --port 1 --device 0 --type hdd --medium "$Base\$w\$w-
scratch.vdi"
  & $VB storageattach $w --storagectl SATA --port 2 --device 0 --type hdd --medium "$Base\$w\$w-
containers.vdi"
}
foreach ($n in "op-controlplane","op-worker1","op-worker2") { "--- $n"; & $VB showvminfo $n --
machinereadable | Select-String '^(cpus|memory|nic[123]|nat-network1|intnet[23])='; & $VB showvminfo $n |
Select-String '^NIC [23]:' }
```

Check:

- **op-controlplane:** `cpus=2`, `memory=4096`, `nic1="natnetwork"` with `nat-network1="k8s-public"`, and `nic2="none"`, `nic3="none"`;
- **each worker:** `cpus=4`, `memory=22528`, `nat-network1="k8s-public"`, `intnet2="op-priv1"`, `intnet3="op-priv2"`, and two `NIC 2:` / `NIC 3:` lines that include `Promisc Policy: allow-all`, each with its own MAC address. (The machine-readable output has no promiscuous-mode line on this VirtualBox version, so the check reads it from the normal output.)

This lab's output:

Output what this lab printed; nothing to run

```
Machine has been successfully cloned as "op-controlplane"
Machine has been successfully cloned as "op-worker1"
Machine has been successfully cloned as "op-worker2"
Medium created. UUID: a6d01e0b-…        (op-worker1 scratch)
Medium created. UUID: cb7bdfad-…        (op-worker1 containers)
Medium created. UUID: 2d09717f-…        (op-worker2 scratch)
Medium created. UUID: b4ca77a0-…        (op-worker2 containers)
--- op-controlplane
memory=4096
cpus=2
nat-network1="k8s-public"
nic1="natnetwork"
nic2="none"
nic3="none"
--- op-worker1
memory=22528
cpus=4
nat-network1="k8s-public"
nic1="natnetwork"
intnet2="op-priv1"
nic2="intnet"
intnet3="op-priv2"
nic3="intnet"
--- op-worker2
(same as op-worker1)
--- op-worker1
NIC 2:   MAC: 080027C33702, Attachment: Internal Network 'op-priv1', … Promisc Policy: allow-all, …
NIC 3:   MAC: 08002743BD30, Attachment: Internal Network 'op-priv2', … Promisc Policy: allow-all, …
--- op-worker2
NIC 2:   MAC: 0800279D8872, Attachment: Internal Network 'op-priv1', … Promisc Policy: allow-all, …
NIC 3:   MAC: 0800279E4885, Attachment: Internal Network 'op-priv2', … Promisc Policy: allow-all, …
```

3.3 The shared ASM disks

What: three disks attached to **both** workers at once, one per ASM disk group:

DISK	SIZE	SATA PORT	DISK GROUP	HOLDS
`vote1.vdi`	10 GB	5	`VOTEDG`	the cluster's own files: voting file, OCR, the ASM password file and spfile
`asm1.vdi`	50 GB	3	`DATA`	the database's data files, control files and redo logs
`asm2.vdi`	50 GB	4	`FRA`	the fast recovery area: archived logs, flashback logs, backups

Why three disk groups instead of one: keeping the cluster's files apart from the database makes each easier to size, move and repair; a full `FRA` can't stop the cluster, and the clusterware disk group stays small. The manual lab put everything into one `DATA` disk group.

Why port 5: the SATA controller has six ports (0–5). Port 5 carries the golden VM's empty DVD drive, which the clones inherited and nothing uses; the block removes it and puts `vote1` there.

Why fixed size and `shareable`: RAC is a shared-disk database, so both nodes need the same block devices; VirtualBox only allows the shareable type on fixed-size disks, and without it refuses to attach the second VM. These are **new, empty** disks in `D:\VMs\rac-op\shared`; the manual lab's disks stay with the manual lab. Don't snapshot the workers: VirtualBox can't snapshot a VM with shareable disks. Creating the fixed disks takes a few minutes.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
New-Item -ItemType Directory -Force -Path "$Base\shared" | Out-Null
foreach ($w in "op-worker1","op-worker2") { & $VB storageattach $w --storagectl SATA --port 5 --device 0
--medium none }
foreach ($disk in @(@{n="asm1";gb=50;port=3}, @{n="asm2";gb=50;port=4}, @{n="vote1";gb=10;port=5})) {
  $f = "$Base\shared\$($disk.n).vdi"
  & $VB createmedium disk --filename $f --size ($disk.gb*1024) --variant Fixed
  & $VB modifymedium disk $f --type shareable
  foreach ($w in "op-worker1","op-worker2") { & $VB storageattach $w --storagectl SATA --port $disk.port
--device 0 --type hdd --medium $f --mtype shareable }
}
foreach ($d in "vote1","asm1","asm2") { & $VB showmediuminfo disk "$Base\shared\$d.vdi" | Select-String
'^(UUID|Location|Type|Capacity|In use by VMs):|op-worker' }
```

Check: for **each** of the three disks, `Type: shareable`, a `Location` under `D:\VMs\rac-op\shared`, the right `Capacity` (10240 MBytes for `vote1`, 51200 for the others), and both `op-worker1` and `op-worker2` under "In use by VMs".

This lab's output:

Output what this lab printed; nothing to run

```
Medium created. UUID: 74717f76-da51-4430-87ce-3b192212f46e      (asm1)
Medium created. UUID: 204551ec-9ae3-4871-b2b9-8c68f5569e80      (asm2)
Medium created. UUID: 67cb0647-d253-4702-8823-e8628f6c6980      (vote1)

UUID:           67cb0647-d253-4702-8823-e8628f6c6980
Type:           shareable
Capacity:       10240 MBytes
In use by VMs:  op-worker1 (UUID: 8164c82b-4b03-49a4-b193-671b310c6962)
                op-worker2 (UUID: 05c9cfea-2eba-4f10-8c54-9c1b2919b592)
UUID:           74717f76-da51-4430-87ce-3b192212f46e
Type:           shareable
Location:       D:\VMs\rac-op\shared\asm1.vdi
In use by VMs:  op-worker1 (UUID: 8164c82b-…)
                op-worker2 (UUID: 05c9cfea-…)
UUID:           204551ec-9ae3-4871-b2b9-8c68f5569e80
Type:           shareable
Location:       D:\VMs\rac-op\shared\asm2.vdi
In use by VMs:  op-worker1 (UUID: 8164c82b-…)
                op-worker2 (UUID: 05c9cfea-…)
```

In this lab `asm1` and `asm2` were created first, in an earlier version of this block, and `vote1` was added later to the running lab (below). The UUIDs are what 10.1 uses to find each disk inside the workers.

Adding vote1 to a lab built with only asm1 and asm2 (what this lab did). Nothing may have been applied yet (`kubectl -n rac get racdatabase` says `No resources found`); the workers are powered off for the change.

PowerShell

```powershell
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"; $f =
"$Base\shared\vote1.vdi"
foreach ($w in "op-worker1","op-worker2") { & $VB controlvm $w acpipowerbutton }
foreach ($w in "op-worker1","op-worker2") { while ("$(& $VB showvminfo $w --machinereadable | Select-
String '^VMState=')" -notmatch 'poweroff') { Start-Sleep 5 }; "$w is off" }
& $VB createmedium disk --filename $f --size 10240 --variant Fixed
& $VB modifymedium disk $f --type shareable
foreach ($w in "op-worker1","op-worker2") {
  & $VB storageattach $w --storagectl SATA --port 5 --device 0 --medium none
  & $VB storageattach $w --storagectl SATA --port 5 --device 0 --type hdd --medium $f --mtype shareable
}
& $VB showmediuminfo disk $f | Select-String '^(UUID|Type|Capacity|In use by VMs):|op-worker'
foreach ($w in "op-worker1","op-worker2") { & $VB startvm $w --type headless }
```

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl wait --for=condition=Ready node/op-worker1 node/op-worker2 --timeout=600s
kubectl -n oracle-database-operator-system get pods
kubectl -n kube-system get pods -o wide | grep -E 'multus|flannel|calico' | grep -v Running
```

Check: `Type: shareable`, `Capacity: 10240 MBytes`, both workers under "In use by VMs"; both nodes `condition met`, the operator pod `Running`, and nothing printed by the last line.

This lab's output:

Output what this lab printed; nothing to run

```
op-worker1 is off
op-worker2 is off
Medium created. UUID: 67cb0647-d253-4702-8823-e8628f6c6980
UUID:           67cb0647-d253-4702-8823-e8628f6c6980
Type:           shareable
Capacity:       10240 MBytes
In use by VMs:  op-worker1 (UUID: 8164c82b-4b03-49a4-b193-671b310c6962)
                op-worker2 (UUID: 05c9cfea-2eba-4f10-8c54-9c1b2919b592)
VM "op-worker1" has been successfully started.
VM "op-worker2" has been successfully started.
node/op-worker1 condition met
node/op-worker2 condition met
NAME                                                        READY   STATUS    RESTARTS   AGE
oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running   1          12h
```

The operator pod shows `RESTARTS 1` because it ran on a worker that was just restarted.

PHASE 4

Personalize each node

Result of this phase: each node has its own name and IP address. The workers also have their private NICs ready for Multus, `/scratch` and `/var/lib/containers` on their own disks, the operator's directories, and 4 GB of hugepages.

Do one node at a time, in the order op-controlplane, op-worker1, op-worker2. Every clone boots with the golden VM's address 10.0.1.5, and two clones running at once would fight over it. Each node has its own section below with every command written out for it: copy the blocks in order, without changing anything. Start the next node only after the previous one passes its last check.

4.1 What happens to each node, and why

STEP	OP-CONTROLPLANE	OP-WORKER1, OP-WORKER2
a. Start and connect on 2205	yes	yes
b. Name, identity, kubelet IP	yes	yes
c. Private NICs for Multus	no: it has no NIC2/NIC3	yes
d, e. Data disks `/var/lib/containers` and `/scratch`	no: it has only the OS disk	yes
f. The operator's directories	no	yes (the same on both workers)
g. HugePages	no: 2048 pages would lock all of its 4 GB	yes
Move to its own address, then check it	yes	yes

Why each step:

- **Name, identity and kubelet IP:** the host name becomes the Kubernetes node name. New SSH host keys replace the ones the seal removed. `--node-ip` matters on the workers, which have three NICs: without it the kubelet may advertise the wrong address, and pod traffic between nodes breaks.
- **Private NICs with no IP:** `enp0s8` and `enp0s9` are only "parents" for Multus macvlan interfaces. Each RAC pod gets its own interface, IP (10.0.11.x and 10.0.12.x) and MAC address on top of them.
- **Promiscuous mode (`accept-all-mac-addresses yes`):** this was the cause of the hardest network failure in the first build, 100% packet loss between pods. VirtualBox's "Allow All" setting only takes effect when the guest NIC itself is promiscuous. Otherwise the virtual NIC silently drops frames addressed to the pods' macvlan MACs.
- **Data disks chosen by size, not by name:** Linux disk names (`sdb`, `sdc` ...) do **not** follow VirtualBox port order. On worker2 in the manual lab's first build, `/dev/sdc` was a shared ASM disk, and formatting by name would have wiped the disk both nodes share. The ASM disks are 50 GB, so the sizes tell the disks apart. They're **mounted by UUID** for the same reason, and `rsync -aAXH` copies the images pulled in Phase 2 together with their SELinux labels.
- **The operator's directories with `container_file_t`:** the operator mounts them into the RAC pods as hostPath volumes; their names follow its `racHostSwLocation` (`/scratch/rac/cluster01`) and `racNodeName` (`racnode`) settings. With SELinux enforcing, a container can only use host files with the container file label. `semanage fcontext` makes the label permanent.
- **HugePages:** the database SGA goes into 2 MB hugepages, which are locked, never swapped and cheaper for the CPU to map. Kubernetes only gives hugepages to pods on nodes that reserved them: the RAC pods ask for `hugepages-2Mi: 4Gi`, so each worker reserves 2048 pages. The doc reserves 16384 (32 GB); this lab scales it to the 22 GB workers.
- **Moving the address last, in the background:** changing the public IP cuts the SSH session you're typing in. A first attempt died halfway through for exactly that reason. So the change runs detached from your session and ends with a reboot.
- **`ssh-keygen -R` before connecting:** every clone has its own host key but answers on the same port, 2205. Without the 0.2 SSH settings, SSH refuses the new key with "REMOTE HOST IDENTIFICATION HAS CHANGED"; removing the old entry avoids that.

4.2 op-controlplane

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-controlplane --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

op-controlplane (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-controlplane
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.20"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-controlplane`, `--node-ip=10.0.1.20`, and a machine ID and product UUID different from the other nodes'.

c. Move to 10.0.1.20 and reboot (always last):

op-controlplane (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-controlplane" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.20/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-controlplane; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.20 and rebooting.

d. Check it on its own address (wait about a minute after the move):

PowerShell

```
ssh -p 2220 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-controlplane`, and `10.0.1.20/24`;
- `Device "enp0s8" does not exist` and `df: /scratch: No such file or directory`: correct, it has no private NICs and no data disks;
- `/var/lib/containers` is on the root filesystem (`/dev/mapper/ol-root`);
- `HugePages_Total: 0`, and CRI-O is `active`.

4.3 op-worker1

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-worker1 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

op-worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-worker1
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.21"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-worker1`, `--node-ip=10.0.1.21`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

`op-worker1` (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The operator's directories, with SELinux labels: `/scratch/rac/cluster01/racnode1` and `/scratch/rac/cluster01/racnode2` (the operator mounts one of them into each RAC pod as `/u01`), and `/scratch/software/stage` (the Oracle zips). **Both workers get both node directories:** the operator may place either RAC pod on either labelled worker.

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for d in /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; do
mkdir -p $d; done
chown -R 54321:54321 /scratch/rac/cluster01
semanage fcontext -a -t container_file_t "/scratch/rac/cluster01(/.*)?"
semanage fcontext -a -t container_file_t "/scratch/software/stage(/.*)?"
restorecon -RF /scratch/rac/cluster01 /scratch/software/stage
ls -Zd /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; ls -ld
/scratch/rac/cluster01/racnode*
```

Check: all three directories show `container_file_t`, and both `racnode*` directories are owned by `54321 54321`, shown as `oracle oinstall` if the host has those users: the IDs of `oracle` and `oinstall` inside the RAC image.

g. HugePages:

`op-worker1` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.21 and reboot (always last):

`op-worker1` (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-worker1" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.21/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-worker1; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.21 and rebooting.

i. Check it on its own address (wait about a minute after the move):

```powershell
ssh -p 2221 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-worker1`, and `10.0.1.21/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

4.4 op-worker2

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

```powershell
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm op-worker2 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

`op-worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname op-worker2
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.22"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `op-worker2`, `--node-ip=10.0.1.22`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

op-worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

op-worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

op-worker2 (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The operator's directories, with SELinux labels: `/scratch/rac/cluster01/racnode1` and `/scratch/rac/cluster01/racnode2` (the operator mounts one of them into each RAC pod as `/u01`), and `/scratch/software/stage` (the Oracle zips). **Both workers get both node directories:** the operator may place either RAC pod on either labelled worker.

`op-worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for d in /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; do
mkdir -p $d; done
chown -R 54321:54321 /scratch/rac/cluster01
semanage fcontext -a -t container_file_t "/scratch/rac/cluster01(/.*)?"
semanage fcontext -a -t container_file_t "/scratch/software/stage(/.*)?"
restorecon -RF /scratch/rac/cluster01 /scratch/software/stage
ls -Zd /scratch/rac/cluster01/racnode1 /scratch/rac/cluster01/racnode2 /scratch/software/stage; ls -ld
/scratch/rac/cluster01/racnode*
```

Check: all three directories show `container_file_t`, and both `racnode*` directories are owned by `54321 54321`, shown as `oracle oinstall` if the host has those users: the IDs of `oracle` and `oinstall` inside the RAC image.

g. HugePages:

`op-worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.22 and reboot (always last):

`op-worker2` (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "op-worker2" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.22/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not op-worker2; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.22 and rebooting.

i. Check it on its own address (wait about a minute after the move):

```powershell
ssh -p 2222 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch /var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `op-worker2`, and `10.0.1.22/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

When op-worker2 passes its check, all three nodes are on their own addresses. Go on to Phase 5.

PHASE 5

Kubernetes with kubeadm and Flannel

Result of this phase: a 3-node Kubernetes v1.36 cluster `rac-op`: the op-controlplane plus op-worker1 and op-worker2, all `Ready`, with Flannel as the pod network, and both workers labelled for the operator.

5.1 The kubeadm configuration (op-controlplane)

Blocks labelled `# op-controlplane` run in a root shell on the op-controlplane. From here on, keep one PowerShell window connected to it:

```powershell
ssh -p 2220 root@127.0.0.1
```

Check: the prompt says `[root@op-controlplane ~]#`. The same goes for `# op-worker1` (port 2221) and `# op-worker2` (port 2222) when a block needs them.

Why each part:

- `advertiseAddress` / `controlPlaneEndpoint` **10.0.1.20:** the API server listens on the public network, not on any other NIC.
- `criSocket` **CRI-O:** the runtime chosen in Phase 2.
- `podSubnet` **10.244.0.0/16:** Flannel's default. The doc's Grid response file names `eth0:10.244.0.0` as the public network inside the pods, so this range has to match.
- `dnsDomain` **cluster.local:** the SCAN name becomes `racnode-scan.rac.svc.cluster.local`.
- `allowedUnsafeSysctls`: the RAC pods set `kernel.shmall`, `kernel.shmmax`, `kernel.shmmni`, `kernel.sem` and some `net.*` settings for themselves. Kubernetes calls these "unsafe" and the kubelet rejects any pod that sets them, unless they're allowed here. These settings are namespaced: they affect only that pod, not

the node. The OCNE doc sets this as a kubelet flag; with kubeadm it belongs in the `KubeletConfiguration`, which every node picks up when it joins.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cat >/root/kubeadm-config.yaml <<'EOF'
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 10.0.1.20
  bindPort: 6443
nodeRegistration:
  name: op-controlplane
  criSocket: unix:///var/run/crio/crio.sock
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
clusterName: rac-op
controlPlaneEndpoint: 10.0.1.20:6443
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
  dnsDomain: cluster.local
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
allowedUnsafeSysctls:
  - "kernel.shm*"
  - "kernel.sem"
  - "net.*"
EOF
```

5.2 Initialise the control plane (op-controlplane)

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubeadm init --config /root/kubeadm-config.yaml
mkdir -p ~/.kube && cp -f /etc/kubernetes/admin.conf ~/.kube/config
kubectl get nodes
```

Check: `Your Kubernetes control-plane has initialized successfully!`. The op-controlplane shows `NotReady` until there's a pod network in 5.3, which is expected.

5.3 Flannel, the pod network (op-controlplane)

What: Flannel gives every pod an `eth0` address in 10.244.0.0/16, and routes pod traffic between nodes over the 10.0.1.x network.

Why: Kubernetes needs a CNI network before any pod can start. In the RAC pods, `eth0` is the **public** network: the node names, VIPs and SCAN all resolve to `eth0` addresses.

This URL always fetches the newest Flannel release. The lab used the release current on 23 Oct 2026. If a newer one misbehaves, pick a specific version from Flannel's releases page instead of `latest`.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl -n kube-flannel rollout status ds/kube-flannel-ds
kubectl get nodes
```

Check: the op-controlplane turns `Ready` within a minute.

5.4 Join the workers

What: a join command with a fresh token, run on each worker. The PC fetches it from the op-controlplane and runs it on both workers, so there's nothing to copy by hand. The token in it expires after 24 hours.

PowerShell

```
$join = ssh -p 2220 root@127.0.0.1 "kubeadm token create --print-join-command"; $join
ssh -p 2221 root@127.0.0.1 $join
ssh -p 2222 root@127.0.0.1 $join
```

Each worker ends with `This node has joined the cluster`.

Back on the op-controlplane, label the workers and wait for them. The second label, `raccluster=raccluster01`, is the operator's: its `workerNodeSelector` places RAC pods only on nodes with this label.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl label node op-worker1 op-worker2 node-role.kubernetes.io/worker= --overwrite
kubectl label node op-worker1 op-worker2 raccluster=raccluster01 --overwrite
kubectl wait --for=condition=Ready node --all --timeout=600s
kubectl get nodes -o wide
kubectl get pods -A
```

Check:

- three nodes, all `Ready` and on `v1.36.x`, with INTERNAL-IP 10.0.1.20, .21 and .22;
- every system pod `Running`: coredns, etcd, kube-apiserver, kube-proxy, kube-flannel;
- `kubectl get nodes -l raccluster=raccluster01` lists op-worker1 and op-worker2.

If a worker shows the wrong INTERNAL-IP, the `--node-ip` setting from Phase 4 is missing.

5.5 kubectl from your PC (optional)

Why: so you can run `kubectl` from Git Bash on Windows without SSH-ing into op-controlplane first. The manual lab may already use `~/.kube/config` and host port 6443, so the operator lab gets **its own file**, `~/.kube/config-op`, and **its own port, 6444**. The two never overwrite each other.

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" natnetwork modify --netname k8s-public --port-forward-4 "k8sapi-op:tcp:[]:6444:[10.0.1.20]:6443"
```

Git Bash

```
mkdir -p ~/.kube
scp -P 2220 root@127.0.0.1:/etc/kubernetes/admin.conf ~/.kube/config-op
kubectl --kubeconfig "$(cygpath -m ~/.kube/config-op)" config set-cluster rac-op --server=https://127.0.0.1:6444 --tls-server-name=kubernetes
sed -i '/alias kop=/d' ~/.bashrc
echo "alias kop='kubectl --kubeconfig \"\$(cygpath -m ~/.kube/config-op)\"'" >> ~/.bashrc
grep -q MSYS_NO_PATHCONV ~/.bashrc || echo 'export MSYS_NO_PATHCONV=1' >> ~/.bashrc
source ~/.bashrc
kop get nodes
```

Why each line:

- `k8sapi-op` **on 6444:** the manual lab's forward uses host port 6443. A second rule can't use the same host port, so the operator lab's API server is reached on 6444; inside the lab it still listens on 6443.
- `--kubeconfig ~/.kube/config-op` **:** keeps the operator lab's credentials apart from the manual lab's `~/.kube/config`.
- `set-cluster rac-op` **:** kubeadm names the cluster entry after `clusterName` (`rac-op`), so that's the entry to point at 127.0.0.1:6444.
- `--tls-server-name=kubernetes` **:** the API server's certificate is for `kubernetes` and 10.0.1.20, not 127.0.0.1; this keeps certificate checking on.
- `kop` **:** short for `kubectl --kubeconfig C:/Users/<you>/.kube/config-op`. Use `kop` for the operator lab and plain `kubectl` for the manual lab. `cygpath -m` turns the Git Bash path into a Windows one, because `kubectl.exe` is a Windows program and `MSYS_NO_PATHCONV=1` stops Git Bash from converting `/c/Users/...` for it. Without that, kubectl can't find the file and falls back to `localhost:8080`.

Check: `kop get nodes` lists `op-controlplane`, `op-worker1` and `op-worker2`. `admin.conf` gives full control of the cluster; keep it out of anything you share.

This lab's output:

Output what this lab printed; nothing to run

```
admin.conf                                            100% 5617     8.8KB/s   00:00
Cluster "rac-op" set.
kop is aliased to `kubectl --kubeconfig "$(cygpath -m ~/.kube/config-op)"'
    server: https://127.0.0.1:6444
    tls-server-name: kubernetes
NAME              STATUS   ROLES           AGE     VERSION
op-controlplane   Ready    control-plane   9m54s   v1.36.5
op-worker1        Ready    worker          8m33s   v1.36.5
op-worker2        Ready    worker          8m25s   v1.36.5
```

This lab's output (the first version of the alias, without cygpath -m: kubectl couldn't find the file and fell back to localhost:8080; it also wrote a stray D:\c\Users\alpoo.kube\config-op, which was deleted):

Output what this lab printed; nothing to run

```
E1026 23:07:45 … couldn't get current server API group list: Get "http://localhost:8080/api?timeout=32s":
dial tcp [::1]:8080: connectex: No connection could be made …
Unable to connect to the server: dial tcp [::1]:8080: …
```

The same in PowerShell: a `kop` function in your PowerShell profile. PowerShell passes Windows paths, so no conversion is needed.

PowerShell

```
if (-not (Test-Path $PROFILE)) { New-Item -ItemType File -Force -Path $PROFILE | Out-Null }
if (-not (Select-String -Path $PROFILE -Pattern 'function kop' -Quiet)) {
  Add-Content -Path $PROFILE -Value 'function kop { kubectl --kubeconfig "$env:USERPROFILE\.kube\config-op" @args }'
}
. $PROFILE
kop get nodes
```

Check: the three `op-*` nodes. If `. $PROFILE` says running scripts is disabled, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again. For a whole session on one lab, `$env:KUBECONFIG = "$env:USERPROFILE\.kube\config-op"` points plain `kubectl` at the operator lab until the window closes.

This lab's output:

Output what this lab printed; nothing to run

```
NAME              STATUS   ROLES           AGE   VERSION
op-controlplane   Ready    control-plane   12m   v1.36.5
op-worker1        Ready    worker          11m   v1.36.5
op-worker2        Ready    worker          11m   v1.36.5
```

PHASE 6

Multus and the RAC private networks

Result of this phase: pods can have two extra interfaces, `eth1` on 10.0.11.0/24 and `eth2` on 10.0.12.0/24, with addresses you choose. A test between the workers proves the interconnect works before any Oracle software is involved.

6.1 Why RAC needs Multus

A Kubernetes pod normally has exactly one network interface, `eth0`, from the cluster network (Flannel here). RAC needs more than that:

- **A public network:** client connections, the VIPs and SCAN. That's `eth0`.
- **A private interconnect:** dedicated, low-latency and not routed through the cluster network. Cache Fusion moves database blocks between the instances over it, and the cluster heartbeat runs on it. If the heartbeat is lost, Grid evicts a node.

Multus is a "meta" CNI plugin that attaches additional networks to a pod. Each extra network is described by a **NetworkAttachmentDefinition** (NAD). This lab uses `macvlan`, which gives the pod its own MAC address directly on the worker's `enp0s8` / `enp0s9`. Two networks give Grid's **HAIP** (highly available IP) two paths, so the interconnect survives losing one.

6.2 Install Multus (op-controlplane)

The thick-plugin variant runs a small daemon on every node. The URL follows Multus's main branch, as it did in the lab on 23 Oct 2026. For a repeatable build you can replace `master` with a release tag.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://raw.githubusercontent.com/k8snetworkplumbingwg/multus-cni/master/deployments/multus-daemonset-thick.yml
kubectl -n kube-system rollout status ds/kube-multus-ds --timeout=300s
```

Check: `daemon set "kube-multus-ds" successfully rolled out`.

This lab's output:

Output what this lab printed; nothing to run

```
customresourcedefinition.apiextensions.k8s.io/network-attachment-definitions.k8s.cni.cncf.io created
clusterrole.rbac.authorization.k8s.io/multus created
clusterrolebinding.rbac.authorization.k8s.io/multus created
serviceaccount/multus created
configmap/multus-daemon-config created
daemonset.apps/kube-multus-ds created
Waiting for daemon set "kube-multus-ds" rollout to finish: 0 of 3 updated pods are available...
Waiting for daemon set "kube-multus-ds" rollout to finish: 1 of 3 updated pods are available...
daemon set "kube-multus-ds" successfully rolled out
```

6.3 The two RAC networks (op-controlplane)

Why each setting:

- **Namespace** `rac` **:** the operator looks the NADs up in the same namespace as the RAC database it creates, and its samples use `rac` . The manual lab used a separate `multus-rac` namespace; this is the one difference from it.
- **Names** `macvlan-conf1` **/** `macvlan-conf2` **:** the names the operator's `privateIPDetails` will refer to.
- `master: enp0s8` **/** `enp0s9` **:** the worker NICs prepared in Phase 4. The doc uses `ens5` / `ens6` , its own NIC names.
- `mode: bridge` **:** pods on the same parent NIC can reach each other.
- `host-local` **IPAM with a range of 140–150:** the doc's subnets and ranges.
- `"capabilities": {"ips": true}` **:** lets each pod request a **specific** address (10.0.11.141 for racnode1-0, .142 for racnode2-0). Without it, `host-local` ignores the request and picks addresses per node on its own, so both workers can hand out the same IP. The doc's NADs don't have this line; this is a delta.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl create namespace rac
cat <<'EOF' | kubectl apply -f -
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf1
  namespace: rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s8", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.11.0/24", "rangeStart": "10.0.11.140", "rangeEnd":
"10.0.11.150" } }'
---
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf2
  namespace: rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s9", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.12.0/24", "rangeStart": "10.0.12.140", "rangeEnd":
"10.0.12.150" } }'
EOF
kubectl -n rac get network-attachment-definitions
```

This lab's output:

Output what this lab printed; nothing to run

```
namespace/rac created
networkattachmentdefinition.k8s.cni.cncf.io/macvlan-conf1 created
networkattachmentdefinition.k8s.cni.cncf.io/macvlan-conf2 created
NAME           AGE
macvlan-conf1  0s
macvlan-conf2  0s
```

6.4 Prove the interconnect (op-controlplane)

What: one throwaway pod on each worker, each with `eth1` and `eth2` at chosen addresses. Then pings across.

Why now: if the private network doesn't work, Grid's installer fails much later with a far less obvious error. In the first build, this test is what exposed the promiscuous-mode problem. `NET_RAW` is added because CRI-O drops it by default and `ping` needs it.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for n in 1 2; do o=$((147+n)); cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nettest-w$n
  namespace: rac
  annotations:
    k8s.v1.cni.cncf.io/networks: '[{"name":"macvlan-conf1","interface":"eth1","ips":["10.0.11.$o/24"]},
{"name":"macvlan-conf2","interface":"eth2","ips":["10.0.12.$o/24"]}]'
spec:
  nodeSelector: { kubernetes.io/hostname: op-worker$n }
  containers:
  - name: net
    image: docker.io/library/busybox:1.36
    command: ["sleep", "infinity"]
    securityContext: { capabilities: { add: ["NET_RAW"] } }
EOF
done
kubectl -n rac wait --for=condition=Ready pod/nettest-w1 pod/nettest-w2 --timeout=300s
kubectl -n rac exec nettest-w1 -- ip -4 addr
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.11.149
kubectl -n rac exec nettest-w1 -- ping -c3 10.0.12.149
kubectl -n rac exec nettest-w2 -- ping -c3 10.0.11.148
kubectl -n rac exec nettest-w2 -- ping -c3 10.0.12.148
```

Check:

- `nettest-w1` has `eth1` 10.0.11.148/24 and `eth2` 10.0.12.148/24;
- all four pings get 0% packet loss.

If you get 100% loss, check that `enp0s8` / `enp0s9` show `PROMISC` on both workers (Phase 4, step c on each worker), and that 3.2's check showed `Promisc Policy: allow-all` for both workers.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac delete pod nettest-w1 nettest-w2
```

This lab's output:

Output what this lab printed; nothing to run

```
pod/nettest-w1 created
pod/nettest-w2 created
pod/nettest-w1 condition met
pod/nettest-w2 condition met
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    inet 127.0.0.1/8 scope host lo
2: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1450 qdisc noqueue qlen 1000
    inet 10.244.1.2/24 brd 10.244.1.255 scope global eth0
3: eth1@eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue qlen 1000
    inet 10.0.11.148/24 brd 10.0.11.255 scope global eth1
4: eth2@eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue qlen 1000
    inet 10.0.12.148/24 brd 10.0.12.255 scope global eth2
PING 10.0.11.149 (10.0.11.149): 56 data bytes
64 bytes from 10.0.11.149: seq=0 ttl=64 time=0.855 ms
…
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.12.149 (10.0.12.149): 56 data bytes
…
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.11.148 (10.0.11.148): 56 data bytes
…
3 packets transmitted, 3 packets received, 0% packet loss
PING 10.0.12.148 (10.0.12.148): 56 data bytes
…
3 packets transmitted, 3 packets received, 0% packet loss
pod "nettest-w1" deleted from rac namespace
pod "nettest-w2" deleted from rac namespace
```

PHASE 7

Install Oracle's database operator

Result of this phase: cert-manager and the Oracle Database Operator running in the cluster, watching namespace `rac`, with the extra permissions the RAC controller needs: reading nodes and the Multus networks, and creating the ASM PersistentVolumes itself.

What the operator is: a controller that runs in its own namespace, `oracle-database-operator-system`. It adds new object types to Kubernetes, `RacDatabase` among them. When you later apply one `RacDatabase` YAML, the operator creates everything the manual lab built by hand in its Phases 7–12: the PVs, the headless Services, the StatefulSets, the Grid and database installs and the database.

7.1 Get the operator's manifests (op-controlplane)

Why a copy of the repository: the install files are generated for the namespaces the operator should watch, and the RAC permission files live in the repository. The lab used the version current on 26 Oct 2026 (release v2.2.0). The block prints the exact commit, so you can note it.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
command -v git >/dev/null || dnf install -y -q git
rm -rf /root/oraop && git clone -q --depth 1 https://github.com/oracle/oracle-database-operator.git /root/oraop
cd /root/oraop && git log -1 --format='commit %h, %cd'
ls oracle-database-operator-rbac.yaml oracle-database-operator-system.yaml scripts/generate-namespace-install.sh rbac docs/rac/rbac
```

Check: a commit line, and the files and two `rbac` folders listed without errors.

7.2 cert-manager (op-controlplane)

Why: the operator checks every `RacDatabase` you apply through a webhook, and Kubernetes only talks to webhooks over TLS. cert-manager issues and renews that certificate. The version is the one the operator's README names.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.20.2/cert-manager.yaml
kubectl -n cert-manager rollout status deploy/cert-manager deploy/cert-manager-webhook deploy/cert-manager-cainjector --timeout=300s
kubectl -n cert-manager get pods
```

Check: three `successfully rolled out` lines, and three cert-manager pods `Running` with `1/1`.

This lab's output:

Output what this lab printed; nothing to run

```
namespace/cert-manager created
customresourcedefinition.apiextensions.k8s.io/challenges.acme.cert-manager.io created
…                                  (about 45 more "created" lines)
deployment.apps/cert-manager-webhook created
mutatingwebhookconfiguration.admissionregistration.k8s.io/cert-manager-webhook created
validatingwebhookconfiguration.admissionregistration.k8s.io/cert-manager-webhook created
Waiting for deployment "cert-manager" rollout to finish: 0 out of 1 new replicas have been updated...
deployment "cert-manager" successfully rolled out
deployment "cert-manager-webhook" successfully rolled out
deployment "cert-manager-cainjector" successfully rolled out
NAME                                       READY   STATUS    RESTARTS   AGE
cert-manager-69c7fcbf78-vv5dd              1/1     Running   0          48s
cert-manager-cainjector-69f8c8cdbf-ptnn2   1/1     Running   0          48s
cert-manager-webhook-84fd89df64-bkjmm      1/1     Running   0          48s
```

7.3 The operator, watching namespace rac (op-controlplane)

Why namespace-scoped: the operator's recommended mode. It only acts in the namespaces it's told to watch, here its own and `rac`, where the Multus networks from 6.3 already are. The script writes the matching RBAC and sets `WATCH_NAMESPACE`, so the two can't disagree.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/oraop
scripts/generate-namespace-install.sh oracle-database-operator-system,rac
kubectl apply -f dist/install/oracle-database-operator-rbac.yaml
kubectl apply -f dist/install/oracle-database-operator-system.yaml
kubectl -n oracle-database-operator-system rollout status deploy/oracle-database-operator-controller-manager --timeout=600s
kubectl -n oracle-database-operator-system get pods -o wide
kubectl -n oracle-database-operator-system get deploy oracle-database-operator-controller-manager -o jsonpath='{range .spec.template.spec.containers[0].env[?(@.name=="WATCH_NAMESPACE")]}{.name}={.value}{"\n"}{end}'
```

Check:

- `successfully rolled out`, and the controller-manager pods `Running`;
- `WATCH_NAMESPACE=oracle-database-operator-system,rac`.

The first start pulls the operator image from `container-registry.oracle.com`, which can take a few minutes. If a pod stays in `ErrImagePull`, run `kubectl -n oracle-database-operator-system describe pod -l control-plane=controller-manager | tail -20` and paste the events.

This lab's output:

Output what this lab printed; nothing to run

```
Generated namespace-scoped install manifests:
  /root/oraop/dist/install/oracle-database-operator-rbac.yaml
  /root/oraop/dist/install/oracle-database-operator-system.yaml

WATCH_NAMESPACE=oracle-database-operator-system,rac
Operator namespace=oracle-database-operator-system
…
namespace/oracle-database-operator-system created
serviceaccount/oracle-database-operator-controller-manager created
…
customresourcedefinition.apiextensions.k8s.io/racdatabases.database.oracle.com created
…
deployment.apps/oracle-database-operator-controller-manager created
Waiting for deployment "oracle-database-operator-controller-manager" rollout to finish: 0 out of 3 new
replicas have been updated...
…
deployment "oracle-database-operator-controller-manager" successfully rolled out
NAME                                                  READY   STATUS    RESTARTS   AGE    IP
NODE
oracle-database-operator-controller-manager-54cf89dd45-9rjws   1/1     Running   0          101s   10.244.2.4
op-worker2
oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running   0          101s   10.244.1.5
op-worker1
oracle-database-operator-controller-manager-54cf89dd45-xl646   1/1     Running   0          101s   10.244.2.5
op-worker2
WATCH_NAMESPACE=oracle-database-operator-system,rac
```

7.4 The RAC controller's extra permissions (op-controlplane)

Why: the namespace-scoped install can only touch objects inside the watched namespaces. The RAC controller also needs a few cluster-wide ones:

FILE	LETS THE OPERATOR	NEEDED BECAUSE
`rbac/node-rbac.yaml`	read nodes	it picks workers by the `raccluster` label and builds NodePort connect strings
`rbac/persistent-volume-rbac.yaml`	read PersistentVolumes	it checks the ASM PVs
`docs/rac/rbac/pv-rbac.yaml`	create and delete PersistentVolumes	it creates the ASM PVs from the disk paths you list
`docs/rac/rbac/multus-rbac.yaml`	read NetworkAttachmentDefinitions	it attaches `macvlan-conf1` / `macvlan-conf2` to the RAC pods

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/oraop
kubectl apply -f rbac/node-rbac.yaml -f rbac/persistent-volume-rbac.yaml -f docs/rac/rbac/pv-rbac.yaml -f docs/rac/rbac/multus-rbac.yaml
SA=system:serviceaccount:oracle-database-operator-system:oracle-database-operator-controller-manager
for c in "list nodes" "create persistentvolumes" "delete persistentvolumes" "list network-attachment-definitions.k8s.cni.cncf.io -n rac" "create statefulsets.apps -n rac" "create racdatabases.database.oracle.com -n rac"; do
  printf '%-62s %s\n' "$c" "$(kubectl auth can-i $c --as=$SA)"
done
kubectl get crd racdatabases.database.oracle.com
```

Check: every line ends in `yes`, and the `racdatabases.database.oracle.com` CRD is listed. A `no` names the permission that's missing; paste the output.

This lab's output:

Output what this lab printed; nothing to run

```
clusterrole.rbac.authorization.k8s.io/oracle-database-operator-manager-role-node created
clusterrolebinding.rbac.authorization.k8s.io/oracle-database-operator-manager-role-node-cluster-role-binding created
clusterrole.rbac.authorization.k8s.io/oracle-database-operator-manager-role-persistent-volume created
clusterrolebinding.rbac.authorization.k8s.io/oracle-database-operator-manager-role-persistent-volume-cluster-role-binding created
clusterrole.rbac.authorization.k8s.io/oracle-operator-pv-role created
clusterrolebinding.rbac.authorization.k8s.io/oracle-operator-pv-rolebinding created
clusterrole.rbac.authorization.k8s.io/oracle-operator-multus-nad-role created
clusterrolebinding.rbac.authorization.k8s.io/oracle-operator-multus-nad-rolebinding created
Warning: resource 'nodes' is not namespace scoped
list nodes                                                  yes
Warning: resource 'persistentvolumes' is not namespace scoped
create persistentvolumes                                    yes
Warning: resource 'persistentvolumes' is not namespace scoped
delete persistentvolumes                                    yes
list network-attachment-definitions.k8s.cni.cncf.io -n rac  yes
create statefulsets.apps -n rac                             yes
create racdatabases.database.oracle.com -n rac              yes
NAME                              CREATED AT
racdatabases.database.oracle.com  2026-10-27T12:06:53Z
```

The `Warning: resource '…' is not namespace scoped` lines are normal: `kubectl auth can-i` notes that nodes and PersistentVolumes are cluster-wide objects.

PHASE 8

The RAC slim image and the software

Result of this phase: fully patched 19.33 gold-image zips of the Grid and DB homes (built once per RU on a separate host), Oracle's RAC slim image, `localhost/oracle/database-rac:19.3.0-slim`, on both workers, and the two 19.32 gold-image zips in `/scratch/software/stage` on both workers.

Why on both workers: the operator may start either RAC pod on either labelled worker, and each worker's CRI-O can only start images it has locally. The image never leaves the worker it was built on, so nothing needs a registry.

8.0 Build the 19.33 gold-image zips (on a separate build host, once per RU)

What: a fully patched, unconfigured Grid home and Oracle (DB) home at **19.33.0.0.261020**, the October 2026 Release Update, each packed into one zip with Oracle's `-createGoldImage`. The `RacDatabase` names these zips (`gridSwZipFile`, `dbSwZipFile`); each RAC pod unzips them and runs only the software-only installs with `-noCopy`, so **nothing is patched during the deployment**.

Why a separate host: patching needs a full 19.3 home, a new OPatch and a 2–3 GB Release Update, and the `grid` and `oracle` users, but no cluster, no shared disks and no Kubernetes. Doing it once on a plain Oracle Linux 9.8 server, `oldblab-db1`, keeps that work out of the RAC lab, and the two zips are reused for every RAC build at 19.33.

Which RU the lab runs: the RAC database built in Phases 10–11 reports `19.32.0.0.0` (11.5, 11.6c), from the 19.32 zips staged in 8.2. This section builds the **19.33** zips; for a build from them, the zip names in 8.2 and in the `RacDatabase` change to `oracle-gridhome-19.33.zip` and `oracle-dbhome-19.33.zip`.

The steps, in the order this run did them:

PART	HOME	WHAT
a	—	the build host: OS, users, the downloaded software
b	—	which patch is which, and unpacking the combo patch
c–f	Grid	unzip 19.3, new OPatch, apply OJVM RU, GI RU and JDK patch, software-only install, gold image
g–j	Oracle	the same for the DB home
k	both	name, test and checksum the zips

Why `-applyRU` before installing: `gridSetup.sh -applyRU` and `runInstaller -applyRU` patch the unzipped home first and then stop (there's no response file). The software-only install that follows then already runs the 19.33 installer, which knows Oracle Linux 9; the unpatched 19.3 installer doesn't, and would need `CV_ASSUME_DISTID`. In this run neither install needed it.

Why the software-only install before `-createGoldImage`: it relinks the binaries for this OS and registers the home in the inventory; `-createGoldImage` then packs the home as installed: patched, relinked, not configured (no cluster, no ASM, no database, no listener). The pod's own software-only install with `-noCopy` registers it again on each RAC node.

a. The build host

The host has the `oracle` and `grid` users with the standard 19c groups, and these directories, owned `oracle:oinstall` (DB side) and `grid:oinstall` (Grid side): `/u01/app/19.0.0/grid`, `/u01/app/grid`, `/u01/app/oraInventory`, `/u01/app/oracle/product/19.0.0/dbhome_1`, and `/u01/app/oracle/software` holding the downloads. (How the users and directories were created wasn't captured; the output below shows the result.) The home paths may differ from the RAC pods' (`/u01/app/19c/…`): a gold image unzips into any home path.

oldblab-db1 as labadmin (the build host)

```
cd
uname -a
cat /etc/os-release
id oracle
id grid
ls -altrh /u01/app/oracle/software/
```

Check: Oracle Linux 9.8 on UEK; `oracle` in `oinstall`, `dba`, `oper`, `backupdba`, `dgdba`, `kmdba`, `racdba`, `asmdba`; `grid` in `oinstall`, `racdba`, `asmdba`, `asmoper`, `asmadmin`; the two 19.3 base zips, the RUs, the JDK patch and OPatch (17 GB in all).

This lab's output:

Output what this lab printed; nothing to run

```
[labadmin@oldblab-db1 ~]$ uname -a
Linux oldblab-db1 6.12.0-207.111.5.1.el9uek.x86_64 #2 SMP PREEMPT_DYNAMIC Thu Oct  1 11:42:26 PDT 2026 x86_64
x86_64 x86_64 GNU/Linux
[labadmin@oldblab-db1 ~]$ cat /etc/os-release
NAME="Oracle Linux Server"
VERSION="9.8"
ID="ol"
ID_LIKE="fedora"
VARIANT="Server"
VARIANT_ID="server"
VERSION_ID="9.8"
PLATFORM_ID="platform:el9"
PRETTY_NAME="Oracle Linux Server 9.8"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:oracle:linux:9:8:server"
HOME_URL="https://linux.oracle.com/"
BUG_REPORT_URL="https://github.com/oracle/oracle-linux"

ORACLE_BUGZILLA_PRODUCT="Oracle Linux 9"
ORACLE_BUGZILLA_PRODUCT_VERSION=9.8
ORACLE_SUPPORT_PRODUCT="Oracle Linux"
ORACLE_SUPPORT_PRODUCT_VERSION=9.8
[labadmin@oldblab-db1 ~]$ id oracle
uid=54321(oracle) gid=54321(oinstall)
groups=54321(oinstall),54322(dba),54323(oper),54324(backupdba),54325(dgdba),54326(kmdba),54330(racdba),54327(
asmdba)
[labadmin@oldblab-db1 ~]$ id grid
uid=54331(grid) gid=54321(oinstall)
groups=54321(oinstall),54330(racdba),54327(asmdba),54328(asmoper),54329(asmadmin)
[labadmin@oldblab-db1 ~]$ ls -altrh /u01/app/oracle/software/
total 17G
-rwxrwxrwx. 1 labadmin labadmin 2.9G Oct 28 12:00 LINUX.X64_193000_db_home.zip
-rwxrwxrwx. 1 labadmin labadmin 2.7G Oct 28 12:09 LINUX.X64_193000_grid_home.zip
-rwxrwxrwx. 1 labadmin labadmin 124M Oct 28 12:09 p39803650_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 125M Oct 28 12:10 p39902038_210000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 125M Oct 28 12:10 p39902051_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 2.3G Oct 28 12:17 p39915792_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 2.7G Oct 28 12:31 p39922757_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 2.4G Oct 28 12:44 p40121635_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 2.8G Oct 28 12:56 p40121665_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 129M Oct 28 12:57 p6880880_190000_Linux-x86-64.zip
-rwxrwxrwx. 1 labadmin labadmin 129M Oct 28 12:58 p6880880_210000_Linux-x86-64.zip
drwxr-xr-x. 4 oracle   oinstall   37 Oct 28 13:16 ..
drwxr-xr-x. 2 oracle   oinstall 4.0K Oct 28 13:19 .
```

b. Which patch is which, and unpacking the combo patch

The patches were downloaded from My Oracle Support on a separate machine, `z11devrm473`, with a README listing each file. Only four of them are used: OPatch, the GI + OJVM combo, and the JDK patch (the two 21c files and the single RU zips aren't needed; the combo contains the GI RU and the OJVM RU).

`z11devrm473` (the download machine, in the patch folder)

```
cat README-patch-numbers-with-descriptions
```

This lab's output:

Output what this lab printed; nothing to run

```
[apreddy@z11devrm473 OCT2026-patches]$ cat README-patch-numbers-with-descriptions
 p40121665_190000_Linux-x86-64.zip   -  Combo OJVM Release Update 19.33.0.0.261020 and GI Release Update
19.33.0.0.261020 PATCH 40121665
 p40121635_190000_Linux-x86-64.zip   -  Combo OJVM Release Update 19.33.0.0.261020 and Database Release
Update 19.33.0.0.261020 PATCH 40121635  for UNIX, or
 p39915792_190000_Linux-x86-64.zip   -  Database Release Update 19.33.0.0.261020 PATCH 39915792  for UNIX, or
 p39922757_190000_Linux-x86-64.zip   -  GI Release Update 19.33.0.0.261020 PATCH 39922757
 p39803650_190000_Linux-x86-64.zip   -  OJVM Release Update 19.33.0.0.261020 PATCH 39803650 for all platforms
 p39902051_190000_Linux-x86-64.zip   -  JDK8u511 PATCH 39902051
 p6880880_190000_Linux-x86-64.zip    -  Opatch for 19C

p39902038_210000_Linux-x86-64.zip   -  JDK for 21C
p6880880_210000_Linux-x86-64.zip    -  Opatch for 21C
```

PATCH	WHAT IT IS	USED HERE
`p6880880_190000`	OPatch for 19c (12.2.0.1.54)	both homes; 19.3 ships 12.2.0.1.17, too old for the RU
`p40121665_190000`	Combo: OJVM RU 19.33 + GI RU 19.33	unpacked to `40121665/39922757` (GI RU) and `40121665/39803650` (OJVM RU), both homes
`p39902051_190000`	JDK 8u511 bundle patch	both homes
`p40121635`, `p39915792`, `p39922757`, `p39803650`	the OJVM + DB RU combo, and the RUs as single zips	not used; the combo above contains them
`p39902038_210000`, `p6880880_210000`	JDK and OPatch for 21c	not used

Why the GI RU in the Oracle home too: the GI RU contains the Database RU (`39915792`) and the OCW RU (`39922783`) that a RAC database home needs, so with the same patch on both homes, Grid and DB are at exactly the same level (see the `lspatches` in e and i).

oldblab-db1 as oracle (sudo su - oracle)

```
cd /u01/app/oracle/software
unzip p40121665_190000_Linux-x86-64.zip
unzip p39902051_190000_Linux-x86-64.zip
cd 40121665
pwd
ls -altrh
```

Check: `40121665` holds `39803650` (OJVM RU) and `39922757` (GI RU), plus Oracle's README; `/u01/app/oracle/software/39902051` exists for the JDK patch. (Only the `pwd` and `ls` were captured in this run.)

This lab's output:

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 40121665]$ pwd
/u01/app/oracle/software/40121665
[oracle@oldblab-db1 40121665]$ ls -altrh
total 64K
drwxrwxrwx. 4 oracle oinstall   67 Oct  1 07:41 39803650
drwxrwxrwx. 8 oracle oinstall  159 Oct 16 05:24 39922757
-rwxrwxrwx. 1 oracle oinstall  34K Oct 19 04:16 PatchSearch.xml
-rwxrwxrwx. 1 oracle oinstall  23K Oct 19 06:28 README.html
drwxrwxrwx. 4 oracle oinstall   80 Oct 19 06:28 .
drwxr-xr-x. 3 oracle oinstall 4.0K Oct 28 13:29 ..
```

c. Grid home: unzip 19.3 and replace OPatch

19c homes are image-based: the base zip's contents **are** the home, so it's unzipped straight into the Grid home directory. The 19.3 home ships OPatch 12.2.0.1.17; the 19.33 RU needs a newer one, so the old `OPatch` directory is removed and the new one unzipped in its place.

oldblab-db1 as grid (sudo su - grid)

```
cd /u01/app/19.0.0/grid/
pwd
unzip /u01/app/oracle/software/LINUX.X64_193000_grid_home.zip
cd OPatch/
./opatch version
cd ..
rm -rf OPatch/
unzip /u01/app/oracle/software/p6880880_190000_Linux-x86-64.zip
cd OPatch/
./opatch version
cd ..
```

Check: `12.2.0.1.17` before and `12.2.0.1.54` after.

This lab's output (the unzip listings left out):

Output what this lab printed; nothing to run

```
[grid@oldblab-db1 grid]$ pwd
/u01/app/19.0.0/grid
[grid@oldblab-db1 grid]$ unzip /u01/app/oracle/software/LINUX.X64_193000_grid_home.zip
…
[grid@oldblab-db1 grid]$ cd OPatch/
[grid@oldblab-db1 OPatch]$ ./opatch version
OPatch Version: 12.2.0.1.17

[grid@oldblab-db1 grid]$ rm -rf OPatch/
[grid@oldblab-db1 grid]$ unzip /u01/app/oracle/software/p6880880_190000_Linux-x86-64.zip
…
[grid@oldblab-db1 grid]$ cd OPatch/
[grid@oldblab-db1 OPatch]$ ./opatch version
OPatch Version: 12.2.0.1.54

OPatch succeeded.
```

d. Grid home: apply the OJVM RU, the GI RU and the JDK patch

oldblab-db1 as grid (sudo su - grid)

```
cd /u01/app/19.0.0/grid
./gridSetup.sh -applyRU /u01/app/oracle/software/40121665/39803650/ -silent
./gridSetup.sh -applyRU /u01/app/oracle/software/40121665/39922757/ -silent
./gridSetup.sh -applyOneOffs /u01/app/oracle/software/39902051 -silent
```

Check: each ends with `Successfully applied the patch.`

This lab's output:

Output what this lab printed; nothing to run

```
[grid@oldblab-db1 grid]$ ./gridSetup.sh -applyRU /u01/app/oracle/software/40121665/39803650/ -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/40121665/39803650/...
Successfully applied the patch.

[grid@oldblab-db1 grid]$ ./gridSetup.sh -applyRU /u01/app/oracle/software/40121665/39922757/ -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/40121665/39922757/...
Successfully applied the patch.

[grid@oldblab-db1 grid]$ pwd
/u01/app/19.0.0/grid
[grid@oldblab-db1 grid]$ ./gridSetup.sh -applyOneOffs /u01/app/oracle/software/39902051 -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/39902051...
Successfully applied the patch.
```

e. Grid home: software-only install and root scripts

`CRS_SWONLY` installs the Grid software on this host without configuring a cluster: it relinks the binaries and registers the home in `/u01/app/oraInventory`. Then the root scripts set the inventory and home permissions.

oldblab-db1 as grid (sudo su - grid)

```
cd /u01/app/19.0.0/grid
./gridSetup.sh \
  -silent \
  -waitforcompletion \
  oracle.install.option=CRS_SWONLY \
  ORACLE_BASE=/u01/app/grid \
  INVENTORY_LOCATION=/u01/app/oraInventory \
  UNIX_GROUP_NAME=oinstall \
  oracle.install.asm.OSDBA=asmdba \
  oracle.install.asm.OSOPER=asmoper \
  oracle.install.asm.OSASM=asmadmin
cd OPatch/
./opatch lspatches
```

Check: `Successfully Setup Software with warning(s).` (the warnings are optional prerequisite checks), and seven patches in `lspatches`: the JDK bundle patch, the OJVM RU, and five that the GI RU brings: ACFS, OCW, Database RU, Tomcat and DBWLM.

This lab's output:

`Output` what this lab printed; nothing to run

```
[grid@oldblab-db1 grid]$ ./gridSetup.sh -silent -waitforcompletion oracle.install.option=CRS_SWONLY …
oracle.install.asm.OSASM=asmadmin
Launching Oracle Grid Infrastructure Setup Wizard...

The response file for this session can be found at:
 /u01/app/19.0.0/grid/install/response/grid_2026-10-28_03-44-56PM.rsp

You can find the log of this install session at:
 /tmp/GridSetupActions2026-10-28_03-44-56PM/gridSetupActions2026-10-28_03-44-56PM.log

As a root user, execute the following script(s):
          1. /u01/app/oraInventory/orainstRoot.sh
          2. /u01/app/19.0.0/grid/root.sh

Execute /u01/app/oraInventory/orainstRoot.sh on the following nodes:
[oldblab-db1]
Execute /u01/app/19.0.0/grid/root.sh on the following nodes:
[oldblab-db1]



Successfully Setup Software with warning(s).
Moved the install session logs to:
 /u01/app/oraInventory/logs/GridSetupActions2026-10-28_03-44-56PM
[grid@oldblab-db1 grid]$ cd OPatch/
[grid@oldblab-db1 OPatch]$ ./opatch lspatches
39902051;JDK BUNDLE PATCH 19.0.0.0.261020
39922802;ACFS RELEASE UPDATE 19.33.0.0.0 (39922802)
39922783;OCW RELEASE UPDATE 19.33.0.0.0 (39922783)
39915792;Database Release Update : 19.33.0.0.261020 (39915792)
39107855;TOMCAT RELEASE UPDATE 19.0.0.0.0 (39107855)
39107825;DBWLM RELEASE UPDATE 19.0.0.0.0 (39107825)
39803650;OJVM RELEASE UPDATE: 19.33.0.0.261020 (39803650)

OPatch succeeded.
```

`oldblab-db1 as root` (after the install above has finished)

```
/u01/app/oraInventory/orainstRoot.sh
/u01/app/19.0.0/grid/root.sh
```

Check: `The execution of the script is complete.` from `orainstRoot.sh`; `root.sh` writes its output to the log file it names.

This lab's output:

Output what this lab printed; nothing to run

```
[root@oldblab-db1 ~]# /u01/app/oraInventory/orainstRoot.sh
Changing permissions of /u01/app/oraInventory.
Adding read,write permissions for group.
Removing read,write,execute permissions for world.

Changing groupname of /u01/app/oraInventory to oinstall.
The execution of the script is complete.
[root@oldblab-db1 ~]# /u01/app/19.0.0/grid/root.sh
Check /u01/app/19.0.0/grid/install/root_oldblab-db1_2026-10-28_15-48-28-438221666.log for the output of root
script
```

f. Grid home: create the gold image

oldblab-db1 as grid (sudo su - grid)

```
cd /u01/app/19.0.0/grid/
./gridSetup.sh -createGoldImage -destinationLocation /tmp/grid-gold -silent
```

Check: `Successfully Setup Software.` and the zip's path.

This lab's output:

Output what this lab printed; nothing to run

```
[grid@oldblab-db1 grid]$ ./gridSetup.sh   -createGoldImage   -destinationLocation /tmp/grid-gold   -silent
Launching Oracle Grid Infrastructure Setup Wizard...

Successfully Setup Software.
Gold Image location: /tmp/grid-gold/grid_home_2026-10-28_03-48-55PM.zip
```

g. Oracle home: unzip 19.3 and replace OPatch

The same as c, as `oracle`, into the DB home. (In this run the Oracle home was unzipped first, before the Grid home; the order doesn't matter.) The `ls` shows what an unpacked 19.3 home looks like: every directory dated April 2019, including `OPatch`, `.patch_storage` and `.opatchauto_storage`.

oldblab-db1 as oracle (sudo su - oracle)

```
cd /u01/app/oracle/product/19.0.0/dbhome_1/
pwd
unzip /u01/app/oracle/software/LINUX.X64_193000_db_home.zip
ls -altrh
cd OPatch/
./opatch version
cd ../
rm -rf OPatch/
unzip /u01/app/oracle/software/p6880880_190000_Linux-x86-64.zip
cd OPatch/
./opatch version
cd ..
```

Check: the home's directories (`bin` , `rdbms` , `OPatch` , `runInstaller` …) directly under `dbhome_1` ; OPatch `12.2.0.1.17` before and `12.2.0.1.54` after.

This lab's output (the unzip listings left out):

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 dbhome_1]$ pwd
/u01/app/oracle/product/19.0.0/dbhome_1
[oracle@oldblab-db1 dbhome_1]$ unzip /u01/app/oracle/software/LINUX.X64_193000_db_home.zip
…
[oracle@oldblab-db1 dbhome_1]$ ls -altrh
total 92K
-rw-r--r--.  1 oracle oinstall  852 Aug 18  2015 env.ora
-rw-r--r--.  1 oracle oinstall 2.9K Oct 14  2016 schagent.conf
-rwxr-x---.  1 oracle oinstall 1.8K Mar  8  2017 runInstaller
drwxr-x---. 14 oracle oinstall 4.0K Apr 12  2019 OPatch
drwxr-x---.  7 oracle oinstall   69 Apr 17  2019 xdk
drwxr-xr-x.  3 oracle oinstall   19 Apr 17  2019 wwg
drwxr-xr-x.  4 oracle oinstall   31 Apr 17  2019 usm
drwxr-xr-x.  5 oracle oinstall   45 Apr 17  2019 suptools
drwxr-xr-x.  6 oracle oinstall   54 Apr 17  2019 srvm
drwxr-xr-x.  3 oracle oinstall   17 Apr 17  2019 sqlj
drwxr-xr-x.  4 oracle oinstall   41 Apr 17  2019 sqldeveloper
drwxr-xr-x.  3 oracle oinstall   18 Apr 17  2019 slax
-rw-r-----.  1 oracle oinstall   10 Apr 17  2019 root.sh.old.1
drwxr-xr-x.  3 oracle oinstall   21 Apr 17  2019 relnotes
drwxr-xr-x.  4 oracle oinstall   29 Apr 17  2019 racg
drwxr-xr-x.  5 oracle oinstall   52 Apr 17  2019 R
drwxr-xr-x.  5 oracle oinstall   39 Apr 17  2019 perl
drwxr-xr-x.  4 oracle oinstall   33 Apr 17  2019 owm
drwxr-xr-x.  3 oracle oinstall   19 Apr 17  2019 oss
drwxr-xr-x.  6 oracle oinstall   52 Apr 17  2019 ord
drwxr-xr-x.  4 oracle oinstall   34 Apr 17  2019 oracore
drwxr-xr-x.  7 oracle oinstall   65 Apr 17  2019 opmn
drwxr-xr-x.  5 oracle oinstall   42 Apr 17  2019 olap
drwxr-xr-x.  5 oracle oinstall   46 Apr 17  2019 nls
drwxr-xr-x.  4 oracle oinstall   31 Apr 17  2019 mgw
drwxr-xr-x.  9 oracle oinstall   98 Apr 17  2019 md
drwxr-xr-x. 10 oracle oinstall  112 Apr 17  2019 ldap
drwxr-xr-x.  3 oracle oinstall   18 Apr 17  2019 has
drwxr-xr-x.  3 oracle oinstall   19 Apr 17  2019 dv
drwxr-xr-x.  3 oracle oinstall   20 Apr 17  2019 diagnostics
drwxr-xr-x.  3 oracle oinstall   20 Apr 17  2019 demo
drwxr-xr-x.  3 oracle oinstall   19 Apr 17  2019 dbjava
drwxr-xr-x.  3 oracle oinstall   20 Apr 17  2019 data
drwxr-xr-x.  7 oracle oinstall   71 Apr 17  2019 cv
drwxr-xr-x.  3 oracle oinstall   18 Apr 17  2019 css
drwxr-xr-x.  6 oracle oinstall   55 Apr 17  2019 crs
drwxr-xr-x.  6 oracle oinstall   78 Apr 17  2019 plsql
drwxr-xr-x.  2 oracle oinstall   22 Apr 17  2019 dbs
drwxr-xr-x.  2 oracle oinstall   33 Apr 17  2019 utl
drwxr-xr-x.  2 oracle oinstall   29 Apr 17  2019 instantclient
drwxr-xr-x. 13 oracle oinstall 4.0K Apr 17  2019 dmu
drwxr-xr-x.  3 oracle oinstall   35 Apr 17  2019 ucp
```

```
drwxr-xr-x.  3 oracle oinstall   35 Apr 17  2019 jdbc
drwxr-xr-x.  2 oracle oinstall   26 Apr 17  2019 QOpatch
drwxr-xr-x.  4 oracle oinstall   66 Apr 17  2019 ords
drwxr-xr-x.  5 oracle oinstall  119 Apr 17  2019 sdk
drwxr-xr-x.  6 oracle oinstall 4.0K Apr 17  2019 apex
drwxr-xr-x.  8 oracle oinstall  101 Apr 17  2019 odbc
drwxr-xr-x.  2 oracle oinstall 4.0K Apr 17  2019 jlib
drwxr-xr-x.  4 oracle oinstall   30 Apr 17  2019 drdaas
drwxr-xr-x. 11 oracle oinstall  119 Apr 17  2019 ctx
-rwx------.  1 oracle oinstall  786 Apr 17  2019 root.sh.old
drwxr-xr-x. 10 oracle oinstall  106 Apr 17  2019 network
drwxr-xr-x.  5 oracle oinstall   41 Apr 17  2019 hs
drwxr-xr-x.  9 oracle oinstall   93 Apr 17  2019 assistants
drwxr-xr-x.  6 oracle oinstall   53 Apr 18  2019 sqlplus
-rwx------.  1 oracle oinstall  638 Apr 18  2019 root.sh
drwxr-xr-x.  8 oracle oinstall 4.0K Apr 18  2019 oui
drwxr-xr-x.  5 oracle oinstall  191 Apr 18  2019 deinstall
drwxr-xr-x.  4 oracle oinstall   87 Apr 18  2019 clone
drwxr-xr-x.  2 oracle oinstall  102 Apr 18  2019 addnode
drwxr-xr-x.  4 oracle oinstall 4.0K Apr 18  2019 sqlpatch
drwxr-xr-x.  3 oracle oinstall   28 Apr 18  2019 .opatchauto_storage
drwxr-xr-x.  5 oracle oinstall  188 Apr 18  2019 .patch_storage
drwxr-xr-x. 13 oracle oinstall  140 Apr 18  2019 rdbms
drwxr-xr-x.  6 oracle oinstall   56 Apr 18  2019 precomp
drwxr-xr-x.  4 oracle oinstall  12K Apr 18  2019 lib
drwxr-xr-x.  6 oracle oinstall 4.0K Apr 18  2019 jdk
drwxr-xr-x.  8 oracle oinstall   82 Apr 18  2019 javavm
drwxr-x---. 13 oracle oinstall 4.0K Apr 18  2019 inventory
drwxr-xr-x.  2 oracle oinstall 8.0K Apr 18  2019 bin
drwxr-xr-x. 10 oracle oinstall 4.0K Apr 18  2019 install
drwxrwxr-x.  3 oracle oinstall   22 Oct 28 11:38 ..
drwxrwxr-x. 68 oracle oinstall 4.0K Oct 28 13:26 .
[oracle@oldblab-db1 dbhome_1]$ cd OPatch/
[oracle@oldblab-db1 OPatch]$ ./opatch version
OPatch Version: 12.2.0.1.17

OPatch succeeded.
[oracle@oldblab-db1 OPatch]$ cd ../
[oracle@oldblab-db1 dbhome_1]$ rm -rf OPatch/
[oracle@oldblab-db1 dbhome_1]$ unzip /u01/app/oracle/software/p6880880_190000_Linux-x86-64.zip
…
[oracle@oldblab-db1 dbhome_1]$ cd OPatch/
[oracle@oldblab-db1 OPatch]$ ./opatch version
OPatch Version: 12.2.0.1.54

OPatch succeeded.
```

h. Oracle home: apply the OJVM RU, the GI RU and the JDK patch

The same three patches as d, with `runInstaller` instead of `gridSetup.sh`.

oldblab-db1 as oracle (sudo su - oracle)

```
cd /u01/app/oracle/product/19.0.0/dbhome_1
./runInstaller -applyRU /u01/app/oracle/software/40121665/39803650/ -silent
./runInstaller -applyRU /u01/app/oracle/software/40121665/39922757/ -silent
./runInstaller -applyOneOffs /u01/app/oracle/software/39902051 -silent
```

Check: each ends with `Successfully applied the patch.`

This lab's output:

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 dbhome_1]$ ./runInstaller -applyRU /u01/app/oracle/software/40121665/39803650/ -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/40121665/39803650/...
Successfully applied the patch.

[oracle@oldblab-db1 dbhome_1]$ ./runInstaller -applyRU /u01/app/oracle/software/40121665/39922757/ -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/40121665/39922757/...
Successfully applied the patch.

[oracle@oldblab-db1 dbhome_1]$ ./runInstaller -applyOneOffs /u01/app/oracle/software/39902051 -silent
Preparing the home to patch...
Applying the patch /u01/app/oracle/software/39902051...
Successfully applied the patch.
```

i. Oracle home: software-only install, root script and patch list

`INSTALL_DB_SWONLY` installs Enterprise Edition without creating a database; the OS groups map the database's admin roles to the `oracle` user's groups from a.

oldblab-db1 as oracle (sudo su - oracle)

```
cd /u01/app/oracle/product/19.0.0/dbhome_1
./runInstaller \
  -silent \
  -waitforcompletion \
  -responseFile \
  /u01/app/oracle/product/19.0.0/dbhome_1/install/response/db_install.rsp \
  oracle.install.option=INSTALL_DB_SWONLY \
  UNIX_GROUP_NAME=oinstall \
  ORACLE_HOME=/u01/app/oracle/product/19.0.0/dbhome_1 \
  ORACLE_BASE=/u01/app/oracle \
  oracle.install.db.InstallEdition=EE \
  oracle.install.db.OSDBA_GROUP=dba \
  oracle.install.db.OSOPER_GROUP=oper \
  oracle.install.db.OSBACKUPDBA_GROUP=backupdba \
  oracle.install.db.OSDGDBA_GROUP=dgdba \
  oracle.install.db.OSKMDBA_GROUP=kmdba \
  oracle.install.db.OSRACDBA_GROUP=racdba \
  DECLINE_SECURITY_UPDATES=true
```

Check: `Successfully Setup Software with warning(s).` and the request to run `root.sh`.

This lab's output:

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 dbhome_1]$ ./runInstaller -silent -waitforcompletion -responseFile …
DECLINE_SECURITY_UPDATES=true
Launching Oracle Database Setup Wizard...



As a root user, execute the following script(s):
          1. /u01/app/oracle/product/19.0.0/dbhome_1/root.sh

Execute /u01/app/oracle/product/19.0.0/dbhome_1/root.sh on the following nodes:
[oldblab-db1]


Successfully Setup Software with warning(s).
```

oldblab-db1 as root (after the install above has finished)

```
/u01/app/oracle/product/19.0.0/dbhome_1/root.sh
```

This lab's output:

Output what this lab printed; nothing to run

```
[root@oldblab-db1 ~]# /u01/app/oracle/product/19.0.0/dbhome_1/root.sh
Check /u01/app/oracle/product/19.0.0/dbhome_1/install/root_oldblab-db1_2026-10-28_16-05-07-235984924.log for
the output of root script
```

oldblab-db1 as oracle (sudo su - oracle)

```
cd /u01/app/oracle/product/19.0.0/dbhome_1/OPatch/
./opatch lspatches
```

Check: four patches: JDK, OCW, Database RU and OJVM, all 19.33 / 261020. The DB home has no ACFS, Tomcat or DBWLM: those parts of the GI RU apply only to a Grid home.

This lab's output:

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 OPatch]$ ./opatch lspatches
39902051;JDK BUNDLE PATCH 19.0.0.0.261020
39922783;OCW RELEASE UPDATE 19.33.0.0.0 (39922783)
39915792;Database Release Update : 19.33.0.0.261020 (39915792)
39803650;OJVM RELEASE UPDATE: 19.33.0.0.261020 (39803650)

OPatch succeeded.
```

j. Oracle home: create the gold image

oldblab-db1 as oracle (sudo su - oracle)

```
mkdir -p /tmp/db-gold
cd /u01/app/oracle/product/19.0.0/dbhome_1
./runInstaller \
  -createGoldImage \
  -destinationLocation /tmp/db-gold \
  -silent
```

Check: `Successfully Setup Software.` and the zip's path.

This lab's output:

Output what this lab printed; nothing to run

```
[oracle@oldblab-db1 dbhome_1]$ ./runInstaller -createGoldImage -destinationLocation /tmp/db-gold -silent
Launching Oracle Database Setup Wizard...

Successfully Setup Software.
Gold Image location: /tmp/db-gold/db_home_2026-10-28_04-05-17PM.zip
```

The two homes side by side (from e and i):

PATCH	GRID HOME	ORACLE HOME
`39915792` Database Release Update 19.33.0.0.261020	yes	yes
`39922783` OCW Release Update 19.33.0.0.0	yes	yes
`39803650` OJVM Release Update 19.33.0.0.261020	yes	yes
`39902051` JDK bundle patch 19.0.0.0.261020	yes	yes
`39922802` ACFS Release Update 19.33.0.0.0	yes	—
`39107855` Tomcat Release Update 19.0.0.0.0	yes	—
`39107825` DBWLM Release Update 19.0.0.0.0	yes	—

For the next RU (all of this worked, so the 19.33 homes are fine as built): the OJVM RU is meant for database homes, so the Grid home doesn't need `39803650`; and Oracle's OJVM README applies it after the database RU, as a one-off. A one-command form per home for the next RU (not tested in this lab):

build host as grid: next RU (untested form)

```
./gridSetup.sh -applyRU <stage>/<combo>/<GI RU> -applyOneOffs <stage>/<JDK patch> -silent
# build host as oracle: next RU (untested form)
./runInstaller -applyRU <stage>/<combo>/<GI RU> -applyOneOffs <stage>/<combo>/<OJVM RU>,<stage>/<JDK
patch> -silent
```

k. Name, test and checksum the zips (not run yet)

The operator finds the zips by the names in the `RacDatabase`, so they get names that say what and which RU; `/tmp` is cleaned over time, so they move next to the software.

oldblab-db1 as root

```
mkdir -p /u01/app/oracle/software/gold
mv /tmp/grid-gold/grid_home_2026-10-28_03-48-55PM.zip /u01/app/oracle/software/gold/oracle-gridhome-19.33.zip
mv /tmp/db-gold/db_home_2026-10-28_04-05-17PM.zip /u01/app/oracle/software/gold/oracle-dbhome-19.33.zip
cd /u01/app/oracle/software/gold
chmod 644 oracle-*.zip
unzip -tq oracle-gridhome-19.33.zip; unzip -tq oracle-dbhome-19.33.zip
sha256sum oracle-gridhome-19.33.zip oracle-dbhome-19.33.zip | tee oracle-gold-19.33.sha256
ls -lh
```

Check: two `No errors detected in compressed data of …` lines, two checksums and both zips listed. Copy both zips and the `.sha256` file to the PC's stage folder; from there 8.2 copies them to the workers and checks them against this file.

8.1 Build the slim image (op-worker1, then op-worker2)

What the slim image is: Oracle Linux 9 with the OS packages, users and groups Oracle needs, plus Oracle's own setup scripts, but **no** Oracle software. The operator starts this image and drives the scripts in it; they install Grid and the database from the zips in 8.2 into `/u01`, on the worker's disk. That's why the manual lab's image, built from the OCNE doc's short Containerfile, can't be used here: it doesn't have those scripts.

Why each line:

- `podman-docker`**:** Oracle's build script calls `docker`. This package makes `docker` run Podman, which writes into the same image store CRI-O reads (`/var/lib/containers`). `/etc/containers/nodocker` silences its "emulating docker" notice.
- **A sparse clone:** only Oracle's RAC folder of the large `docker-images` repository is downloaded.
- `-v 19.3.0`**:** the 19c folder, which the README says to use for all 19c builds, RUs such as 19.32 included. Its `Containerfile` uses Oracle Linux 9.
- `-i`**:** skip the checksum of install zips; the slim image doesn't contain any.
- `SLIMMING=true`**:** build without the Oracle software.
- `TMPDIR` **on** `/scratch`**:** Podman's temporary files go to the 80 GB disk instead of the small root disk.

The build downloads Oracle Linux packages and takes about 10–20 minutes. You can run it in op-worker1 and op-worker2 at the same time, in two windows.

`op-worker1, then op-worker2` (10-20 minutes each)

CONNECT FIRST `ssh -p 2221 root@127.0.0.1` ; for op-worker2: `ssh -p 2222 root@127.0.0.1`

```
dnf install -y -q git podman-docker && touch /etc/containers/nodocker
mkdir -p /scratch/image/tmp && rm -rf /scratch/image/docker-images
git clone -q --depth 1 --filter=blob:none --sparse https://github.com/oracle/docker-images.git
/scratch/image/docker-images
cd /scratch/image/docker-images && git sparse-checkout set
OracleDatabase/RAC/OracleRealApplicationClusters && git log -1 --format='docker-images commit %h, %cd'
cd OracleDatabase/RAC/OracleRealApplicationClusters/containerfiles
export TMPDIR=/scratch/image/tmp
./buildContainerImage.sh -v 19.3.0 -t localhost/oracle/database-rac:19.3.0-slim -i -o '--build-arg
SLIMMING=true' > /scratch/image/build.log 2>&1; echo "build exit $?"
tail -8 /scratch/image/build.log
podman images localhost/oracle/database-rac
crictl images | grep database-rac
```

Check, on each worker:

- `build exit 0`, and the log ends with `Oracle Database container Image for Real Application Clusters (RAC) version 19.3.0 is ready to be extended` and `--> localhost/oracle/database-rac:19.3.0-slim`;
- `podman images` and `crictl images` both list `localhost/oracle/database-rac` with tag `19.3.0-slim`. `crictl` seeing it proves CRI-O, and so Kubernetes, can start it.

If the build fails, the full log is `/scratch/image/build.log`; paste its last 40 lines (`tail -40 /scratch/image/build.log`).

This lab's output (the two image IDs differ because each worker builds its own copy; that's expected):

Output what this lab printed; nothing to run

```
[root@op-worker1 containerfiles]#
build exit 0
e9302892b20d51de90d6eaba11a31d25a830538ca1c7fc432cc65620ec895556

  Oracle Database container Image for Real Application Clusters (RAC) version 19.3.0 is ready to be extended:

    --> localhost/oracle/database-rac:19.3.0-slim

  Build completed in 168 seconds.

REPOSITORY                    TAG          IMAGE ID      CREATED       SIZE
localhost/oracle/database-rac  19.3.0-slim  e9302892b20d  1 second ago  443 MB
localhost/oracle/database-rac                19.3.0-slim         e9302892b20d5       443MB

[root@op-worker2 containerfiles]#
build exit 0
ad073d6cdfc02e06a761d0ede9daa43c9972042bdc1e5786cc19752e82ad5b68
  …
    --> localhost/oracle/database-rac:19.3.0-slim

  Build completed in 169 seconds.

REPOSITORY                    TAG          IMAGE ID      CREATED       SIZE
localhost/oracle/database-rac  19.3.0-slim  ad073d6cdfc0  1 second ago  443 MB
localhost/oracle/database-rac                19.3.0-slim         ad073d6cdfc02       443MB
```

8.2 Stage the gold-image zips (from the PC)

What: both 19.32 gold-image zips copied from `C:\Users\alpoo\Downloads\oracle-stage` to `/scratch/software/stage` on **both** workers, then checked against the PC's own checksums.

Why each part:

- **Both workers:** the operator installs from the stage directory of the worker each RAC pod runs on, and either pod may land on either worker. It mounts the directory as `hostSwStageLocation`.
- **Zipped, not unpacked:** the operator unzips them itself; the `RacDatabase` names them in `gridSwZipFile` and `dbSwZipFile`. Oracle's prerequisites accept a 19.28 or later gold image, and these are 19.32.
- **Mode 755:** the permissions Oracle's prerequisites ask for on the zips.
- `restorecon`: gives the files the `container_file_t` label set up in 4f, so the pods may read them under SELinux.
- **Checksums:** 9.6 GB through a NAT port forward is easy to cut short. `sha256sum -c` on each worker compares against the checksums taken on the PC; a truncated or damaged copy fails here instead of an hour into the Grid install.

The copy takes a few minutes per worker. The checksums take about a minute on each side.

Git Bash

```
cd ~/Downloads/oracle-stage
sha256sum oracle-gridhome-19.32.zip oracle-dbhome-19.32.zip | tee /tmp/zips.sha256
for p in 2221 2222; do for f in oracle-gridhome-19.32.zip oracle-dbhome-19.32.zip; do
  for try in 1 2 3; do
    echo "--- port $p: $f (try $try)"
    scp -P $p $f root@127.0.0.1:/scratch/software/stage/ && grep " \*$f$" /tmp/zips.sha256 | ssh -p $p
root@127.0.0.1 "cd /scratch/software/stage && sha256sum -c" && break
  done
done; done
for p in 2221 2222; do echo "--- checking port $p"
  ssh -p $p root@127.0.0.1 "hostname; cd /scratch/software/stage && chmod 755 oracle-*.zip && restorecon
-F oracle-*.zip && ls -lhZ oracle-*.zip && sha256sum -c" < /tmp/zips.sha256
done
```

Check: for `op-worker1` and `op-worker2`, both zips listed with `-rwxr-xr-x`, about 4.8 G and `container_file_t`, and two lines ending in `OK`. Each file is checked right after its copy and copied again, up to three times, if the checksum doesn't match: a copy through the NAT port forward can drop just before the end (`lost connection` at 99%). A `FAILED` line in the final check means all three tries failed for that file; paste the output.

This lab's output (the first run, before the block had its retry loop: the copy to op-worker1 dropped at 99 % and the checksum caught it):

Output — what this lab printed; nothing to run

```
d02d04276e767d7715006fa3690851f2203ff578ff35e963e18b67b15ddcc917 *oracle-gridhome-19.32.zip
da75d72a2c1491d129e70ac0c3cdbb86c19024d5d80931f0e9c7714d25ef7e5f *oracle-dbhome-19.32.zip
--- copying to port 2221
oracle-gridhome-19.32.zip                      99% 4832MB  57.6MB/s   00:00 ETAlost connection
--- copying to port 2222
oracle-gridhome-19.32.zip                     100% 4864MB  23.1MB/s   03:30
oracle-dbhome-19.32.zip                       100% 4904MB  26.2MB/s   03:06
--- checking port 2221
op-worker1
-rwxr-xr-x. 1 root root system_u:object_r:container_file_t:s0 4.8G Oct 27 19:27 oracle-gridhome-19.32.zip
oracle-gridhome-19.32.zip: FAILED
oracle-dbhome-19.32.zip: FAILED open or read
sha256sum: WARNING: 1 listed file could not be read
sha256sum: WARNING: 1 computed checksum did NOT match
--- checking port 2222
op-worker2
-rwxr-xr-x. 1 root root system_u:object_r:container_file_t:s0 4.8G Oct 27 19:34 oracle-dbhome-19.32.zip
-rwxr-xr-x. 1 root root system_u:object_r:container_file_t:s0 4.8G Oct 27 19:31 oracle-gridhome-19.32.zip
oracle-gridhome-19.32.zip: OK
oracle-dbhome-19.32.zip: OK
```

This lab's output (op-worker1 again, with the retry loop):

Output what this lab printed; nothing to run

```
--- port 2221: oracle-gridhome-19.32.zip (try 1)
oracle-gridhome-19.32.zip                     100% 4864MB 163.4MB/s   00:29
oracle-gridhome-19.32.zip: OK
--- port 2221: oracle-dbhome-19.32.zip (try 1)
oracle-dbhome-19.32.zip                       100% 4904MB  59.0MB/s   01:23
oracle-dbhome-19.32.zip: OK
op-worker1
-rwxr-xr-x. 1 root root system_u:object_r:container_file_t:s0 4.8G Oct 27 19:39 oracle-dbhome-19.32.zip
-rwxr-xr-x. 1 root root system_u:object_r:container_file_t:s0 4.8G Oct 27 19:37 oracle-gridhome-19.32.zip
oracle-gridhome-19.32.zip: OK
oracle-dbhome-19.32.zip: OK
```

PHASE 9

The secrets

Result of this phase: two Kubernetes Secrets in namespace `rac`: `ssh-key-secret` with an SSH key pair the RAC pods use to reach each other, and `db-user-pass` with the database password, encrypted.

Why the operator wants them: Grid's and the database's installers run on one RAC node and work on the other over SSH, so both pods need the same key pair from the start; the manual lab created the same `ssh-key-secret` in its 10.2. The password is for `SYS`, `SYSTEM`, the PDB admin and ASM. The operator only takes it encrypted: the Secret holds the encrypted password file and the private key that decrypts it, and the setup scripts inside the pod decrypt it there.

9.1 The SSH key pair (op-controlplane)

The key names inside the Secret, `ssh-privkey` and `ssh-pubkey`, are the ones the `RacDatabase` will refer to.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
install -d -m 700 /root/rac-op/secrets
rm -f /root/rac-op/secrets/id_rsa /root/rac-op/secrets/id_rsa.pub
ssh-keygen -q -t rsa -b 4096 -N "" -C "rac-op RAC nodes" -f /root/rac-op/secrets/id_rsa
kubectl -n rac delete secret ssh-key-secret --ignore-not-found
kubectl -n rac create secret generic ssh-key-secret \
  --from-file=ssh-privkey=/root/rac-op/secrets/id_rsa --from-file=ssh-pubkey=/root/rac-
op/secrets/id_rsa.pub
kubectl -n rac get secret ssh-key-secret -o jsonpath='{.data}' | grep -o '"ssh-p[a-z]*key"'
```

Check: `"ssh-privkey"` and `"ssh-pubkey"`.

This lab's output:

Output what this lab printed; nothing to run

```
secret/ssh-key-secret created
"ssh-privkey"
"ssh-pubkey"
```

9.2 The database password, encrypted (op-controlplane)

What happens to the password: you type it twice at a hidden prompt. It lives only in a shell variable, is piped straight into OpenSSL, and is encrypted with a new RSA key; the variable is then cleared. It's never written to a file, the screen or this guide. The `-pkeyopt` settings are the operator's default (`rsa_padding_mode:oaep;rsa_oaep_md:sha256;rsa_mgf1_md:sha256`), which the `RacDatabase` names again in Phase 11.

Password rules: 8–30 characters with at least one upper-case letter, one lower-case letter and one digit, and only letters, digits, `_` , `#` and `-` . Those rules satisfy Oracle's password checks; leaving out quotes, spaces, `@` and `$` keeps them out of trouble in the operator's shell scripts.

op-controlplane (you type the password twice)

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/rac-op/secrets
while :; do
  read -rsp "Database password: " PW; echo; read -rsp "Same again: " PW2; echo
  if [ "$PW" != "$PW2" ]; then echo "They differ; try again."; continue; fi
  if [[ ${#PW} -ge 8 && ${#PW} -le 30 && "$PW" =~ [A-Z] && "$PW" =~ [a-z] && "$PW" =~ [0-9] && "$PW" =~
^[A-Za-z0-9_#-]+$ ]]; then break; fi
  echo "8-30 characters, upper and lower case and a digit, only letters, digits, _ # -; try again."
done
rm -f key.pem key.pub pwdfile.enc
openssl genrsa -out key.pem 2>/dev/null && openssl rsa -in key.pem -pubout -out key.pub 2>/dev/null
printf '%s\n' "$PW" | openssl pkeyutl -encrypt -pubin -inkey key.pub -out pwdfile.enc \
  -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256
LEN=$(( ${#PW} + 1 )); unset PW PW2
GOT=$(openssl pkeyutl -decrypt -inkey key.pem -in pwdfile.enc -pkeyopt rsa_padding_mode:oaep -pkeyopt
rsa_oaep_md:sha256 -pkeyopt rsa_mgf1_md:sha256 | wc -c)
[ "$GOT" -eq "$LEN" ] && echo "OK: pwdfile.enc decrypts to the password you typed ($LEN bytes with the
newline)" || echo "STOP: decrypt test failed"
chmod 600 key.pem pwdfile.enc
kubectl -n rac delete secret db-user-pass --ignore-not-found
kubectl -n rac create secret generic db-user-pass --from-file=pwdfile.enc --from-file=key.pem
kubectl -n rac get secret db-user-pass -o jsonpath='{.data}' | grep -o '"[a-z.]*"'
```

Check: `OK: pwdfile.enc decrypts…` (only the length is shown, never the password), then `"key.pem"` and `"pwdfile.enc"` . Remember the password: it's the one for `SYS` , `SYSTEM` and the PDB admin when you connect in Phase 12.

Where the files stay: `/root/rac-op/secrets` on op-controlplane, readable only by root. To change the password later, run 9.2 again and recreate the database.

This lab's output (the byte count is left out here on purpose: it gives away the password's length):

Output what this lab printed; nothing to run

```
Database password:
Same again:
OK: pwdfile.enc decrypts to the password you typed (NN bytes with the newline)
secret/db-user-pass created
"key.pem"
"pwdfile.enc"
```

PHASE 10

The ASM disks and the room on the workers

Result of this phase: the three shared disks' stable names, the same on both workers and saved on op-controlplane for the `RacDatabase`, and the memory, CPU and hugepages each worker can give a RAC pod.

Why no PVs by hand: in the manual lab you created the ASM PersistentVolumes and claims yourself (its Phase 7). Here the `RacDatabase` lists the disks' paths on the workers, and the operator creates the PVs from them. It only needs the right names.

10.1 Find the disks' by-id names (from the PC)

Why by-id and not `/dev/sdd`: `sdX` letters depend on discovery order and can differ between the workers. The by-id name comes from the disk's serial number, which VirtualBox derives from the VDI's UUID, so it's the same on both workers.

The block looks up each disk's UUID, finds its by-id name on both workers, checks they match and that the disk is blank, and saves the three paths on op-controlplane as `VOTE=`, `DATA=` and `FRA=` lines in `/root/rac-op/asm-disks` for the `RacDatabase`.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-op"
$ASM = [ordered]@{}; $Disks = [ordered]@{VOTE="vote1"; DATA="asm1"; FRA="asm2"}
foreach ($d in $Disks.GetEnumerator()) {
  $pre = ((& $VB showmediuminfo disk "$Base\shared\$($d.Value).vdi") | Select-String '^UUID:\s+([0-9a-f]
{8})').Matches[0].Groups[1].Value
  $w1 = ssh -p 2221 root@127.0.0.1 "ls /dev/disk/by-id/ | grep -v part | grep ^ata-VBOX_HARDDISK_VB$pre-"
  $w2 = ssh -p 2222 root@127.0.0.1 "ls /dev/disk/by-id/ | grep -v part | grep ^ata-VBOX_HARDDISK_VB$pre-"
  $blank = ssh -p 2221 root@127.0.0.1 "blkid -p /dev/disk/by-id/$w1 >/dev/null 2>&1 && echo used || echo
blank"
  if ($w1 -and ($w1 -eq $w2) -and (@($w1).Count -eq 1)) { $ASM[$d.Key] = "/dev/disk/by-id/$w1"; "OK
$($d.Key) ($($d.Value)) = $w1 (same on both workers, $blank)" }
  else { "STOP  $($d.Key) ($($d.Value), VB$pre): op-worker1 sees '$w1', op-worker2 sees '$w2'" }
}
if ($ASM.Count -eq 3) {
  ($ASM.GetEnumerator() | ForEach-Object { "$($_.Key)=$($_.Value)" }) -join "`n" | ssh -p 2220
root@127.0.0.1 "tr -d '\r' > /root/rac-op/asm-disks; cat /root/rac-op/asm-disks"
}
```

Check: three `OK` lines (`VOTE` , `DATA` , `FRA`), each ending in `blank` , then the three `VOTE=…` , `DATA=…` , `FRA=…` lines as saved on op-controlplane. `used` instead of `blank` means the disk carries a signature from earlier use; ASM refuses such disks, so stop and paste the output.

This lab's output:

Output what this lab printed; nothing to run

```
OK    VOTE (vote1) = ata-VBOX_HARDDISK_VB67cb0647-80696c8f (same on both workers, blank)
OK    DATA (asm1) = ata-VBOX_HARDDISK_VB74717f76-6ef41222 (same on both workers, blank)
OK    FRA (asm2) = ata-VBOX_HARDDISK_VB204551ec-809e56f5 (same on both workers, blank)
VOTE=/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f
DATA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222
FRA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
```

This lab's output (the first version of this block, on Windows PowerShell 5.1: [ordered]@{…} can't be followed directly by .GetEnumerator(); the block now builds the list in $Disks first. Nothing was saved, because the save step only runs when all three disks were found):

Output what this lab printed; nothing to run

```
foreach ($d in [ordered]@{VOTE="vote1"; DATA="asm1"; FRA="asm2"}.GetEnumerator()) {
The ordered attribute can be specified only on a hash literal node.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
    + FullyQualifiedErrorId : OrderedAttributeOnlyOnHashLiteralNode
```

10.2 What each worker can give a RAC pod (op-controlplane)

Why it matters here: the operator's admission check refuses a `RacDatabase` whose pods get less than **16 GiB** of memory, and it wants the SGA to fit into the hugepages. The manual lab ran its pods with 12 GiB. This shows

what Kubernetes has left on each worker after the system and the pods already running there, including the operator's own controllers.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for w in op-worker1 op-worker2; do
  echo "--- $w allocatable: $(kubectl get node $w -o jsonpath='cpu={.status.allocatable.cpu} memory=
{.status.allocatable.memory} hugepages-2Mi={.status.allocatable.hugepages-2Mi}')"
  kubectl describe node $w | sed -n '/Allocated resources/,/Events/p' | grep -E '^\s+
(cpu|memory|hugepages-2Mi)\s'
done
```

Check: note for each worker the allocatable `memory` (about 18 GiB is expected: 22 GB minus the 4 GiB of hugepages) and `hugepages-2Mi` (`4Gi`), and how much memory is already requested. Paste the output: the sizes in Phase 11 are chosen from it.

This lab's output:

Output what this lab printed; nothing to run

```
--- op-worker1 allocatable: cpu=4 memory=17845504Ki hugepages-2Mi=4Gi
  cpu                600m (15%)  500m (12%)
  memory             500Mi (2%)  450Mi (2%)
  hugepages-2Mi      0 (0%)      0 (0%)
--- op-worker2 allocatable: cpu=4 memory=17845504Ki hugepages-2Mi=4Gi
  cpu                1 (25%)     900m (22%)
  memory             900Mi (5%)  850Mi (4%)
  hugepages-2Mi      0 (0%)      0 (0%)
```

PHASE 11

The RacDatabase

Result of this phase: a 2-node RAC database `ORCLCDB` with the pluggable database `ORCLPDB` and the service `orclpdbsvc` , created by the operator from one YAML file: pods `racnode1-0` and `racnode2-0` , Grid Infrastructure, the ASM disk groups `VOTEDG` , `DATA` and `FRA` , and the database.

11.1 Make room for 16 GiB pods (op-controlplane)

The sizing, from 10.2: each worker has 17,845,504 KiB (17.0 GiB) allocatable. The operator refuses pods with less than 16 GiB. op-worker1 already has 500 MiB requested and op-worker2 900 MiB, most of it by the operator's own three controller pods (two of them on op-worker2). A 16 GiB RAC pod would fit on op-worker2 with only about 140 MiB to spare, and the pod would stay `Pending` after any restart that moved a controller pod there.

The fix: run the operator with one controller pod instead of three. Only one of the three is ever active (the others wait in leader election), so the lab loses nothing but standby copies. That frees about 400 MiB per controller pod removed.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n oracle-database-operator-system scale deploy/oracle-database-operator-controller-manager --replicas=1
kubectl -n oracle-database-operator-system rollout status deploy/oracle-database-operator-controller-manager --timeout=300s
kubectl -n oracle-database-operator-system get pods -o wide
for w in op-worker1 op-worker2; do echo "--- $w"; kubectl describe node $w | sed -n '/Allocated resources/,/Events/p' | grep -E '^\s+memory\s'; done
```

Check: one controller pod `Running`, and each worker's requested memory below about 900 MiB. With 16 GiB for the RAC pod, that leaves more than 1 GiB per worker.

This lab's output (the two extra pods were still `Terminating`, so op-worker2's memory hadn't dropped yet; 11.3's first block shows it a few minutes later):

Output — what this lab printed; nothing to run

```
deployment.apps/oracle-database-operator-controller-manager scaled
deployment "oracle-database-operator-controller-manager" successfully rolled out
NAME                                                READY   STATUS        RESTARTS   AGE   IP
NODE
oracle-database-operator-controller-manager-54cf89dd45-9rjws   1/1     Terminating   0          11h
10.244.2.4   op-worker2
oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running       0          11h
10.244.1.5   op-worker1
oracle-database-operator-controller-manager-54cf89dd45-xl646   1/1     Terminating   0          11h
10.244.2.5   op-worker2
--- op-worker1
  memory             500Mi (2%)  450Mi (2%)
--- op-worker2
  memory             900Mi (5%)  850Mi (4%)
```

What the pods get:

SETTING	VALUE	WHY
`resources` memory	16Gi (requests = limits)	the operator's minimum; requests = limits keeps the pod in the Guaranteed class, as in the manual lab
`resources` hugepages-2Mi	4Gi	the 2048 pages each worker reserved in 4g
`resources` cpu	3	4 vCPUs per worker, one left for the worker itself
`sgaSize` / `pgaSize`	3G / 1G	the SGA must fit in the hugepages, and SGA + PGA must stay within 80 % of the pod's memory
`cpuCount`	3	matches the CPU limit

11.2 Write and check the RacDatabase (op-controlplane)

What each part maps to (the manual lab step it replaces in brackets):

- `instanceDetails`: 2 nodes named `racnode1-0` / `racnode2-0`, Oracle homes under `/scratch/rac/cluster01/racnode<n>` (manual 4f, 10.4), private interfaces `eth1` / `eth2` from the two Multus networks (6.3), on workers labelled `raccluster=raccluster01` (5.4).
- `envVars`: the setup log in `/tmp/orod`, and `IGNORE_CRS_PREREQS` / `IGNORE_DB_PREREQS`: in a container on VirtualBox some optional prerequisite checks always fail (no swap, `/dev/shm` as tmpfs), as the manual lab's `INS-13014` warnings showed (manual 11.4).
- `CRS_ASM_DISCOVERY_STRING`: the list of disks ASM may look at (its `asm_diskstring`). **Without it the install fails at the DATA/FRA step** (see 11.4): Oracle's setup script builds the discovery string from the first `CRSDG` disk only, by cutting the last 2 characters of its path and adding `*`. That works for names like `/dev/sdd` → `/dev/sd*`, but each by-id name starts with its own UUID, so only the vote disk would match. Listing all three paths lets ASM see every disk from the start.
- `asmDiskGroupDetails`: three disk groups, each on one disk from 10.1 and with external redundancy (VirtualBox is the only copy anyway): `VOTEDG` of type `CRSDG` for the cluster's voting file and OCR, `DATA` of type `DBDATAFILESDG` for the database files, and `FRA` of type `DBRECOVERY` for the recovery area. The operator creates the PVs (manual Phase 7) and Grid creates `VOTEDG` during its install; the database groups follow before DBCA.
- `sshKeySecret`, `dbSecret`: the Secrets from Phase 9, with the same OpenSSL settings used to encrypt the password.
- `image`, `imagePullPolicy: IfNotPresent`: the slim image built on each worker in 8.1; `IfNotPresent` because it exists only locally.
- `resources`, `securityContext.sysctls`: sizes from 11.1, and the kernel settings the manual lab put in its StatefulSets (10.4). `kernel.shmmax` must be at least the SGA and `kernel.shmall` at least SGA / 4096; 8 GiB and 2,097,152 pages cover the 3 GB SGA.
- `configParams`: the same homes as the manual lab, the 19.32 gold-image zips in `/scratch/software/stage` (8.2), `ORCLCDB` with `ORCLPDB`.

The block fills in the three disk paths from `/root/rac-op/asm-disks`, then asks the API server to **check** the file without creating anything (`--dry-run=server`). That runs the operator's own admission check, so any sizing or field problem shows up now.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
source /root/rac-op/asm-disks; echo "VOTE=$VOTE"; echo "DATA=$DATA"; echo "FRA=$FRA"
cat > /root/rac-op/racdb.yaml <<EOF
apiVersion: database.oracle.com/v4
kind: RacDatabase
metadata:
  name: racdb
  namespace: rac
spec:
  instanceDetails:
    nodeCount: 2
    racHostSwLocation: /scratch/rac/cluster01
    racNodeName: racnode
    privateIPDetails:
      - { name: macvlan-conf1, interface: eth1 }
      - { name: macvlan-conf2, interface: eth2 }
    workerNodeSelector: { raccluster: raccluster01 }
  envVars:
    - { name: LOG_DIR, value: /tmp/orod }
    - { name: IGNORE_CRS_PREREQS, value: "true" }
    - { name: IGNORE_DB_PREREQS, value: "true" }
    - { name: CRS_ASM_DISCOVERY_STRING, value: "$VOTE,$DATA,$FRA" }
  asmDiskGroupDetails:
    - { name: VOTEDG, redundancy: EXTERNAL, type: CRSDG, disks: [ $VOTE ] }
    - { name: DATA, redundancy: EXTERNAL, type: DBDATAFILESDG, disks: [ $DATA ] }
    - { name: FRA, redundancy: EXTERNAL, type: DBRECOVERY, disks: [ $FRA ] }
  sshKeySecret: { name: ssh-key-secret, privKeySecretName: ssh-privkey, pubKeySecretName: ssh-pubkey }
  dbSecret:
    name: db-user-pass
    keyFileName: key.pem
    pwdFileName: pwdfile.enc
    encryptionType: pkeyutl
    pkeyopt: "rsa_padding_mode:oaep;rsa_oaep_md:sha256;rsa_mgf1_md:sha256"
  image: localhost/oracle/database-rac:19.3.0-slim
  imagePullPolicy: IfNotPresent
  scanSvcName: racnode-scan
  scanSvcTargetPort: 31521
  serviceDetails: { name: orclpdbsvc }
  resources:
    requests: { cpu: "3", memory: 16Gi, hugepages-2Mi: 4Gi }
    limits:   { cpu: "3", memory: 16Gi, hugepages-2Mi: 4Gi }
  securityContext:
    sysctls:
      - { name: kernel.shmall, value: "2097152" }
      - { name: kernel.sem, value: "250 32000 100 128" }
      - { name: kernel.shmmax, value: "8589934592" }
      - { name: kernel.shmmni, value: "4096" }
      - { name: net.ipv4.conf.all.rp_filter, value: "2" }
  configParams:
    gridHome: /u01/app/19c/grid
```

```
    gridBase: /u01/app/grid
    dbHome: /u01/app/oracle/product/19c/dbhome_1
    dbBase: /u01/app/oracle
    inventory: /u01/app/oraInventory
    hostSwStageLocation: /scratch/software/stage
    gridSwZipFile: oracle-gridhome-19.32.zip
    dbSwZipFile: oracle-dbhome-19.32.zip
    dbName: ORCLCDB
    pdbName: ORCLPDB
    sgaSize: "3G"
    pgaSize: "1G"
    cpuCount: 3
EOF
kubectl apply --dry-run=server -f /root/rac-op/racdb.yaml
```

Check: the three `VOTE=`, `DATA=` and `FRA=` paths printed, and `racdatabase.database.oracle.com/racdb created (server dry run)`. Nothing has been created yet. Any `admission webhook ... denied the request` message names the field it objects to; paste it.

This lab's output (this run's YAML didn't have the CRS_ASM_DISCOVERY_STRING line yet; see 11.4):

Output what this lab printed; nothing to run

```
VOTE=/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f
DATA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222
FRA=/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
racdatabase.database.oracle.com/racdb created (server dry run)
```

11.3 Create the database and follow it (op-controlplane)

What happens after `apply`: the operator creates the ASM PVs and claims, the headless Services (node names, VIPs, SCAN) and one StatefulSet per node. When both pods are running, the setup scripts in `racnode1-0` unzip the Grid gold image, install and configure Grid on both nodes, create the `VOTEDG` disk group, install the database home, create `DATA` and `FRA` and run DBCA. That's the manual lab's Phases 7 and 10–12 in one go, and took about three hours in this lab: the shared D: drive makes the unzips and DBCA slow.

First, confirm the file is complete: a terminal can drop lines from a long paste. The last lines must be the `configParams` values down to `cpuCount: 3`.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
tail -16 /root/rac-op/racdb.yaml
grep -c . /root/rac-op/racdb.yaml
for w in op-worker1 op-worker2; do echo "--- $w"; kubectl describe node $w | sed -n '/Allocated
resources/,/Events/p' | grep -E '^\s+memory\s'; done
```

Check: the tail ends with `dbName: ORCLCDB`, `pdbName: ORCLPDB`, `sgaSize: "3G"`, `pgaSize: "1G"`, `cpuCount: 3`; the file has 59 lines; and the requested memory has dropped now that the two extra controller pods are gone (about 500 MiB on op-worker2).

This lab's output (58 lines: this run's file was one line shorter, without CRS_ASM_DISCOVERY_STRING. The operator pod had moved to op-worker1 when the workers were restarted for vote1):

Output what this lab printed; nothing to run

```
      - { name: kernel.shmmni, value: "4096" }
      - { name: net.ipv4.conf.all.rp_filter, value: "2" }
  configParams:
    gridHome: /u01/app/19c/grid
    gridBase: /u01/app/grid
    dbHome: /u01/app/oracle/product/19c/dbhome_1
    dbBase: /u01/app/oracle
    inventory: /u01/app/oraInventory
    hostSwStageLocation: /scratch/software/stage
    gridSwZipFile: oracle-gridhome-19.32.zip
    dbSwZipFile: oracle-dbhome-19.32.zip
    dbName: ORCLCDB
    pdbName: ORCLPDB
    sgaSize: "3G"
    pgaSize: "1G"
    cpuCount: 3
58
--- op-worker1
  memory             500Mi (2%)  450Mi (2%)
--- op-worker2
  memory             100Mi (0%)  50Mi (0%)
```

Then create it and watch the pods come up:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl apply -f /root/rac-op/racdb.yaml
sleep 30; kubectl -n rac get racdatabase,sts,svc,pvc
kubectl -n rac get pods -o wide -w
```

Check: two StatefulSets, the headless Services and three `Bound` PVCs appear, then `racnode1-0` and `racnode2-0` reach `Running` (`0/1` READY is expected for a long time). Note which worker each pod is on. Press Ctrl+C to stop watching.

This lab's output:

Output what this lab printed; nothing to run

```
racdatabase.database.oracle.com/racdb created
NAME                                  DBNAME    DBSTATE   ROLE          VERSION       PDB CONNECT STR
STATE
racdatabase.database.oracle.com/racdb   ORCLCDB   PENDING   NOT_DEFINED   NOT_DEFINED   NOT_DEFINED
PENDING

NAME                        READY   AGE
statefulset.apps/racnode1   0/1     14s
statefulset.apps/racnode2   0/1     14s

NAME                        TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
service/racnode-scan        ClusterIP   None          <none>        <none>           30s
service/racnode-scan-lsnr   NodePort    10.99.59.33   <none>        1521:31521/TCP   30s
service/racnode1-0          ClusterIP   None          <none>        <none>           30s
service/racnode1-0-vip      ClusterIP   None          <none>        <none>           30s
service/racnode2-0          ClusterIP   None          <none>        <none>           30s
service/racnode2-0-vip      ClusterIP   None          <none>        <none>           30s

NAME                                        STATUS   VOLUME                  CAPACITY   ACCESS MODES
persistentvolumeclaim/asm-pvc-833f4ea4-racdb   Bound    asm-pv-833f4ea4-racdb   10Gi       RWX
persistentvolumeclaim/asm-pvc-c51c7cc4-racdb   Bound    asm-pv-c51c7cc4-racdb   50Gi       RWX
persistentvolumeclaim/asm-pvc-eaac2e15-racdb   Bound    asm-pv-eaac2e15-racdb   50Gi       RWX
NAME                          READY   STATUS        RESTARTS   AGE   IP           NODE
disk-check-daemonset-9p4gz    1/1     Terminating   0          30s   10.244.1.5   op-worker1
disk-check-daemonset-s7fhr    1/1     Terminating   0          30s   10.244.2.3   op-worker2
racnode1-0                    0/1     Running       0          14s   10.244.1.6   op-worker1
racnode2-0                    0/1     Running       0          14s   10.244.2.4   op-worker2
disk-check-daemonset-9p4gz    0/1     Error         0          42s   10.244.1.5   op-worker1
disk-check-daemonset-s7fhr    0/1     Error         0          42s   10.244.2.3   op-worker2
```

The PVC sizes (10Gi, 50Gi, 50Gi) are the sizes the disk check found; the operator only creates the claims and StatefulSets after that check succeeds. The `Error` on the disk-check pods is how Kubernetes records them being stopped when the operator deletes its DaemonSet.

If a pod stays `Pending` or `ContainerCreating` for more than five minutes, run `kubectl -n rac describe pod racnode1-0 | tail -20` and paste the events. If no pods appear at all, the operator's log says why: `kubectl -n oracle-database-operator-system logs deploy/oracle-database-operator-controller-manager --tail=40`.

Follow the install:

op-controlplane (about three hours)

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

Check: the log ends with a banner saying `ORACLE RAC DATABASE IS READY TO USE`. Press Ctrl+C to stop following; the install carries on regardless. If the log stops moving for more than 20 minutes or shows `ERROR` /

`failed` / `[FATAL]`, paste its last 40 lines (`kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -40 /tmp/orod/oracle_db_setup.log`). The times in the log are UTC.

This lab's output (key lines of /tmp/orod/oracle_db_setup.log, UTC; … marks cut text):

Output — what this lab printed; nothing to run

```
Defaulted container "racnode1-0" out of: racnode1-0, racnode1-init1 (init)
00:25:51  orasetupenv - setupsoftware :Grid software file is set : /mnt/stage/software/oracle-gridhome-
19.32.zip
00:25:51  … su - grid -c " unzip -q /mnt/stage/software/oracle-gridhome-19.32.zip -d /u01/app/19c/grid"
00:50:39  … OS command returned code : 0                       (Grid unzip done, 25 min)
00:50:42  … mv /usr/bin/scp /usr/bin/scp.openssh && printf … 'exec /usr/bin/scp.openssh -O -T "$@"' >
/usr/bin/scp
00:50:42  orasetupenv - configurelegacyscpfor19c :Installed Oracle 19c OpenSSH scp wrapper at /usr/bin/scp
for OL9
00:50:42  oracommon - setasmdiskperm :CRS_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f;chown grid:asmadmin …
00:50:42  oracommon - setasmdiskperm :RECO_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5;chown grid:asmadmin …
00:50:42  oracommon - setasmdiskperm :DB_ASM_DEVICE_LIST variable is set
00:50:42  … chmod 660 /dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222;chown grid:asmadmin …
00:50:42  … su - oracle -c " unzip -q /mnt/stage/software/oracle-dbhome-19.32.zip -d
/u01/app/oracle/product/19c/dbhome_1"
01:17:57  orasshsetup - setupsshusekey :Configuring SSH trust racnode1-0 -> racnode1-0
01:18:03  orasshsetup - setupsshusekey :Configuring SSH trust racnode1-0 -> racnode2-0
01:19:49  WARNING: orasshsetup - setupsshusekey :SSH trust not ready yet for racnode1-0 -> racnode2-0. This
is expected during setup …
01:19:49  … cat /mnt/.ssh/ssh-privkey > /home/oracle/.ssh/id_rsa
01:19:51  orasetupenv - setbanner :
          ========================================================================
          Since OP_TYPE is set to setuprac, setup will be initiated on this node
          ========================================================================
01:19:51  orasetupenv - finalizeorasetupenv :Total time for orasetupenv setup() = [ 3245.418 ] seconds
01:20:02  orasetupenv - waitfororasetupenvcompletion :orasetupenv completed successfully on all CRS nodes
01:20:02  orasetupenv - finalizeorasetupenv :Final SSH verification successful on all cluster nodes
01:20:02  WARNING: … Env variable ASM_DISCOVERY_DIR is not set .Ignoring the variable and procedding
further..
01:20:02  … openssl pkeyutl -decrypt -pkeyopt "rsa_padding_mode:oaep" … -in "/mnt/.dbsecrets/pwdfile.enc" …
01:20:02  oracommon - setpassword :ORACLE_PWD set to HIDDEN_STRING generated using encrypted password file
01:20:02  oragiprov - performsshsetup :Injected SSH keys detected; skipping legacy SSH setup
01:20:03  … su - grid -c "ssh racnode1-0 'export CV_ASSUME_DISTID=OL8; /u01/app/19c/grid/gridSetup.sh  -
ignorePreReq
             -waitforcompletion  -noCopy  -silent oracle.install.option=CRS_SWONLY
INVENTORY_LOCATION=/u01/app/oraInventory
             ORACLE_HOME=/u01/app/19c/grid ORACLE_BASE=/u01/app/grid oracle.install.asm.OSDBA=asmdba …'"
01:20:13  … (the same on racnode2-0)
01:23:14  … Successfully Setup Software.
01:23:14  … ssh racnode1-0 sudo /u01/app/oraInventory/orainstRoot.sh   … The execution of the script is
complete.
01:23:15  … ssh racnode2-0 sudo /u01/app/oraInventory/orainstRoot.sh   … The execution of the script is
complete.
01:23:16  … ssh racnode1-0 sudo /u01/app/19c/grid/root.sh / racnode2-0 … returned code : 0
01:23:20  oragiprov - installcvuqdisk :Successfully installed cvuqdisk file.       (both nodes)
01:23:20  oracommon - extractdomain :Fully Qualified Domain Name (FQDN): racnode1-
```

```
0.racnode.rac.svc.cluster.local
01:23:20  oracommon - getnwlist :The network card: eth0 for the ip: 10.244.1.6
01:23:20  oracommon - getnwlist :The network card: eth1 for the ip: 10.0.11.140
01:24:45  WARNING: … OS command returned code : 6 … [WARNING] [INS-13014] Target environment does not meet
some optional requirements.
          … Successfully Setup Software with warning(s).
01:24:45  WARNING: oracommon - checkoserr :OS command  failed. Flag is set to ignore this error!
01:24:46  … su - grid -c "ssh racnode1-0 sudo /u01/app/19c/grid/root.sh"
01:30:48  … returned code : 0 … Check /u01/app/19c/grid/install/root_racnode1-0_….log
01:30:48  … su - grid -c "ssh racnode2-0 sudo /u01/app/19c/grid/root.sh"
01:35:27  … returned code : 0 … Check /u01/app/19c/grid/install/root_racnode2-0_….log
01:35:27  … su - grid -c "/u01/app/19c/grid/gridSetup.sh -executeConfigTools -responseFile /tmp/grid.rsp -
silent"
01:38:39  WARNING: … OPatch failed with error code 106 … cannot locate your -invPtrLoc
'…/dbhome_1/oraInst.loc'
01:38:39  oraracprov - logstep :[RAC] db_sw_install :: start
01:38:39  … su - oracle -c "ssh racnode1-0 'export CV_ASSUME_DISTID=OL8; …/dbhome_1/runInstaller -
ignorePrereq -waitforcompletion
             -noCopy -silent oracle.install.option=INSTALL_DB_SWONLY …
oracle.install.db.CLUSTER_NODES=racnode1-0,racnode2-0 …'"
01:38:39  … (the same on racnode2-0)
01:46:45  … Successfully Setup Software.            (both nodes)
01:46:45  … ssh racnode1-0 sudo /u01/app/oracle/product/19c/dbhome_1/root.sh   … returned code : 0
01:46:46  … ssh racnode2-0 sudo /u01/app/oracle/product/19c/dbhome_1/root.sh   … returned code : 0
01:46:47  WARNING: … ASMCMD-8001: diskgroup 'FRA' does not exist or is not mounted
01:47:06  ERROR: … ORA-15014: path '/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5' is not in the
discovery set   (see 11.4)
```

What the log goes through (times from this lab's run, US Eastern):

STAGE	WHAT HAPPENS	TOOK
Disk check	a short-lived `disk-check-daemonset` pod per worker reads the disk sizes, then is deleted; its pods show `Error` while terminating, which is normal	seconds
Unzips	each pod unzips the Grid home (about 11 GB) and the DB home into its own `/u01`, both pods at once; the log is silent during `unzip -q`	8:25–9:17 PM
SSH	trust for `grid` and `oracle` between the pods, using the keys from `ssh-key-secret`	2 min
Grid software	`gridSetup.sh … -noCopy … CRS_SWONLY` on each node, then `orainstRoot.sh` and `root.sh`; `-noCopy` because every node already has its home, so nothing is copied over the network	3 min
Grid config	`gridSetup.sh -responseFile … -noCopy`, then `root.sh` on racnode1-0 (creates **VOTEDG**, OCR and voting file) and racnode2-0; then `-executeConfigTools`	15 min
DB software	`runInstaller … -noCopy INSTALL_DB_SWONLY` on each node (relinks `bin/oracle`), then the DB `root.sh`	8 min

STAGE	WHAT HAPPENS	TOOK
Disk groups	`asmca -createDiskGroup` for **FRA** and **DATA**	1 min
DBCA	creates `ORCLCDB` (instances on both pods) and `ORCLPDB`, sets up listeners and the service	80 min (10:03–11:22 PM)
Total	from `apply` to `ORACLE RAC DATABASE IS READY TO USE`, including the 11.4 stop	about 3 hours

Warnings that are expected (the script logs them and carries on):

- `SSH trust not ready yet for racnode1-0 -> racnode2-0` : racnode2-0 was still finishing its own setup; the later "Final SSH verification successful" is what counts.
- `OPatch failed with error code 106 … cannot locate … oraInst.loc` : the script tries OPatch on a home that isn't registered yet; there are no patches to apply to the 19.32 gold image.
- `returned code : 6 … [INS-13014] Target environment does not meet some optional requirements` : the same container prerequisite warnings as the manual lab.
- `home directory is not empty. skipping software unzipping…` (logged as ERROR): only on a rerun (11.4).
- `ASMCMD-8001: diskgroup 'FRA' does not exist` : the script checks for the disk group before creating it.
- `Could not locate file: …/install/files.lst` in the installer log: there's no file list because nothing was copied.

Quick checks while it runs (each takes seconds):

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do echo -n "$p  "; kubectl -n rac exec $p -c $p -- du -sh
/u01/app/19c/grid /u01/app/oracle/product/19c/dbhome_1 2>/dev/null | tr '\n' ' '; echo; done
for p in racnode1-0 racnode2-0; do echo "--- $p"; kubectl -n rac exec $p -c $p -- ls -l
/u01/app/19c/grid/bin/oracle; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'f=$(ls -t
/u01/app/19c/grid/install/root_racnode1-0_*.log 2>/dev/null | head -1); [ -n "$f" ] && tail -20 "$f"'
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "crsctl check cluster -all; olsnodes -n -s;
crsctl query css votedisk; asmcmd lsdg; srvctl status nodeapps; srvctl status scan_listener" 2>/dev/null
```

The first line shows the unzips growing (about 11G each when done). The second shows `bin/oracle` once the Grid software-only `root.sh` has run: `-rwsr-s--x grid oinstall`, about 440 MB, on both pods. The third is racnode1-0's `root.sh` log (use `racnode2-0` in both places for the second node). The last works once Grid is configured: CRS, CSS and EVM online on both nodes, both nodes `Active`, one voting disk in `VOTEDG`, the disk groups mounted so far, the VIPs and SCAN listeners running.

This lab's output (the checks run during the install):

Output what this lab printed; nothing to run

```
--- the unzips (watch -n 10 …du -sh …)
racnode1-0  5.8G        /u01/app/19c/grid
racnode2-0  5.5G        /u01/app/19c/grid
…
racnode1-0  128M        /u01/app/oracle/product/19c/dbhome_1
racnode2-0  121M        /u01/app/oracle/product/19c/dbhome_1

--- bin/oracle after the Grid software-only root.sh
--- racnode1-0
-rwsr-s--x. 1 grid oinstall 440858040 Oct 28 01:22 /u01/app/19c/grid/bin/oracle
--- racnode2-0
-rwsr-s--x. 1 grid oinstall 440858040 Oct 28 01:23 /u01/app/19c/grid/bin/oracle

--- root.sh on racnode1-0 (/u01/app/19c/grid/install/root_racnode1-0_*.log)
2026/10/28 01:25:21 CLSRSC-594: Executing installation step 8 of 19: 'ConfigOLR'.
…
2026/10/28 01:26:10 CLSRSC-594: Executing installation step 16 of 19: 'InitConfig'.
2026/10/28 01:27:33 CLSRSC-4002: Successfully installed Oracle Trace File Analyzer (TFA) Collector.
ASM has been created and started successfully.
[DBT-30001] Disk groups created successfully. Check /u01/app/grid/cfgtoollogs/asmca/asmca-261028AM012650.log
for details.
2026/10/28 01:27:56 CLSRSC-482: Running command: '/u01/app/19c/grid/bin/ocrconfig -upgrade grid oinstall'
CRS-4256: Updating the profile
Successful addition of voting disk f2a52506ee824fecbf0b6936be04ac26.
Successfully replaced voting disk group with +VOTEDG.
CRS-4256: Updating the profile
CRS-4266: Voting file(s) successfully replaced
##  STATE    File Universal Id               File Name Disk group
--  -----    -----------------               --------- ---------
 1. ONLINE   f2a52506ee824fecbf0b6936be04ac26 (/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f)
[VOTEDG]
Located 1 voting disk(s).
2026/10/28 01:28:46 CLSRSC-594: Executing installation step 17 of 19: 'StartCluster'.
2026/10/28 01:29:33 CLSRSC-343: Successfully started Oracle Clusterware stack
2026/10/28 01:29:34 CLSRSC-594: Executing installation step 18 of 19: 'ConfigNode'.
2026/10/28 01:30:29 CLSRSC-594: Executing installation step 19 of 19: 'PostConfig'.
2026/10/28 01:30:48 CLSRSC-325: Configure Oracle Grid Infrastructure for a Cluster ... succeeded

--- root.sh on racnode2-0
2026/10/28 01:31:58 CLSRSC-594: Executing installation step 16 of 19: 'InitConfig'.
2026/10/28 01:32:25 CLSRSC-594: Executing installation step 17 of 19: 'StartCluster'.
2026/10/28 01:34:30 CLSRSC-4002: Successfully installed Oracle Trace File Analyzer (TFA) Collector.
2026/10/28 01:35:04 CLSRSC-343: Successfully started Oracle Clusterware stack
2026/10/28 01:35:27 CLSRSC-325: Configure Oracle Grid Infrastructure for a Cluster ... succeeded

--- the cluster after both root.sh runs
racnode1-0:
CRS-4537: Cluster Ready Services is online
```

```
CRS-4529: Cluster Synchronization Services is online
CRS-4533: Event Manager is online
racnode2-0:
CRS-4537: Cluster Ready Services is online
CRS-4529: Cluster Synchronization Services is online
CRS-4533: Event Manager is online
racnode1-0      1       Active
racnode2-0      2       Active
State    Type    Rebal  Sector  Logical_Sector  Block       AU  Total_MB  Free_MB  …  Voting_files  Name
MOUNTED  EXTERN  N         512             512   4096  4194304     10240     9900  …             Y  VOTEDG/
VIP 10.244.1.6 is enabled
VIP 10.244.1.6 is running on node: racnode1-0
VIP 10.244.2.4 is enabled
VIP 10.244.2.4 is running on node: racnode2-0
SCAN listener LISTENER_SCAN1 is running on node racnode1-0
SCAN listener LISTENER_SCAN2 is running on node racnode2-0

--- -executeConfigTools (gridSetupActions*.log)
INFO:  [Oct 28, 2026 1:37:13 AM]   Voting Disk ...PASSED
INFO:  [Oct 28, 2026 1:37:21 AM]   ASM Integrity ...PASSED
INFO:  [Oct 28, 2026 1:37:26 AM]   Network configuration consistency checks ...PASSED
INFO:  [Oct 28, 2026 1:37:29 AM] Exit Status is 0
```

Following DBCA (the main log is silent while it runs):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'tail -5 $(ls -t
/u01/app/oracle/cfgtoollogs/dbca/ORCLCDB/*.log 2>/dev/null | grep -v trace | head -1)'
```

Check: percentage lines such as `Copying database files`, `Creating cluster database views`, `Creating Pluggable Databases`, ending at `100% complete`.

This lab's output (the end of the log, after the 11.4 fix and DBCA; UTC, so 11:22 PM Eastern):

Output what this lab printed; nothing to run

```
2026-10-28T03:21:31  Post plug operations are now complete.
                     Pluggable database ORCLPDB with pdb id - 3 is now marked as NEW.
2026-10-28T03:22:12  Pluggable database ORCLPDB opened read write
2026-10-28T03:22:38  Completed: alter pluggable database "ORCLPDB" open  instances=all
2026-10-28T03:22:56  ORCLPDB(3):Started service orclpdbsvc/orclpdbsvc/orclpdbsvc
ORACLE RAC DATABASE IS READY TO USE
```

11.4 If DATA or FRA creation fails with ORA-15014 (what happened in this lab)

The first run of this lab, without `CRS_ASM_DISCOVERY_STRING` in the YAML, stopped right after the DB home install:

Output what this lab printed; nothing to run

```
asmca -silent -createDiskGroup -diskGroupName FRA -redundancy EXTERNAL -disk '/dev/disk/by-id/ata-
VBOX_HARDDISK_VB204551ec-…'
[FATAL] [DBT-30006] Exception while Processing Request.
   CAUSE: ORA-15018: diskgroup cannot be created
ORA-15031: disk specification '/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-…' matches no disks
ORA-15014: path '/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-…' is not in the discovery set
```

Cause: when Grid was configured, the setup script set ASM's discovery string from the first `CRSDG` disk only: `/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c*` . That pattern matches the vote disk and nothing else, so ASM couldn't see the DATA and FRA disks. The script marked itself `failed` and stopped. Grid, VOTEDG and both homes were fine.

Fix without starting over: widen the discovery string, then run the setup script again. It skips everything already done (unzips, Grid, DB home) and continues with the disk groups and DBCA.

a. Check the state (changes nothing):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'echo "state: $(cat /tmp/orod/.statefile
2>/dev/null)"'
kubectl -n rac exec racnode1-0 -c racnode1-0 -- ps -eo pid,etime,args --sort=etime | grep -E '[m]ain\.py|
[r]unOracle'
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd dsget"
```

Check: `state: failed` ; no `main.py` / `runOracle` line, or only short-lived `main.py --checkracinst` / `--checkdbrole` lines (the pod's health probes); `dsget` shows the single `…VB<vote>…*` pattern.

This lab's output (only the state was captured here; `dsget` wasn't run before b in this lab):

Output what this lab printed; nothing to run

```
state before: failed
```

b. Widen the discovery string to all three disks. `dsset` writes it to both places ASM keeps it, the cluster profile (GPnP) and the ASM spfile in VOTEDG, so it's cluster-wide and survives restarts; nothing restarts.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
source /root/rac-op/asm-disks
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd dsset '$VOTE,$DATA,$FRA'"
for p in racnode1-0 racnode2-0; do echo "--- $p"; kubectl -n rac exec $p -c $p -- su - grid -c "asmcmd dsget; asmcmd lsdsk -p --candidate"; done
```

Check: on both pods, `parameter:` and `profile:` list all three paths, and the DATA and FRA disks are listed as `CANDIDATE`. (`command terminated with exit code 1` after `lsdsk --candidate` is normal.)

This lab's output:

Output — what this lab printed; nothing to run

```
--- racnode1-0
parameter:/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f, /dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222, /dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
profile:/dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f,/dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222,/dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
Group_Num  Disk_Num      Incarn  Mount_Stat  Header_Stat  Mode_Stat  State   Path
        0         1           0  CLOSED      CANDIDATE    ONLINE     NORMAL  /dev/disk/by-id/ata-VBOX_HARDDISK_VB204551ec-809e56f5
        0         0           0  CLOSED      CANDIDATE    ONLINE     NORMAL  /dev/disk/by-id/ata-VBOX_HARDDISK_VB74717f76-6ef41222
command terminated with exit code 1
--- racnode2-0
(the same)
```

c. Run the setup script again, detached so it keeps running after you disconnect. The check looks only for a python `main.py` that isn't a health probe; a plain `pgrep -f main.py` would match its own `bash -c` command line and always say "running".

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c '
running=$(ps -eo pid,args | awk "\$2 ~ /python/ && /main\\.py/ && !/check/")
if [ -n "$running" ]; then echo "STOP: setup already running:"; echo "$running"; exit 1; fi
setsid nohup /opt/scripts/startup/runOracle.sh > /tmp/orod/rerun.out 2>&1 < /dev/null &
sleep 8; ps -eo pid,etime,args | awk "\$3 ~ /python/ && /main\\.py/ && !/check/" && echo "setup restarted"'
kubectl -n rac exec racnode1-0 -c racnode1-0 -- tail -f /tmp/orod/oracle_db_setup.log
```

Check: `python3 main.py` with a few seconds of runtime and `setup restarted`; then new log lines with the current UTC time: "Grid is already installed on this machine ... Skipping Grid setup", CVU checks passing on both nodes, `asmca … FRA` and `asmca … DATA` returning code 0, and DBCA starting. From there the install continues as in 11.3.

This lab's output (the start block's ps lines, then key lines of the log, UTC. The --checkdbrole line is a health probe, correctly ignored):

Output what this lab printed; nothing to run

```
 268876       00:07 python3 main.py
 270334       00:01 /bin/python3 /opt/scripts/startup/scripts/main.py --checkdbrole=true
setup restarted
02:00:08  oracommon - checkgiinstalled :
          ================================================================================================
          Grid is already installed on this machine and /etc/oracle also exist. Skipping Grid setup..
          ================================================================================================
02:00:16  oragiprov - setup :
          ========================================
          Grid is already configured on this machine
          ========================================
02:00:35  … Verification of ASM Integrity was successful.
02:00:40  oraracprov - cluchecks :All required CVU checks passed successfully on node racnode1-0
02:02:13  … asmca -silent -createDiskGroup -diskGroupName FRA -redundancy EXTERNAL -disk '/dev/disk/by-
id/ata-VBOX_HARDDISK_VB204551ec-809e56f5'
02:02:52  … returned code : 0 … [INFO] [DBT-30001] Disk groups created successfully.
02:02:54  … asmca -silent -createDiskGroup -diskGroupName DATA -redundancy EXTERNAL -disk '/dev/disk/by-
id/ata-VBOX_HARDDISK_VB74717f76-6ef41222'
02:03:31  … returned code : 0 … [INFO] [DBT-30001] Disk groups created successfully.
02:03:32  oraracprov - logstep :[RAC] create_db :: start
02:03:32  … dbca -silent -ignorePreReqs -createDatabase -templateName General_Purpose.dbc -gdbname ORCLCDB
             -createAsContainerDatabase true -sysPassword ******** -systemPassword ******** -
datafileDestination +DATA
             -storageType ASM -characterSet AL32UTF8 -redoLogFileSize 1024 -databaseType OLTP -
databaseConfigType RAC
             -nodelist racnode1-0,racnode2-0 -useOMF true -numberOfPDBs 1 -pdbAdminPassword ******** -pdbName
ORCLPDB
             -initparams
'db_recovery_file_dest=+FRA,db_create_file_dest=+DATA,REMOTE_LOGIN_PASSWORDFILE=EXCLUSIVE,
             LOG_ARCHIVE_FORMAT=%t_%s_%r.arc,db_unique_name=ORCLCDB,db_files=1024,LOG_BUFFER=256M,…,
             DG_BROKER_CONFIG_FILE1=+DATA,DG_BROKER_CONFIG_FILE2=+FRA,DB_CREATE_ONLINE_LOG_DEST_1=+FRA,
             sga_target=3G,sga_max_size=3G,pga_aggregate_target=1G,cpu_count=3' -enableArchive true
```

11.5 Verify the database (op-controlplane)

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac get racdatabase racdb; kubectl -n rac get pods -o wide
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB -v; srvctl status service -d ORCLCDB"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd lsdg"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c 'export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba <<EOF
set lines 200 pages 100
col name for a60
select inst_id, instance_name, host_name, status from gv\$instance;
select name, open_mode from v\$pdbs;
select log_mode from v\$database;
select name from v\$controlfile;
select group#, thread#, bytes/1024/1024 mb, member from v\$log join v\$logfile using (group#) order by 1;
show parameter db_recovery_file_dest
EOF'
```

Check (this lab's values):

WHAT	EXPECTED
`racdb`	`ORCLCDB OPEN PRIMARY 19.32.0.0.0 racnode-scan:1521/orclpdbsvc AVAILABLE`
Pods	`racnode1-0` and `racnode2-0` both `1/1 Running`
Instances	`ORCLCDB1` on racnode1-0, `ORCLCDB2` on racnode2-0, both `OPEN`; `orclpdbsvc` on both
PDBs	`PDB$SEED READ ONLY`, `ORCLPDB READ WRITE`
Log mode	`ARCHIVELOG`
Disk groups	`DATA` and `FRA` 51200 MB, `VOTEDG` 10240 MB (`Voting_files Y`), all `MOUNTED EXTERN`
Control file	**one**, `+FRA/ORCLCDB/CONTROLFILE/current.…`
Redo	4 groups of 1024 MB (2 per thread), **one member each, all in** `+FRA`
Recovery area	`db_recovery_file_dest +FRA`, `db_recovery_file_dest_size 23895M` (DBCA's default)

This lab's output:

Output what this lab printed; nothing to run

```
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE
racnode1-0   1/1     Running   0          11h   10.244.1.6   op-worker1
racnode2-0   1/1     Running   0          11h   10.244.2.4   op-worker2
Instance ORCLCDB1 is running on node racnode1-0 with online services orclpdbsvc. Instance status: Open.
Instance ORCLCDB2 is running on node racnode2-0 with online services orclpdbsvc. Instance status: Open.
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
State    Type    Rebal  Sector  Logical_Sector  Block       AU  Total_MB  Free_MB  Req_mir_free_MB
Usable_file_MB  Offline_disks  Voting_files  Name
MOUNTED  EXTERN  N         512             512   4096  1048576     51200    44128                0
44128              0             N  DATA/
MOUNTED  EXTERN  N         512             512   4096  1048576     51200    46197                0
46197              0             N  FRA/
MOUNTED  EXTERN  N         512             512   4096  4194304     10240     9884                0
9884               0             Y  VOTEDG/

   INST_ID INSTANCE_NAME    HOST_NAME            STATUS
---------- ---------------- -------------------- ------------
         2 ORCLCDB2         racnode2-0           OPEN
         1 ORCLCDB1         racnode1-0           OPEN

NAME                                                         OPEN_MODE
------------------------------------------------------------ ----------
PDB$SEED                                                     READ ONLY
ORCLPDB                                                      READ WRITE

LOG_MODE
------------
ARCHIVELOG

NAME
------------------------------------------------------------
+FRA/ORCLCDB/CONTROLFILE/current.256.1247796495

    GROUP#    THREAD#         MB MEMBER
---------- ---------- ---------- --------------------------------------------
         1          1       1024 +FRA/ORCLCDB/ONLINELOG/group_1.258.1247796499
         2          1       1024 +FRA/ORCLCDB/ONLINELOG/group_2.257.1247796499
         3          2       1024 +FRA/ORCLCDB/ONLINELOG/group_3.259.1247800443
         4          2       1024 +FRA/ORCLCDB/ONLINELOG/group_4.260.1247800527

NAME                                 TYPE        VALUE
------------------------------------ ----------- ------------------------------
db_recovery_file_dest                string      +FRA
db_recovery_file_dest_size           big integer 23895M
```

What the alert log shows afterwards

(`/u01/app/oracle/diag/rdbms/orclcdb/ORCLCDB1/trace/alert_ORCLCDB1.log`): regular `Warning: long redo log write elapsed times detected` . All VM disks share one D: drive and redo shares the FRA disk with archiving, so log writes are slow; harmless in the lab. The repeated `ALTER SYSTEM SET _ipddb_enable=TRUE` lines come from the operator's health checks.

This lab's output (alert_ORCLCDB1.log the next morning, UTC):

`Output` what this lab printed; nothing to run

```
2026-10-28T03:28:35  Resize operation completed for file# 3, fname
+DATA/ORCLCDB/DATAFILE/sysaux.258.1247796421, old size 880640K, new size 911360K
2026-10-28T03:30:29  ALTER SYSTEM SET _ipddb_enable=TRUE SCOPE=MEMORY SID='ORCLCDB1';
…
2026-10-28T04:05:53  Warning: long redo log write elapsed times detected, the LG* process tracefiles have
more details
…
2026-10-28T10:00:46  Thread 1 advanced to log sequence 6 (LGWR switch),  current SCN: 2601276
                       Current log# 2 seq# 6 mem# 0: +FRA/ORCLCDB/ONLINELOG/group_2.257.1247796499
2026-10-28T10:01:25  ARC2 (PID:827359): Archived Log entry 2 added for B-1247796498.T-1.S-5 …
2026-10-28T11:06:05  Warning: long redo log write elapsed times detected, the LG* process tracefiles have
more details
```

11.6 Put a second control file and redo member in DATA (op-controlplane)

Why: the setup script sets `DB_CREATE_ONLINE_LOG_DEST_1=+FRA` when there's no `DBRED0` disk group, and with that set, Oracle's managed file naming puts the control file only there. The result in 11.5 is a single control file and single-member redo groups, all in FRA: losing that one disk group loses the database's only control file and its online redo. The usual layout has one copy of each in DATA and one in FRA.

a. A redo member in DATA for each group (the database stays open):

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c 'export ORACLE_SID=ORCLCDB1; sqlplus -s /
as sysdba <<EOF
alter database add logfile member '"'"'+DATA'"'"' to group 1;
alter database add logfile member '"'"'+DATA'"'"' to group 2;
alter database add logfile member '"'"'+DATA'"'"' to group 3;
alter database add logfile member '"'"'+DATA'"'"' to group 4;
alter system archive log current;
set lines 200
col member for a70
select group#, status, member from v\$logfile order by 1, member;
EOF'
```

Check: four `Database altered.` , then two members per group, one `+DATA/ORCLCDB/ONLINELOG/group_N.…` and one `+FRA/ORCLCDB/ONLINELOG/group_N.…` . In this lab the new members of groups 2 and 4 showed `INVALID` : a new member stays `INVALID` until its group is next written, which clears by itself after log switches (checked in c).

This lab's output:

Output what this lab printed; nothing to run

```
Database altered.
Database altered.
Database altered.
Database altered.
System altered.

    GROUP# STATUS  MEMBER
---------- ------- --------------------------------------------------------------------------------
         1         +DATA/ORCLCDB/ONLINELOG/group_1.274.1247831211
         1         +FRA/ORCLCDB/ONLINELOG/group_1.258.1247796499
         2 INVALID +DATA/ORCLCDB/ONLINELOG/group_2.275.1247831285
         2         +FRA/ORCLCDB/ONLINELOG/group_2.257.1247796499
         3         +DATA/ORCLCDB/ONLINELOG/group_3.276.1247831353
         3         +FRA/ORCLCDB/ONLINELOG/group_3.259.1247800443
         4 INVALID +DATA/ORCLCDB/ONLINELOG/group_4.277.1247831421
         4         +FRA/ORCLCDB/ONLINELOG/group_4.260.1247800527

8 rows selected.
```

b. A second control file in DATA (about 5 minutes of downtime). The block changes the setting to "one copy in DATA, one in FRA", stops the database, starts only instance 1 without mounting, lets RMAN write two new copies from the current file (RMAN also writes their names into the spfile), and starts the database again. The original file isn't touched, so if anything fails you can go back to it.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
CF=$(kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; echo
'set head off feed off
select name from v\$controlfile;' | sqlplus -s / as sysdba" | grep -m1 '^+')
echo "current control file: $CF"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; echo \"alter
system set control_files='+DATA','+FRA' scope=spfile sid='*';\" | sqlplus -s / as sysdba
srvctl stop database -d ORCLCDB
srvctl start instance -d ORCLCDB -i ORCLCDB1 -startoption nomount
rman target / <<EOF
restore controlfile from '$CF';
EOF
srvctl stop instance -d ORCLCDB -i ORCLCDB1
srvctl start database -d ORCLCDB
echo 'select name from v\$controlfile;' | sqlplus -s / as sysdba
srvctl status database -d ORCLCDB"
kubectl -n rac get racdatabase racdb
```

Check (this lab's output): `System altered.` ; RMAN `connected to target database: ORCLCDB (not mounted)` , `channel ORA_DISK_1: copied control file copy` , `output file`

`name=+DATA/ORCLCDB/CONTROLFILE/current.278.…` and `output file name=+FRA/ORCLCDB/CONTROLFILE/current.267.…`; then both names from `v$controlfile`, and `Instance ORCLCDB1 is running on node racnode1-0`, `Instance ORCLCDB2 is running on node racnode2-0`. If anything fails after `srvctl stop database`, stop and paste the output.

This lab's output (this run set CF by hand to the same value instead of the CF=$(...) line):

`Output` what this lab printed; nothing to run

```
current control file: +FRA/ORCLCDB/CONTROLFILE/current.256.1247796495
System altered.

Recovery Manager: Release 19.0.0.0.0 - Production on Wed Oct 28 11:52:51 2026
Version 19.32.0.0.0
connected to target database: ORCLCDB (not mounted)

RMAN> restore controlfile from '+FRA/ORCLCDB/CONTROLFILE/current.256.1247796495';
Starting restore at 28-OCT-26
using target database control file instead of recovery catalog
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=357 instance=ORCLCDB1 device type=DISK

channel ORA_DISK_1: copied control file copy
output file name=+DATA/ORCLCDB/CONTROLFILE/current.278.1247831575
output file name=+FRA/ORCLCDB/CONTROLFILE/current.267.1247831577
Finished restore at 28-OCT-26

Recovery Manager complete.

NAME
--------------------------------------------------------------------------------
+DATA/ORCLCDB/CONTROLFILE/current.278.1247831575
+FRA/ORCLCDB/CONTROLFILE/current.267.1247831577

Instance ORCLCDB1 is running on node racnode1-0
Instance ORCLCDB2 is running on node racnode2-0
```

c. Check the database is fully back, then remove the old control file. The old `current.256…` in FRA is no longer in `control_files`; the block only removes it if the database doesn't list it any more.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
OLD=+FRA/ORCLCDB/CONTROLFILE/current.256.1247796495
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB -v; srvctl status service -d ORCLCDB"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c 'export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba <<EOF
set lines 200 pages 100
col member for a70
col name for a60
alter system archive log current;
alter system archive log current;
select name, open_mode from v\$pdbs;
select name from v\$controlfile;
select group#, status, member from v\$logfile order by 1, member;
EOF'
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; echo 'select name from v\$controlfile;' | sqlplus -s / as sysdba" | grep -q "$OLD" && echo "STOP: $OLD is still in use" || kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "asmcmd rm $OLD; asmcmd ls +FRA/ORCLCDB/CONTROLFILE/"
kubectl -n rac get racdatabase racdb
```

Check (this lab's output): both instances `Open` with `orclpdbsvc`; `ORCLPDB READ WRITE`; the two control files `+DATA/…/current.278…` and `+FRA/…/current.267…`; all eight redo members without a status (the `INVALID` from a is gone); `asmcmd ls` shows only `current.267.…`; `racdb … OPEN PRIMARY 19.32.0.0.0 racnode-scan:1521/orclpdbsvc AVAILABLE`. If `ORCLPDB` shows `MOUNTED` instead, run `alter pluggable database ORCLPDB open instances=all;` and `alter pluggable database ORCLPDB save state instances=all;` as sysdba.

This lab's output:

Output what this lab printed; nothing to run

```
Instance ORCLCDB1 is running on node racnode1-0 with online services orclpdbsvc. Instance status: Open.
Instance ORCLCDB2 is running on node racnode2-0 with online services orclpdbsvc. Instance status: Open.
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
System altered.
System altered.

NAME                                                             OPEN_MODE
---------------------------------------------------------------- ----------
PDB$SEED                                                         READ ONLY
ORCLPDB                                                          READ WRITE

NAME
------------------------------------------------------------
+DATA/ORCLCDB/CONTROLFILE/current.278.1247831575
+FRA/ORCLCDB/CONTROLFILE/current.267.1247831577

    GROUP# STATUS  MEMBER
---------- ------- --------------------------------------------------------------------------------
         1         +DATA/ORCLCDB/ONLINELOG/group_1.274.1247831211
         1         +FRA/ORCLCDB/ONLINELOG/group_1.258.1247796499
         2         +DATA/ORCLCDB/ONLINELOG/group_2.275.1247831285
         2         +FRA/ORCLCDB/ONLINELOG/group_2.257.1247796499
         3         +DATA/ORCLCDB/ONLINELOG/group_3.276.1247831353
         3         +FRA/ORCLCDB/ONLINELOG/group_3.259.1247800443
         4         +DATA/ORCLCDB/ONLINELOG/group_4.277.1247831421
         4         +FRA/ORCLCDB/ONLINELOG/group_4.260.1247800527

8 rows selected.

current.267.1247831577
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
```

For the next build: a fourth disk group of type `DBREDO` in the YAML sets the online log destination separately from FRA; this lab didn't test that.

PHASE 12

Access from Windows

Result of this phase: a client on the Windows PC connects through SCAN (`127.0.0.1:31521/orclpdbsvc`), is sent on to either instance, and reconnects to the other instance when one goes away.

How a RAC connection works, and why it fails from Windows today:

1. The client connects to a SCAN listener (`LISTENER_SCAN1` or `2` , port 1521).

2. The SCAN listener picks an instance offering the service and **redirects** the client to that instance's node listener, using the address the instance registered: its `local_listener`.
3. The client opens a second connection to that address.

Inside the cluster all three steps work. From Windows, step 1 reaches SCAN once there's a port forward to the NodePort `racnode-scan-lsnr` (31521), but step 2 hands out a pod IP and port 1522 (12.1), which Windows can't reach. So each instance needs a redirect address that Windows **and** the pods can reach.

The plan (12.1 shows the starting point):

	RACNODE1-0 / ORCLCDB1	RACNODE2-0 / ORCLCDB2
extra node listener port (both nodes listen on both)	31522	31523
`local_listener`	`racnode1-0-vip.rac.svc.cluster.local:31522`	`racnode2-0-vip.rac.svc.cluster.local:31523`
that name, inside the cluster	the pod IP (headless Service), so the instance registers on 31522	the pod IP, on 31523
new NodePort Service	31522 → racnode1-0:31522	31523 → racnode2-0:31523
Windows NAT forward	127.0.0.1:31522 → 10.0.1.21:31522	127.0.0.1:31523 → 10.0.1.21:31523
Windows `hosts` (already there)	`racnode1-0-vip.rac.svc.cluster.local` → 127.0.0.1	`racnode2-0-vip.rac.svc.cluster.local` → 127.0.0.1

Plus 127.0.0.1:31521 → 10.0.1.21:31521 for SCAN. All forwards may go to op-worker1: a NodePort answers on every node and Kubernetes routes on to the right pod.

Why a different port per node: the port SCAN hands out must be the same number on Windows, on the NodePort and on the listener, and Windows reaches both nodes through the one address 127.0.0.1. Two nodes on 1522 can't share it.

Why not the operator's NodePort option: the `RacDatabase` fields for node-listener NodePorts (`baseLsnrTargetPort`) only take effect when the database is first created; this cluster already exists, so the same result is set up by hand here. For a new build they belong in the YAML (not tested in this lab).

12.1 Where things stand (read-only)

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl get nodes; kubectl -n rac get racdatabase racdb
kubectl -n rac get pods -o wide
kubectl -n rac get svc -o wide
kubectl -n rac get endpoints
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl config scan; srvctl config
scan_listener; srvctl config listener; srvctl status listener; lsnrctl status LISTENER | grep -E
'Alias|Listening|Service \"'"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl config service -d ORCLCDB -s
orclpdbsvc | grep -E 'Service name|Preferred|Available|Connection Load|Failover'"
for i in 1 2; do echo "=== ORCLCDB$i"; kubectl -n rac exec -i racnode$i-0 -c racnode$i-0 -- su - oracle -
c "export ORACLE_SID=ORCLCDB$i; sqlplus -s / as sysdba" <<'EOF'
set lines 200 pages 50
col name for a16
col value for a110
select name, value from v$parameter where name in ('local_listener','remote_listener');
EOF
done
for i in 1 2; do echo "=== racnode$i-0"; kubectl -n rac exec racnode$i-0 -c racnode$i-0 -- su - grid -c
"lsnrctl status LISTENER | grep DESCRIPTION; lsnrctl status LISTENER_SCAN$i | grep -E
'DESCRIPTION|orclpdbsvc'"; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl status scan_listener"
```

PowerShell on the Windows PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
"--- NAT forwards on k8s-public"; & $VB natnetwork list | Select-String 'Name:|ssh-op|:31|:15|:64'
"--- hosts entries"; Select-String -Path C:\Windows\System32\drivers\etc\hosts -Pattern 'racnode|scan' -
ErrorAction SilentlyContinue
"--- python"; python --version; python -m pip show oracledb 2>$null | Select-String '^(Name|Version):'
"--- port 31521 from Windows"; Test-NetConnection 127.0.0.1 -Port 31521 -InformationLevel Quiet
```

What it showed in this lab:

WHAT	FOUND
cluster	three nodes `Ready`; `racdb` `OPEN`, `19.32.0.0.0`, `AVAILABLE`; both pods `1/1 Running`
SCAN	`racnode-scan`, two SCAN VIPs = the two pod IPs; `LISTENER_SCAN1` on racnode1-0, `LISTENER_SCAN2` on racnode2-0, both **TCP 1521**
node listener	`LISTENER` on **TCP 1522** (not 1521), on each pod's own IP
`racnode-scan-lsnr`	NodePort `1521:31521`, selecting **only racnode1-0** (endpoint `10.244.1.6:1521`): from outside, 31521 always reaches `LISTENER_SCAN1`, which knows both instances

WHAT	FOUND
`local_listener`	`(ADDRESS=(PROTOCOL=TCP)(HOST=10.244.1.6)(PORT=1522))` and `…10.244.2.4…1522`: pod IPs, the address SCAN hands to clients
`remote_listener`	`racnode-scan:1521` on both instances; `orclpdbsvc` registered with both SCAN listeners (2 instances each)
`orclpdbsvc`	preferred on `ORCLCDB1,ORCLCDB2`, load balancing goal `LONG`, **no failover type** set
Windows	only the SSH and API forwards; `hosts` already maps both `-vip` names to 127.0.0.1; Python 3.14.5 with `oracledb` 26.0.1 (thin mode, no Oracle client needed); 31521 not reachable yet

This lab's output (cluster):

Output what this lab printed; nothing to run

```
NAME              STATUS   ROLES           AGE   VERSION
op-controlplane   Ready    control-plane   45h   v1.36.5
op-worker1        Ready    worker          45h   v1.36.5
op-worker2        Ready    worker          45h   v1.36.5
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   0          23h   10.244.1.6   op-worker1   <none>           <none>
racnode2-0   1/1     Running   0          23h   10.244.2.4   op-worker2   <none>           <none>
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE   SELECTOR
racnode-scan        ClusterIP   None          <none>        <none>           23h   app=oracle-database-
operator,database.oracle.com/component=RAC,database.oracle.com/managed-by=racdatabase-
controller,database.oracle.com/name=racdb
racnode-scan-lsnr   NodePort    10.99.59.33   <none>        1521:31521/TCP   23h
statefulset.kubernetes.io/pod-name=racnode1-0
racnode1-0          ClusterIP   None          <none>        <none>           23h
statefulset.kubernetes.io/pod-name=racnode1-0
racnode1-0-vip      ClusterIP   None          <none>        <none>           23h
statefulset.kubernetes.io/pod-name=racnode1-0
racnode2-0          ClusterIP   None          <none>        <none>           23h
statefulset.kubernetes.io/pod-name=racnode2-0
racnode2-0-vip      ClusterIP   None          <none>        <none>           23h
statefulset.kubernetes.io/pod-name=racnode2-0
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                ENDPOINTS               AGE
racnode-scan        10.244.1.6,10.244.2.4   23h
racnode-scan-lsnr   10.244.1.6:1521         23h
racnode1-0          10.244.1.6              23h
racnode1-0-vip      10.244.1.6              23h
racnode2-0          10.244.2.4              23h
racnode2-0-vip      10.244.2.4              23h
SCAN name: racnode-scan, Network: 1
Subnet IPv4: 10.244.0.0/255.255.0.0/eth0, static
Subnet IPv6:
SCAN 1 IPv4 VIP: 10.244.1.6
SCAN VIP is enabled.
SCAN 2 IPv4 VIP: 10.244.2.4
SCAN VIP is enabled.
SCAN Listeners for network 1:
Registration invited nodes:
Registration invited subnets:
Endpoints: TCP:1521
SCAN Listener LISTENER_SCAN1 exists
SCAN Listener is enabled.
SCAN Listener LISTENER_SCAN2 exists
SCAN Listener is enabled.
Name: LISTENER
Type: Database Listener
```

```
Network: 1, Owner: grid
Home: <CRS home>
End points: TCP:1522
Listener is enabled.
Listener is individually enabled on nodes:
Listener is individually disabled on nodes:
Listener LISTENER is enabled
Listener LISTENER is running on node(s): racnode1-0,racnode2-0
Alias                     LISTENER
Listening Endpoints Summary...
Service "+ASM" has 1 instance(s).
Service "+ASM_DATA" has 1 instance(s).
Service "+ASM_FRA" has 1 instance(s).
Service "+ASM_VOTEDG" has 1 instance(s).
Service "5edf079783cfb131e0630601f40a4240" has 1 instance(s).
Service "ORCLCDB" has 1 instance(s).
Service "ORCLCDBXDB" has 1 instance(s).
Service "orclpdb" has 1 instance(s).
Service "orclpdbsvc" has 1 instance(s).
Service name: orclpdbsvc
Failover type:
Failover method:
Failover retries:
Failover delay:
Failover restore: NONE
Connection Load Balancing Goal: LONG
Preferred instances: ORCLCDB1,ORCLCDB2
Available instances:
=== ORCLCDB1

NAME             VALUE
---------------- --------------------------------------------------------------------------------------------------
------------------
local_listener    (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.1.6)(PORT=1522))
remote_listener   racnode-scan:1521

=== ORCLCDB2

NAME             VALUE
---------------- --------------------------------------------------------------------------------------------------
------------------
local_listener    (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.2.4)(PORT=1522))
remote_listener   racnode-scan:1521

=== racnode1-0
Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=IPC)(KEY=LISTENER)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=LISTENER)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.1.6)(PORT=1522)))
Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=IPC)(KEY=LISTENER_SCAN1)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=LISTENER_SCAN1)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.1.6)(PORT=1521)))
Service "orclpdbsvc" has 2 instance(s).
=== racnode2-0
```

```
Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=IPC)(KEY=LISTENER)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=LISTENER)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.2.4)(PORT=1522)))
Connecting to (DESCRIPTION=(ADDRESS=(PROTOCOL=IPC)(KEY=LISTENER_SCAN2)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=ipc)(KEY=LISTENER_SCAN2)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.2.4)(PORT=1521)))
Service "orclpdbsvc" has 2 instance(s).
SCAN Listener LISTENER_SCAN1 is enabled
SCAN listener LISTENER_SCAN1 is running on node racnode1-0
SCAN Listener LISTENER_SCAN2 is enabled
SCAN listener LISTENER_SCAN2 is running on node racnode2-0
```

This lab's output (Windows):

`Output` what this lab printed; nothing to run

```
--- NAT forwards on k8s-public

Name:         k8s-public
        k8sapi:tcp:[]:6443:[10.0.1.10]:6443
        k8sapi-op:tcp:[]:6444:[10.0.1.20]:6443
        ssh-opcp:tcp:[]:2220:[10.0.1.20]:22
        ssh-opw1:tcp:[]:2221:[10.0.1.21]:22
        ssh-opw2:tcp:[]:2222:[10.0.1.22]:22

--- hosts entries

C:\Windows\System32\drivers\etc\hosts:41:127.0.0.1 racnode1-0-vip.rac.svc.cluster.local
C:\Windows\System32\drivers\etc\hosts:42:127.0.0.1 racnode2-0-vip.rac.svc.cluster.local

--- python
Python 3.14.5

Name: oracledb
Version: 26.0.1

--- port 31521 from Windows
WARNING: TCP connect to (127.0.0.1 : 31521) failed
False
```

The first version of the `local_listener` query failed (fixed above): it put the parameter names in double quotes inside the `su -c "…"` line. In SQL, double quotes mark identifiers, so Oracle read `"remote_listener"` as a column name. The block above feeds the SQL through a heredoc, so it can use the single quotes SQL needs.

`Output` what this lab printed; nothing to run

```
select name, value from v$parameter where name in ("local_listener","remote_listener")
                                                   *

ERROR at line 1:
ORA-00904: "remote_listener": invalid identifier
```

12.2 Add ports 31522 and 31523 to the node listener

What: `LISTENER` on both nodes listens on 31522 and 31523 as well as 1522.

Why both ports on both nodes: a listener's endpoints are one cluster-wide setting, the same on every node. Each instance will use only its own port (12.3); the other one is unused but harmless.

Interruption: the new ports take effect when the listener restarts; the block restarts it one node at a time, for a few seconds each. Connected sessions keep running, SCAN keeps working, new connections to the node being restarted fail for those seconds, and the instance re-registers on its own (the `sleep 60` gives it time).

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl modify listener -listener LISTENER -endpoints 'TCP:1522/TCP:31522/TCP:31523'; srvctl config listener -listener LISTENER | grep 'End points'"
for n in racnode1-0 racnode2-0; do
  echo "=== restart LISTENER on $n"
  kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl stop listener -listener LISTENER -node $n; srvctl start listener -listener LISTENER -node $n; srvctl status listener -listener LISTENER -node $n"
  sleep 10
done
sleep 60
for i in 1 2; do echo "=== racnode$i-0"; kubectl -n rac exec racnode$i-0 -c racnode$i-0 -- su - grid -c "lsnrctl status LISTENER | grep -E 'PROTOCOL=tcp|orclpdbsvc'"; done
for i in 1 2; do echo "=== ORCLCDB$i"; kubectl -n rac exec -i racnode$i-0 -c racnode$i-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB$i; sqlplus -s / as sysdba" <<'EOF'
set lines 200 pages 50
col value for a110
select value from v$parameter where name = 'local_listener';
EOF
done
kubectl -n rac get racdatabase racdb
```

Check: `End points: TCP:1522,31522,31523` ; the listener running on each node after its restart; three tcp lines per pod (its own pod IP, ports 1522, 31522, 31523) and `orclpdbsvc` registered again; `racdb` still `AVAILABLE` .

This lab's output:

Output what this lab printed; nothing to run

```
End points: TCP:1522,31522,31523
=== restart LISTENER on racnode1-0
Listener LISTENER is enabled on node(s): racnode1-0
Listener LISTENER is running on node(s): racnode1-0
=== restart LISTENER on racnode2-0
Listener LISTENER is enabled on node(s): racnode2-0
Listener LISTENER is running on node(s): racnode2-0

=== racnode1-0
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.1.6)(PORT=31523)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.1.6)(PORT=31522)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.1.6)(PORT=1522)))
Service "orclpdbsvc" has 1 instance(s).
=== racnode2-0
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.2.4)(PORT=31523)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.2.4)(PORT=31522)))
  (DESCRIPTION=(ADDRESS=(PROTOCOL=tcp)(HOST=10.244.2.4)(PORT=1522)))
Service "orclpdbsvc" has 1 instance(s).
=== ORCLCDB1

VALUE
--------------------------------------------------------------------------------------------------------------
-
 (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.1.6)(PORT=1522)), (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.1.6)(PORT=31522)),
(ADDRESS=(PROTOCOL=TCP)(HOST=10.244.1.6)(PORT=31523))


=== ORCLCDB2

VALUE
--------------------------------------------------------------------------------------------------------------
-
 (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.2.4)(PORT=1522)), (ADDRESS=(PROTOCOL=TCP)(HOST=10.244.2.4)(PORT=31522)),
(ADDRESS=(PROTOCOL=TCP)(HOST=10.244.2.4)(PORT=31523))


NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR               STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
```

What the `local_listener` lines show: Grid's agent rewrote each instance's `local_listener` by itself, from the pod IP and port 1522 to the pod IP with **all three** ports. The agent manages `local_listener` for a RAC database as long as it isn't set explicitly; it follows the listener's endpoints. The addresses are still pod IPs, which Windows can't reach, so 12.3 sets the parameter explicitly per instance.

12.3 Set `local_listener` explicitly, one value per instance

What: `ORCLCDB1` gets `racnode1-0-vip.rac.svc.cluster.local` port 31522 and `ORCLCDB2` gets `racnode2-0-vip.rac.svc.cluster.local` port 31523, with `scope=both` (now and in the spfile); `alter system register`

makes each instance tell the SCAN listeners its new address at once.

Why it works on both sides: inside the pod each name resolves to the pod's own IP (the block checks this first and skips an instance if it doesn't), so the instance reaches its listener on that port. On Windows the same names point to 127.0.0.1 (`hosts`), and 12.4 forwards the ports. No session is interrupted and nothing restarts.

The test logins: `nobody/x` is a deliberately wrong login, so no password is involved. `ORA-01017: invalid username/password` can only come from an instance, so it proves the client got through SCAN **and** through the redirect; `ORA-12541` or `ORA-12545` would mean the redirect address doesn't work.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for i in 1 2; do
  h=racnode$i-0-vip.rac.svc.cluster.local; p=$((31521+i))
  echo "=== ORCLCDB$i -> $h:$p"
  kubectl -n rac exec racnode$i-0 -c racnode$i-0 -- getent hosts $h || { echo "STOP: $h does not resolve in racnode$i-0"; continue; }
  kubectl -n rac exec -i racnode$i-0 -c racnode$i-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB$i; sqlplus -s / as sysdba" <<EOF
alter system set local_listener='(ADDRESS=(PROTOCOL=TCP)(HOST=$h)(PORT=$p))' scope=both sid='ORCLCDB$i';
alter system register;
EOF
done
sleep 30
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba" <<'EOF'
set lines 200 pages 50
col sid for a10
col value for a90
select inst_id, value from gv$parameter where name = 'local_listener' order by 1;
select sid, value from v$spparameter where name = 'local_listener';
EOF
for i in 1 2; do echo "=== LISTENER_SCAN$i"; kubectl -n rac exec racnode$i-0 -c racnode$i-0 -- su - grid -c "lsnrctl services LISTENER_SCAN$i" | awk '/Service "orclpdbsvc"/{f=1;print;next} /^Service "/{f=0} f && /Instance|ADDRESS/'; done
for t in 1 2 3 4; do kubectl -n rac exec racnode2-0 -c racnode2-0 -- su - oracle -c "echo exit | sqlplus -L -s nobody/x@//racnode-scan:1521/orclpdbsvc" | grep -E 'ORA-|SP2-'; done
kubectl -n rac get racdatabase racdb
```

Check: each name resolves to its pod's own IP; `System altered.` twice per instance; the new value for both instances in `gv$parameter` and stored per instance in `v$spparameter` ; both SCAN listeners listing `ORCLCDB1` at `racnode1-0-vip…:31522` and `ORCLCDB2` at `racnode2-0-vip…:31523` ; four `ORA-01017` ; `racdb` still `AVAILABLE` .

This lab's output:

Output what this lab printed; nothing to run

```
=== ORCLCDB1 -> racnode1-0-vip.rac.svc.cluster.local:31522
10.244.1.6      racnode1-0-vip.rac.svc.cluster.local

System altered.


System altered.

=== ORCLCDB2 -> racnode2-0-vip.rac.svc.cluster.local:31523
10.244.2.4      racnode2-0-vip.rac.svc.cluster.local

System altered.


System altered.


   INST_ID VALUE
---------- --------------------------------------------------------------------------------
         1 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
         2 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))


SID        VALUE
---------- --------------------------------------------------------------------------------
*          -oraagent-dummy-
ORCLCDB1   (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
ORCLCDB2   (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))

=== LISTENER_SCAN1
Service "orclpdbsvc" has 2 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
=== LISTENER_SCAN2
Service "orclpdbsvc" has 2 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
ORA-01017: invalid username/password; logon denied
SP2-0751: Unable to connect to Oracle.  Exiting SQL*Plus
command terminated with exit code 1
ORA-01017: invalid username/password; logon denied
SP2-0751: Unable to connect to Oracle.  Exiting SQL*Plus
command terminated with exit code 1
ORA-01017: invalid username/password; logon denied
```

```
SP2-0751: Unable to connect to Oracle.  Exiting SQL*Plus
command terminated with exit code 1
ORA-01017: invalid username/password; logon denied
SP2-0751: Unable to connect to Oracle.  Exiting SQL*Plus
command terminated with exit code 1
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
```

The `-oraagent-dummy-` line: that's the spfile entry for all instances (`*`), and the marker Grid's agent leaves when it manages `local_listener` itself. The new per-instance entries for `ORCLCDB1` and `ORCLCDB2` take precedence over it. 12.5 checks they survive a listener restart.

12.4 Open the ports: NodePort Services, then Windows forwards

What: two NodePort Services in namespace `rac`, `racnode1-0-lsnr-ext` (31522 → `racnode1-0`:31522) and `racnode2-0-lsnr-ext` (31523 → `racnode2-0`:31523), and three NAT forwards on `k8s-public`: 127.0.0.1:31521, 31522, 31523 → op-worker1 (10.0.1.21), same port.

Why each part:

- **Own Services for the node listeners:** the operator's `racnode-scan-lsnr` (31521) only covers SCAN. Each new Service selects one pod by its StatefulSet pod name, as the operator's own Services do.
- **Same number everywhere:** port, `targetPort` and `nodePort` are the number SCAN hands out (12.3), so the redirect works unchanged from Windows.
- **Own label:** `app: rac-ext-access` and none of the operator's labels, so the operator doesn't treat them as its own.
- **Forwards to op-worker1 only:** a NodePort answers on every node and passes the traffic on to the right pod. The rule names start with `op-ora-`, so they're easy to find and remove.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for i in 1 2; do p=$((31521+i)); cat <<EOF
---
apiVersion: v1
kind: Service
metadata:
  name: racnode$i-0-lsnr-ext
  namespace: rac
  labels: { app: rac-ext-access }
spec:
  type: NodePort
  selector: { statefulset.kubernetes.io/pod-name: racnode$i-0 }
  ports:
    - { name: lsnr, protocol: TCP, port: $p, targetPort: $p, nodePort: $p }
EOF
done | kubectl apply -f -
kubectl -n rac get svc racnode-scan-lsnr racnode1-0-lsnr-ext racnode2-0-lsnr-ext
kubectl -n rac get endpoints racnode-scan-lsnr racnode1-0-lsnr-ext racnode2-0-lsnr-ext
sleep 10
for h in 10.0.1.21 10.0.1.22; do for p in 31521 31522 31523; do timeout 3 bash -c "</dev/tcp/$h/$p"
2>/dev/null && echo "$h:$p open" || echo "$h:$p CLOSED"; done; done
echo "--- straight to the pods"
for a in 10.244.1.6:1521 10.244.1.6:31522 10.244.2.4:31523; do timeout 3 bash -c "
</dev/tcp/${a%:*}/${a#*:}" 2>/dev/null && echo "$a open" || echo "$a CLOSED"; done
```

Check: two Services `created`; three NodePorts `1521:31521`, `31522:31522`, `31523:31523`; endpoints `10.244.1.6:1521`, `10.244.1.6:31522`, `10.244.2.4:31523`; all six NodePort lines and all three pod lines `open`.

The first run tested too early. It checked the ports in the same second the Services were created (age `0s`), before kube-proxy had set up the new NodePorts, and 31522 and 31523 were refused. Ten seconds later all were open, and kube-proxy showed no errors; the block above now waits (`sleep 10`) and also tests both workers and the pods directly. A refused connection, unlike a timeout, also rules out a firewall drop.

This lab's output (the first run):

Output what this lab printed; nothing to run

```
service/racnode1-0-lsnr-ext created
service/racnode2-0-lsnr-ext created
NAME                  TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)           AGE
racnode-scan-lsnr     NodePort   10.99.59.33      <none>        1521:31521/TCP    25h
racnode1-0-lsnr-ext   NodePort   10.105.16.139    <none>        31522:31522/TCP   0s
racnode2-0-lsnr-ext   NodePort   10.109.147.162   <none>        31523:31523/TCP   0s
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                  ENDPOINTS          AGE
racnode-scan-lsnr     10.244.1.6:1521    25h
racnode1-0-lsnr-ext   10.244.1.6:31522   0s
racnode2-0-lsnr-ext   10.244.2.4:31523   0s
10.0.1.21:31521 open
bash: connect: Connection refused
bash: line 1: /dev/tcp/10.0.1.21/31522: Connection refused
10.0.1.21:31522 CLOSED
bash: connect: Connection refused
bash: line 1: /dev/tcp/10.0.1.21/31523: Connection refused
10.0.1.21:31523 CLOSED
```

This lab's output (the check ten seconds later, with kube-proxy):

Output what this lab printed; nothing to run

```
10.0.1.21:31521 open
10.0.1.21:31522 open
10.0.1.21:31523 open
10.0.1.22:31521 open
10.0.1.22:31522 open
10.0.1.22:31523 open
--- straight to the pods
10.244.1.6:1521 open
10.244.1.6:31522 open
10.244.2.4:31523 open
NAME               READY   STATUS    RESTARTS   AGE   IP          NODE              NOMINATED NODE
READINESS GATES
kube-proxy-7sbhj   1/1     Running   0          47h   10.0.1.20   op-controlplane   <none>           <none>
kube-proxy-hqvkq   1/1     Running   1          47h   10.0.1.22   op-worker2        <none>           <none>
kube-proxy-smrqj   1/1     Running   1          47h   10.0.1.21   op-worker1        <none>           <none>
no errors in the last kube-proxy log lines
```

(The kube-proxy lines came from `kubectl -n kube-system get pods -l k8s-app=kube-proxy -o wide` and `kubectl -n kube-system logs -l k8s-app=kube-proxy --tail=5 --prefix 2>&1 | grep -iE 'error|fail|31522|31523' || echo "no errors in the last kube-proxy log lines"`, run with the check.)

Then on Windows:

PowerShell on the Windows PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
foreach ($p in 31521, 31522, 31523) {
  & $VB natnetwork modify --netname k8s-public --port-forward-4 "op-ora-$($p):tcp:[]:$($p):
[10.0.1.21]:$($p)"
}
& $VB natnetwork list | Select-String 'Name:|op-ora-'
foreach ($p in 31521, 31522, 31523) { "127.0.0.1:$p  $(Test-NetConnection 127.0.0.1 -Port $p -
InformationLevel Quiet -WarningAction SilentlyContinue)" }
```

Check: three `op-ora-…` rules to `[10.0.1.21]`, and `True` for all three ports. (If a port shows `False`, wait a few seconds and run the last line again.)

This lab's output:

Output what this lab printed; nothing to run

```
Name:         k8s-public
        op-ora-31521:tcp:[]:31521:[10.0.1.21]:31521
        op-ora-31522:tcp:[]:31522:[10.0.1.21]:31522
        op-ora-31523:tcp:[]:31523:[10.0.1.21]:31523

127.0.0.1:31521  True
127.0.0.1:31522  True
127.0.0.1:31523  True
```

12.5 Connect from Windows through SCAN

What: a small Python script opens 10 connections to `127.0.0.1:31521/orclpdbsvc` and prints, for each, which instance, host and container served it.

Why it proves the whole path: each connection goes Windows → forward 31521 → `LISTENER_SCAN1` → redirect to `racnode1-0-vip…:31522` or `racnode2-0-vip…:31523` → Windows `hosts` (127.0.0.1) → that forward → that instance's node listener. A line with an instance name has passed every step.

The password: asked at a hidden prompt (`getpass`); it isn't written to the file, the screen or the history. It's the database password from Phase 9, which DBCA used for SYS, SYSTEM and the PDB admin.

Driver: `python-oracledb` in thin mode needs no Oracle client. Thin mode has no TAF, so failover here means a **new** connection lands on the surviving instance (12.6), not a session that survives unchanged.

PowerShell on the Windows PC

```
@'
import collections, getpass, oracledb

DSN = "127.0.0.1:31521/orclpdbsvc"
SQL = """select sys_context('USERENV','INSTANCE_NAME'),
                sys_context('USERENV','SERVER_HOST'),
                sys_context('USERENV','CON_NAME') from dual"""

pw = getpass.getpass("SYSTEM password: ")
seen = collections.Counter()
for n in range(1, 11):
    with oracledb.connect(user="system", password=pw, dsn=DSN) as conn:
        with conn.cursor() as cur:
            cur.execute(SQL)
            inst, host, con = cur.fetchone()
    seen[inst] += 1
    print(f"{n:2}  {inst:9} {host:11} {con}")
print("oracledb", oracledb.__version__, "thin mode:", oracledb.is_thin_mode())
print("connections per instance:", dict(seen))
'@ | Set-Content -Encoding UTF8 D:\rac-k8s-poc-operator\scan_test.py
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: ten lines with `ORCLCDB1 racnode1-0 ORCLPDB` or `ORCLCDB2 racnode2-0 ORCLPDB`, `thin mode: True`, and both instances in the count. (With the service goal `LONG`, SCAN sends each new connection to the less busy instance, so a run of ten may split unevenly.)

If it fails, the error says where: `DPY-6005` or a timeout at the start → the forward or NodePort on 31521; `DPY-6000` / `ORA-12514` → SCAN doesn't know the service; `DPY-6005` naming `racnodeN-0-vip` → the redirect (`hosts` entry or the forward on 31522 / 31523); `ORA-01017` → everything works except the password.

This lab's output:

Output — what this lab printed; nothing to run

```
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB1  racnode1-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB2  racnode2-0  ORCLPDB
 7  ORCLCDB1  racnode1-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB2  racnode2-0  ORCLPDB
10  ORCLCDB1  racnode1-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 5, 'ORCLCDB1': 5}
```

12.6 Does it hold: listener restart, and new connections when one instance is out

What, in three parts:

- **a. (op-controlplane)** read the RAC pods' health probes; restart the node listeners again as in 12.2 and check that `local_listener` keeps the values from 12.3; take `orclpdbsvc` off ORCLCDB1.
- **b. (Windows)** run the test script: every connection should now land on ORCLCDB2.
- **c. (op-controlplane, then Windows)** put the service back on ORCLCDB1 and run the script again: both instances again.

Why the service and not the whole instance: for new connections, stopping the service on one instance has the effect of losing that instance (SCAN stops sending clients there), without touching the database. Part a also reads the pods' probes first, to know whether stopping an instance could make Kubernetes restart a pod.

Interruption: a few seconds per node for the listener restart (as in 12.2). Without `-force`, stopping the service on ORCLCDB1 leaves existing sessions there running; only new connections go to ORCLCDB2.

a.

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do echo "=== $p probes"; kubectl -n rac get pod $p -o jsonpath='{range
.spec.containers[*]}{.name}{"\n"}  liveness:  {.livenessProbe}{"\n"}  readiness: {.readinessProbe}{"\n"}
startup:   {.startupProbe}{"\n"}{end}'; done
for n in racnode1-0 racnode2-0; do
  echo "=== restart LISTENER on $n"
  kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl stop listener -listener LISTENER -
node $n; srvctl start listener -listener LISTENER -node $n; srvctl status listener -listener LISTENER -
node $n"
  sleep 10
done
sleep 60
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
set lines 200 pages 50
col value for a90
select inst_id, value from gv$parameter where name = 'local_listener' order by 1;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl stop service -d ORCLCDB -s
orclpdbsvc -i ORCLCDB1; srvctl status service -d ORCLCDB -s orclpdbsvc"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "lsnrctl services LISTENER_SCAN1" | awk
'/Service "orclpdbsvc"/{f=1;print;next} /^Service "/{f=0} f && /Instance|ADDRESS/'
```

Check: the probes printed; both listeners restarted; `local_listener` still `racnode1-0-vip…:31522` and `racnode2-0-vip…:31523`; `orclpdbsvc` running only on `ORCLCDB2`, and `LISTENER_SCAN1` listing only `Instance "ORCLCDB2"` for it.

Run it on op-controlplane, not in Git Bash. The first try of this block ran in Git Bash on the PC (the `$` prompt). There, plain `kubectl` uses the default kubeconfig, whose `127.0.0.1:6443` forward leads to another VM that wasn't running, so every call failed with `TLS handshake timeout` and nothing changed. Had that VM been running, the commands would have gone to the wrong cluster; from Git Bash only `kop` reaches this lab.

This lab's output (the first try, in Git Bash; first lines):

Output what this lab printed; nothing to run

```
$ # op-controlplane  (connect: ssh -p 2220 root@127.0.0.1)
…
=== racnode1-0 probes
E1028 22:34:18.848649   90392 memcache.go:265] couldn't get current server API group list: Get
"https://127.0.0.1:6443/api?timeout=32s": net/http: TLS handshake timeout
…
Unable to connect to the server: net/http: TLS handshake timeout
```

This lab's output (on op-controlplane):

Output what this lab printed; nothing to run

```
=== racnode1-0 probes
racnode1-0
  liveness:
  readiness:
{"failureThreshold":3,"initialDelaySeconds":180,"periodSeconds":5,"successThreshold":1,"tcpSocket":
{"port":1522},"timeoutSeconds":1}
  startup:
=== racnode2-0 probes
racnode2-0
  liveness:
  readiness:
{"failureThreshold":3,"initialDelaySeconds":180,"periodSeconds":5,"successThreshold":1,"tcpSocket":
{"port":1522},"timeoutSeconds":1}
  startup:
=== restart LISTENER on racnode1-0
Listener LISTENER is enabled on node(s): racnode1-0
Listener LISTENER is running on node(s): racnode1-0
=== restart LISTENER on racnode2-0
Listener LISTENER is enabled on node(s): racnode2-0
Listener LISTENER is running on node(s): racnode2-0

   INST_ID VALUE
---------- ------------------------------------------------------------------------------------------
         1 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
         2 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))

Service orclpdbsvc is running on instance(s) ORCLCDB2
Service "orclpdbsvc" has 1 instance(s).
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
```

What a shows:

- `local_listener` **survives a listener restart.** Unlike in 12.2, Grid's agent didn't rewrite it: an explicit per-instance value in the spfile is left alone.

- **The pods have only a readiness probe:** a TCP check of port 1522 every 5 s, failing after 3 misses (after a first 180 s). No liveness or startup probe, so Kubernetes never restarts a RAC pod because of a failed check; at most it marks it not ready. A stopped *instance* doesn't even do that, because the probe tests the node listener, not the database. A node listener down for longer than about 15 s does make its pod not ready, and Services may then stop sending traffic to it.

b.

PowerShell on the Windows PC

```
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: all ten connections on `ORCLCDB2 racnode2-0`.

This lab's output:

Output what this lab printed; nothing to run

```
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB2  racnode2-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB2  racnode2-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB2  racnode2-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB2  racnode2-0  ORCLPDB
 9  ORCLCDB2  racnode2-0  ORCLPDB
10  ORCLCDB2  racnode2-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 10}
```

c.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl start service -d ORCLCDB -s
orclpdbsvc -i ORCLCDB1; srvctl status service -d ORCLCDB -s orclpdbsvc"
kubectl -n rac get racdatabase racdb
```

Check: the service on both instances again, `racdb` `AVAILABLE`.

This lab's output:

Output what this lab printed; nothing to run

```
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
```

PowerShell on the Windows PC

```
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: both instances in the count again.

This lab's output:

Output what this lab printed; nothing to run

```
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB1  racnode1-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB1  racnode1-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB1  racnode1-0  ORCLPDB
10  ORCLCDB1  racnode1-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 4, 'ORCLCDB1': 6}
```

Phase 12 result: from Windows, `127.0.0.1:31521/orclpdbsvc` reaches both instances through SCAN, new connections go to the remaining instance when the service is off one of them, and the settings survive a listener restart. What was changed, for reference or to undo:

WHERE	CHANGE	UNDO
Grid (cluster-wide)	`LISTENER` endpoints `TCP:1522/TCP:31522/TCP:31523`	`srvctl modify listener -listener LISTENER -endpoints 'TCP:1522'`, then restart it per node
spfile, per instance	`local_listener` = `racnode1-0-vip…:31522` (ORCLCDB1), `racnode2-0-vip…:31523` (ORCLCDB2)	`alter system reset local_listener scope=spfile sid='ORCLCDB1';` (and `ORCLCDB2`); the agent sets it again at the next instance start
Kubernetes, namespace `rac`	Services `racnode1-0-lsnr-ext` (31522), `racnode2-0-lsnr-ext` (31523), label `app: rac-ext-access`	`kubectl -n rac delete svc -l app=rac-ext-access`

WHERE	CHANGE	UNDO
VirtualBox `k8s-public`	forwards `op-ora-31521`, `op-ora-31522`, `op-ora-31523` → 10.0.1.21	`& $VB natnetwork modify --netname k8s-public --port-forward-4 delete op-ora-31521` (and the other two)
Windows	`D:\rac-k8s-poc-operator\scan_test.py`; the `hosts` lines were already there	delete the file; keep or remove the two `hosts` lines

PHASE 13

Restarting the RAC pods safely

Result of this phase: the RAC pods survive a restart on their own: Grid's local files are restored, Clusterware and the database start, Clusterware moves the VIPs and SCAN to the pods' new IPs, `ORCLPDB` opens on both instances, the Phase 12 access from Windows keeps working, and a guard (`SKIP_DBCA=true`) makes sure Oracle's setup script can't try to create the database again. The phase also records how the operator behaves when its configuration changes, which caused two unplanned restarts here.

Why a restart needs care in this lab:

- **Grid's local files live in the container, not on a volume.** `/etc/oracle` (`ocr.loc`, `olr.loc`), `/etc/oratab`, `/etc/oraInst.loc` and the `oracle-ohasd` unit are on the container's writable layer (`overlay`). A new container, after any pod restart or VM reboot, starts without them. Only `/u01` (the homes, on `/scratch/rac/cluster01/racnodeN`), the stage folder, the Secrets, the env ConfigMap and the ASM disks persist.
- **Every container start runs Oracle's setup script again** (`/usr/sbin/init` → systemd → `runOracle.sh` → `main.py`), with `OP_TYPE=setuprac` on both pods, rebuilt by the operator from the spec on every reconcile.
- **The script restores Grid by itself** (`check_gi_installed` → `restore_gi_files` → `rootcrs.sh -updateosfiles`, then `start_crs`) **but then checks for the database with** `check_dbinst` **, which only counts an instance that is** `OPEN` **at that moment.** Otherwise it goes on to `create_db()`, DBCA. After a restart the instances are often still starting, so without a guard the script could try to create the database again.
- **The guard,** `SKIP_DBCA=true` **,** makes `oraracprov` skip its whole database block (disk-group checks, instance check, DBCA). The scripts read it from the env ConfigMap (`/etc/rac_env_vars/envfile`).
- **The operator rolls both RAC pods at once whenever the env ConfigMap changes.** It stores a hash of that ConfigMap in the pod template; a changed ConfigMap means a changed template, and the operator updates both StatefulSets together. In this lab that happened one to three minutes after each change, with the database `AVAILABLE`; the pods were stopped within Kubernetes' 30 s grace period, with the database open: a hard restart of both nodes.

What happened, in order (times US Eastern, 29 October 2026):

TIME	STEP	WHAT HAPPENED
19:42–19:56	13.1	read-only checks: what persists, what the scripts and the operator do on a restart

TIME	STEP	WHAT HAPPENED
19:57–20:00	13.2	backups (`RacDatabase` , OCR, pfile), then a clean stop: database, then Clusterware on both pods
20:02–20:10	13.3	`SKIP_DBCA` added to the `RacDatabase` `envVars` ; the operator did nothing visible while the pods were not ready; Clusterware started again, database kept closed
20:14	13.3	database started by hand; outage of the planned stop: about 16 minutes
20:15–20:17	13.4	**the first patch of the ConfigMaps was faulty** (`CPU_COUNT=3SKIP_DBCA=true`); the changed ConfigMap made the operator roll both pods at 20:16:28, with the database open
20:18–20:21	13.4	setup script stopped on racnode1-0 before its DBCA check (no guard in effect); Clusterware had moved the VIPs and SCAN to the new pod IPs; both instances open, `ORCLPDB` only mounted on instance 1
20:22	13.5	`ORCLPDB` opened and its state saved for both instances
20:27–20:36	13.6	ConfigMaps repaired from the backup with `SKIP_DBCA=true` on its own line; the operator rolled both pods again at about 20:28:30; everything came back by itself, guard in effect, no DBCA

13.1 What starts on its own, and what keeps state (read-only)

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl get nodes; kubectl -n rac get racdatabase racdb; kubectl -n rac get pods -o wide
echo "--- StatefulSets: start order, grace period"
kubectl -n rac get sts racnode1 racnode2 -o custom-
columns='NAME:.metadata.name,POLICY:.spec.podManagementPolicy,UPDATE:.spec.updateStrategy.type,GRACE:.spe
c.template.spec.terminationGracePeriodSeconds'
echo "--- what the container runs"
kubectl -n rac get pod racnode1-0 -o jsonpath='command: {.spec.containers[0].command}{"\n"}args:
{.spec.containers[0].args}{"\n"}'
for p in racnode1-0 racnode2-0; do
  echo "=== $p"
  kubectl -n rac exec $p -c $p -- bash -c 'echo "statefile: $(cat /tmp/orod/.statefile 2>/dev/null)";
findmnt -T /tmp/orod -no TARGET,SOURCE,FSTYPE; findmnt -T /u01 -no TARGET,SOURCE,FSTYPE; systemctl is-
enabled oracle-ohasd 2>&1; /u01/app/19c/grid/bin/crsctl config crs'
done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl config database -d ORCLCDB | grep
-E 'Management policy|Start options|Stop options|Database instances'; srvctl config service -d ORCLCDB -s
orclpdbsvc | grep -E 'Management policy'"
kubectl -n oracle-database-operator-system get deploy,pods
```

PowerShell on the Windows PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
"--- VM states"; foreach ($v in "op-controlplane","op-worker1","op-worker2") { "$v  $((& $VB showvminfo
$v --machinereadable | Select-String '^VMState=') -replace 'VMState=','')" }
foreach ($p in 2220, 2221, 2222) { "--- port $p"; ssh -p $p root@127.0.0.1 "hostname; systemctl is-
enabled kubelet crio; grep HugePages_Total /proc/meminfo; cat /etc/sysctl.d/92-hugepages.conf
2>/dev/null; ip -br link show | grep -E 'enp0s[89]' | grep -o 'enp0s[89].*PROMISC' || true" }
```

This lab's output (cluster):

Output what this lab printed; nothing to run

```
NAME              STATUS   ROLES           AGE     VERSION
op-controlplane   Ready    control-plane   2d20h   v1.36.5
op-worker1        Ready    worker          2d20h   v1.36.5
op-worker2        Ready    worker          2d20h   v1.36.5
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   0          47h   10.244.1.6   op-worker1   <none>           <none>
racnode2-0   1/1     Running   0          47h   10.244.2.4   op-worker2   <none>           <none>
--- StatefulSets: start order, grace period
NAME       POLICY         UPDATE          GRACE
racnode1   OrderedReady   RollingUpdate   30
racnode2   OrderedReady   RollingUpdate   30
--- what the container runs
command: ["/usr/sbin/init"]
args:
=== racnode1-0
statefile: completed
/tmp   tmpfs  tmpfs
/u01   /dev/sdb[/rac/cluster01/racnode1] xfs
enabled
CRS-4622: Oracle High Availability Services autostart is enabled.
=== racnode2-0
statefile: provisioning
/tmp   tmpfs  tmpfs
/u01   /dev/sdb[/rac/cluster01/racnode2] xfs
enabled
CRS-4622: Oracle High Availability Services autostart is enabled.
Start options: open
Stop options: immediate
Management policy: AUTOMATIC
Database instances: ORCLCDB1,ORCLCDB2
Management policy: AUTOMATIC
NAME                                                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/oracle-database-operator-controller-manager   1/1     1            1           2d11h

NAME                                                         READY   STATUS    RESTARTS   AGE
pod/oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running   1          2d11h
```

This lab's output (Windows):

Output what this lab printed; nothing to run

```
--- VM states
op-controlplane  "running"
op-worker1  "running"
op-worker2  "running"
--- port 2220
op-controlplane
enabled
enabled
HugePages_Total:       0
--- port 2221
op-worker1
enabled
enabled
HugePages_Total:    2048
vm.nr_hugepages = 2048
enp0s8           UP             08:00:27:c3:37:02 <BROADCAST,MULTICAST,PROMISC
enp0s9           UP             08:00:27:43:bd:30 <BROADCAST,MULTICAST,PROMISC
--- port 2222
op-worker2
enabled
enabled
HugePages_Total:    2048
vm.nr_hugepages = 2048
enp0s8           UP             08:00:27:9d:88:72 <BROADCAST,MULTICAST,PROMISC
enp0s9           UP             08:00:27:9e:48:85 <BROADCAST,MULTICAST,PROMISC
```

What it shows: the VM layer survives reboots (kubelet and CRI-O enabled, hugepages from `/etc/sysctl.d`, promiscuous NICs). Clusterware and the database start by themselves once Grid runs (`ohasd` enabled, CRS autostart, `AUTOMATIC` policy). Kubernetes gives a pod **30 s** to stop, too short for RAC, so the database and Clusterware must be stopped by hand before a planned restart. `/tmp` is a tmpfs, so the setup script's state file is lost on a restart; `OrderedReady` doesn't matter, because each StatefulSet has one pod.

What persists, and where Grid's local files are (read-only):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac get pod racnode1-0 -o jsonpath='{range .spec.containers[0].volumeMounts[*]}{.mountPath}{"
<-  "}{.name}{"\n"}{end}'
kubectl -n rac get pod racnode1-0 -o jsonpath='{range .spec.volumes[*]}{.name}{"  "}{.hostPath.path}
{.emptyDir}{.persistentVolumeClaim.claimName}{.secret.secretName}{"\n"}{end}'
for p in racnode1-0 racnode2-0; do
  echo "=== $p: where Grid's local files live"
  kubectl -n rac exec $p -c $p -- bash -c 'for f in /etc/oracle /etc/oratab /etc/oraInst.loc
/etc/systemd/system/oracle-ohasd.service /etc/init.d/init.ohasd; do printf "%-45s " $f; if [ -e $f ];
then findmnt -T $f -no TARGET,SOURCE | head -1; else echo MISSING; fi; done; ls /etc/oracle'
done
```

This lab's output (racnode2-0 the same):

Output what this lab printed; nothing to run

```
/mnt/.ssh  <-  racnode1-ssh-secretmap-vol
/mnt/.dbsecrets  <-  racnode1-dbsecret-pwd-vol
/dev/shm  <-  racnode1-oradshm-vol
/etc/rac_env_vars  <-  racnode1-oradata-envfile
/etc/rac_env_vars_writable  <-  racnode1-oradata-envfile-writable
/u01  <-  racnode1-oradata-sw-vol
/mnt/stage/software  <-  racnode1-oradata-swstage-vol
/var/run/secrets/kubernetes.io/serviceaccount  <-  kube-api-access-cbkq5
racnode1-ssh-secretmap-vol  ssh-key-secret
racnode1-oradshm-vol  {"medium":"Memory"}
racnode1-dbsecret-pwd-vol  db-user-pass
racnode1-oradata-envfile
racnode1-oradata-sw-vol  /scratch/rac/cluster01/racnode1
racnode1-oradata-swstage-vol  /scratch/software/stage
asm-pvc-833f4ea4-racdb  asm-pvc-833f4ea4-racdb
asm-pvc-eaac2e15-racdb  asm-pvc-eaac2e15-racdb
asm-pvc-c51c7cc4-racdb  asm-pvc-c51c7cc4-racdb
racnode1-oradata-envfile-writable  {}
kube-api-access-cbkq5
=== racnode1-0: where Grid's local files live
/etc/oracle                                 /      overlay
/etc/oratab                                 /      overlay
/etc/oraInst.loc                            /      overlay
/etc/systemd/system/oracle-ohasd.service    /      overlay
/etc/init.d/init.ohasd                      /      overlay
lastgasp
maps
ocr.loc
ocr.loc.orig
olr.loc
olr.loc.orig
oprocd
scls_scr
setasmgid
```

What Oracle's setup script does on a restart. These read-only blocks print the relevant parts of the scripts in the image (`/opt/scripts/startup/scripts`). The excerpts below are cut to the lines that matter (... marks cut text).

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'cd /opt/scripts/startup/scripts
sed -n "1925,2070p" oracommon.py
grep -n -A30 "def check_dbinst" oracommon.py | head -45
grep -n -A25 "def get_inst_sid" oracommon.py | head -35
sed -n "80,125p" oraracprov.py
grep -n "GI_HOME_CONFIGURED_FLAG" *.py | head'
```

This lab's output (excerpts):

Output what this lab printed; nothing to run

```
    def check_gi_installed(self, retcode1, gihome, giuser, node, oinv):
        if retcode1 == 0:
            if os.path.isdir("/etc/oracle"):
                bstr = "Grid is already installed on this machine and /etc/oracle also exist. Skipping Grid
setup.."
                …
                return True
            else:
                …
                    status = self.check_home_inv(None, gihome, giuser)
                    if status:
                        status = self.restore_gi_files(gihome, giuser)
                        …
                            self.run_orainstsh_local(giuser, node, oinv)
                            if self.check_key("CRS_GPC", self.ora_env_dict):
                                return True
                            else:
                                status = self.start_crs(gihome, giuser)
…
    def restore_gi_files(self, gihome, giuser):
        …
        srcdir = "{0}/.etcoraclebackup".format(gibase)
        …
        # 1. Restore /etc/oracle if backup exists
        …
            self.log_info_message("No /etc/oracle backup found; skipping restore", self.file_name)
        # 2. Update OS files (always)
        …
            cmd = "{0}/crs/install/rootcrs.sh -updateosfiles".format(gihome)
…
    def check_dbinst(self):
        …
        inst_sid = self.get_inst_sid(osuser, dbhome, osid, hostname)
        …
        if inst_sid:
            status = self.get_dbinst_status(osuser, dbhome, inst_sid, connect_str)
            if not self.check_substr_match(status, "OPEN"):
                return False, inst_sid, hostname, status
…
    def get_inst_sid(self, dbuser, dbhome, osid, hostname):
        …
        cmd = '''su - {5} -c "… {0}/bin/srvctl status database -d {3} | grep {4}"'''…
…
       if not self.ocommon.check_key("SKIP_DBCA",self.ora_env_dict):
            self.create_asmdg()
            status,osid,host,mode=self.ocommon.check_dbinst()
            …
            if status:
```

```
                msg='''Database instance {0} already exists on this machine {1}.'''.format(osid,hostname)
                self.ocommon.update_statefile("completed")
            …
            else:
                …
                self.create_db()
…
orasetupenv.py:945:
self.ora_env_dict=self.ocommon.add_key("GI_HOME_CONFIGURED_FLAG","true",self.ora_env_dict)
```

(The 19c rule that skips the `/etc/oracle` backup is in `oragiprov.py`: *"Oracle 19c detected and CRS_GPC not set; skipping /etc/oracle backup"*. So on this lab the restore is always `rootcrs.sh -updateosfiles`.)

Where `OP_TYPE` comes from, and what the operator does with changes (read-only):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do echo "=== $p"; kubectl -n rac exec $p -c $p -- grep -E '^(export )?
(OP_TYPE|SKIP_DBCA|CRS_GPC|INSTALL_NODE|CRS_NODES|DB_NAME)=' /etc/rac_env_vars/envfile; done
for p in racnode1-0 racnode2-0; do kubectl -n rac get pod $p -o jsonpath='{range .spec.volumes[?
(@.configMap)]}{.name}{"  ->  configMap "}{.configMap.name}{"\n"}{end}'; done
cd /root/oraop/controllers/database
sed -n "4095,4135p" racdatabase_controller.go
n=$(grep -n "func (r \*RacDatabaseReconciler) createOrReplaceSfs(" racdatabase_controller.go | cut -d: -
f1); sed -n "${n},$((n+25))p" racdatabase_controller.go
```

This lab's output (excerpts):

Output what this lab printed; nothing to run

```
=== racnode1-0
CRS_NODES=pubhost:racnode1-0,viphost:racnode1-0-vip;pubhost:racnode2-0,viphost:racnode2-0-vip
OP_TYPE=setuprac
INSTALL_NODE=racnode1-0
DB_NAME=ORCLCDB
=== racnode2-0
(the same)
racnode1-oradata-envfile  ->  configMap racnode1racdb-cmap
racnode2-oradata-envfile  ->  configMap racnode2racdb-cmap
…
        if cfg.OpType == "" {
                env.AddKV("OP_TYPE", "setuprac")
        }
…
func appendRACSpecEnvVars(instance *racdb.RacDatabase, env *racEnvAccumulator) {
        for _, e := range instance.Spec.EnvVars {
                env.AddKV(e.Name, e.Value)
        }
}
…
func (r *RacDatabaseReconciler) createOrReplaceSfs(
        …
        configMapHash, err := r.computeRACConfigMapHash(ctx, racDatabase.Namespace, &dep.Spec.Template)
        …
        dep.Spec.Template.Annotations[racConfigMapHashAnnotation] = configMapHash
        result, err := sharedk8sobjects.ReconcileStatefulSet(ctx, r.Client, racDatabase.Namespace, dep, …
```

What that means: `OP_TYPE` is `setuprac` on both pods and comes back on every reconcile unless `configParams.opType` is set. `SKIP_DBCA` appears nowhere in the operator; only Oracle's scripts read it. The operator puts a hash of each env ConfigMap into its pod template, so **a changed ConfigMap rolls the pods**.

13.2 Save the current state, then stop the database and Clusterware

a. Backups (no outage): the `RacDatabase` as the cluster has it, a copy of the YAML, a manual OCR backup, and a pfile on `/u01`.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac get racdatabase racdb -o yaml > /root/rac-op/racdb-before-13.yaml
cp -p /root/rac-op/racdb.yaml /root/rac-op/racdb.yaml.before-13
ls -l /root/rac-op/racdb-before-13.yaml /root/rac-op/racdb.yaml.before-13
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c '/u01/app/19c/grid/bin/ocrconfig -manualbackup;
/u01/app/19c/grid/bin/ocrconfig -showbackup manual'
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
create pfile='/u01/app/oracle/pfile-ORCLCDB-before-13.ora' from spfile;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'ls -l /u01/app/oracle/pfile-ORCLCDB-before-
13.ora; grep -E "local_listener|control_files|db_recovery" /u01/app/oracle/pfile-ORCLCDB-before-13.ora'
```

This lab's output:

Output what this lab printed; nothing to run

```
-rw-r--r--. 1 root root 10285 Oct 29 19:57 /root/rac-op/racdb-before-13.yaml
-rw-r--r--. 1 root root  2257 Oct 27 20:22 /root/rac-op/racdb.yaml.before-13

racnode1-0     2026/10/29 23:57:58     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261029_235758.ocr.263.1247961479
1753837251

racnode1-0     2026/10/29 23:57:58     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261029_235758.ocr.263.1247961479
1753837251

File created.

-rw-r--r--. 1 oracle asmadmin 2497 Oct 29 23:58 /u01/app/oracle/pfile-ORCLCDB-before-13.ora
*.control_files='+DATA/ORCLCDB/CONTROLFILE/current.278.1247831575','+FRA/ORCLCDB/CONTROLFILE/current.267.1247
831577'#Restore Controlfile
*.db_recovery_file_dest='+FRA'
*.db_recovery_file_dest_size=23895m
*.local_listener='-oraagent-dummy-'
ORCLCDB1.local_listener='(ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))'
ORCLCDB2.local_listener='(ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))'
```

b. Clean stop: the database first (stop option `immediate`), then Clusterware on racnode2-0, then racnode1-0.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl stop database -d ORCLCDB; srvctl status database -d ORCLCDB"
for p in racnode2-0 racnode1-0; do
  echo "=== crsctl stop crs on $p"
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl stop crs
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl check crs
done
sleep 20
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
```

This lab's output (the `crsctl stop crs` lines cut):

Output what this lab printed; nothing to run

```
Instance ORCLCDB1 is not running on node racnode1-0
Instance ORCLCDB2 is not running on node racnode2-0
=== crsctl stop crs on racnode2-0
CRS-2791: Starting shutdown of Oracle High Availability Services-managed resources on 'racnode2-0'
CRS-2673: Attempting to stop 'ora.crsd' on 'racnode2-0'
…
CRS-2793: Shutdown of Oracle High Availability Services-managed resources on 'racnode2-0' has completed
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
=== crsctl stop crs on racnode1-0
…
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
NAME                          READY   STATUS        RESTARTS   AGE   IP           NODE         NOMINATED NODE
READINESS GATES
disk-check-daemonset-fstch    1/1     Terminating   0          15s   10.244.1.8   op-worker1   <none>
<none>
disk-check-daemonset-mcvbh    1/1     Terminating   0          15s   10.244.2.6   op-worker2   <none>
<none>
racnode1-0                    0/1     Running       0          47h   10.244.1.6   op-worker1   <none>
<none>
racnode2-0                    0/1     Running       0          47h   10.244.2.4   op-worker2   <none>
<none>
NAME    DBNAME    DBSTATE         ROLE            VERSION         PDB CONNECT STR   STATE
racdb   ORCLCDB   NOTAVAILABLE    NOTAVAILABLE    NOTAVAILABLE    NOTAVAILABLE      PENDING
```

What it shows: every resource stopped cleanly; the pods dropped to `0/1` (the readiness probe checks the listener on 1522) but kept running; the operator noticed (a new disk-check run, `PENDING`) without changing anything.

13.3 `SKIP_DBCA` in the `RacDatabase` (no effect on this cluster)

a. Preview. `kubectl diff` showed that the file on op-controlplane lacked `accessMode: ReadWriteMany` on the three disk groups, which the live object has. Applying that would have been a disk-group change, which sends

the operator down its storage path; the file was fixed first so `SKIP_DBCA` was the only change.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/rac-op
grep -n 'SKIP_DBCA' racdb.yaml || sed -i '/name: IGNORE_DB_PREREQS/a\    - { name: SKIP_DBCA, value: "true" }' racdb.yaml
grep -q 'accessMode: ReadWriteMany' racdb.yaml || sed -i -E '/type: (CRSDG|DBDATAFILESDG|DBRECOVERY)/s/redundancy: EXTERNAL,/redundancy: EXTERNAL, accessMode: ReadWriteMany,/' racdb.yaml
kubectl diff -f racdb.yaml; echo "(kubectl diff exit $?)"
kubectl apply --dry-run=server -f racdb.yaml
```

This lab's output (first diff, before the `accessMode` fix; excerpt):

Output what this lab printed; nothing to run

```
-  generation: 1
+  generation: 2
…
   asmDiskGroupDetails:
-  - accessMode: ReadWriteMany
-    disks:
+  - disks:
     - /dev/disk/by-id/ata-VBOX_HARDDISK_VB67cb0647-80696c8f
…
+  - name: SKIP_DBCA
+    value: "true"
```

This lab's output (after the fix):

Output what this lab printed; nothing to run

```
-  generation: 1
+  generation: 2
…
   - name: IGNORE_DB_PREREQS
     value: "true"
+  - name: SKIP_DBCA
+    value: "true"
…
(kubectl diff exit 1)
racdatabase.database.oracle.com/racdb configured (server dry run)
```

(The live object has no `CRS_ASM_DISCOVERY_STRING` either: this build's first apply didn't have it, and 11.4 set the discovery string with `asmcmd dsset`, which is stored in the cluster profile and the ASM spfile.)

b. Apply. The operator accepted it (`generation 2 observed 2`) but didn't write the ConfigMaps or roll the pods. While the pods weren't ready, each reconcile stopped at *"unable to validate RAC pod. The pod not ready is:*

racnode1-0" and retried every 30 s.

Output what this lab printed; nothing to run

```
racdatabase.database.oracle.com/racdb configured
--- racnode1racdb-cmap
OP_TYPE=setuprac
--- racnode2racdb-cmap
OP_TYPE=setuprac
…
generation 2  observed 2
state PENDING
ReconcileComplete=True  LastReconcileCycleCompleted: no reconcile errors
ReconcileWaiting=True  LastReconcileCycleWaiting: RAC pods pending: [racnode1-0]
…  RAC topology update encountered a non-fatal issue       {"racDatabase": "racdb", "error": "unable to
validate RAC pod. The  pod not ready  is: racnode1-0"}
```

c. Clusterware started again, database kept closed. Both pods turned ready about a minute later (20:08:12); for 2½ minutes after that the ConfigMaps still had no `SKIP_DBCA`. Later checks showed why: for this `instanceDetails`-style spec, `envVars` reach neither the ConfigMaps nor the container environment after the first creation. **Changing `envVars` on an existing database has no effect here.** The database was then started by hand:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl start
crs; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl start database -d ORCLCDB; srvctl
status database -d ORCLCDB -v; srvctl status service -d ORCLCDB"
```

Output what this lab printed; nothing to run

```
CRS-4123: Oracle High Availability Services has been started.
CRS-4123: Oracle High Availability Services has been started.
…
Instance ORCLCDB1 is running on node racnode1-0 with online services orclpdbsvc. Instance status: Open.
Instance ORCLCDB2 is running on node racnode2-0 with online services orclpdbsvc. Instance status: Open.
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
```

13.4 A faulty ConfigMap patch, and the unplanned restart it caused

What went wrong: the first patch of the two ConfigMaps (20:15) was meant to add `SKIP_DBCA=true` as its own line. The `envfile` has no newline at its end, and the line that should have added one, `printf '%sSKIP_DBCA=true\n' "$( … && echo)"`, added nothing: command substitution drops the newline it produces. The text was glued onto the last line, `CPU_COUNT=3SKIP_DBCA=true`, in both ConfigMaps. Don't use that block; 13.6 has the correct one.

What it caused: the changed ConfigMaps changed the operator's hash; at 20:16:28 it updated both StatefulSets and Kubernetes deleted both RAC pods together, with the database open:

Output what this lab printed; nothing to run

```
2026-10-30T00:16:28Z   SuccessfulDelete        racnode1                    statefulset-controller   Delete
Pod racnode1-0 in StatefulSet racnode1 successful
2026-10-30T00:16:29Z   Killing                 racnode1-0                  kubelet                  Stopping
container racnode1-0
2026-10-30T00:16:29Z   SuccessfulDelete        racnode2                    statefulset-controller   Delete
Pod racnode2-0 in StatefulSet racnode2 successful
2026-10-30T00:16:29Z   Killing                 racnode2-0                  kubelet                  Stopping
container racnode2-0
2026-10-30T00:17:01Z   SuccessfulCreate        racnode2                    statefulset-controller   Create
Pod racnode2-0 in StatefulSet racnode2 successful
2026-10-30T00:17:01Z   SuccessfulCreate        racnode1                    statefulset-controller   Create
Pod racnode1-0 in StatefulSet racnode1 successful
…  INFO    controllers.controllers.RacDatabase     Updating StatefulSet to desired spec    {…
"StatefulSet.Name": "racnode1"}
…  INFO    controllers.controllers.RacDatabase     Updating StatefulSet to desired spec    {…
"StatefulSet.Name": "racnode2"}
```

The new pods had no guard (`SKIP_DBCA` was part of `CPU_COUNT` 's value), so the setup script on racnode1-0 was stopped once its Grid part was done and before it reached the database check:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c '
L=/tmp/orod/oracle_db_setup.log
grep -n -E "restore_gi_files|updateosfiles|Starting Oracle Clusterware|already configured|create_db|dbca -silent" $L | tail -12
if grep -qE "create_db :: start|dbca -silent" $L; then echo "STOP: DBCA has already started; not killing anything"; exit 0; fi
P=$(ps -eo pid,args | awk "\$2 ~ /python/ && /main\\.py/ && !/check/ {print \$1}")
echo "setup process(es): $P"; [ -n "$P" ] && kill $P; sleep 5
echo "--- still running after the kill (should be empty):"; ps -eo pid,etime,args | awk "\$3 ~ /python/ && /main\\.py/ && !/check/"
echo "--- last log lines"; tail -3 $L'
```

Output — what this lab printed; nothing to run

```
3356:10/30/2026 12:17:40 AM     INFO:   oracommon   -   restoregifiles   :restore_gi_files invoked (Oracle major version: 19)
3359:10/30/2026 12:17:40 AM     INFO:   oracommon   -   executeprocess   :Received Command : /u01/app/19c/grid/crs/install/rootcrs.sh -updateosfiles
3421:          Grid is already configured on this machine
setup process(es): 56
--- still running after the kill (should be empty):
--- last log lines
…
10/30/2026 12:19:16 AM     INFO:  oraracprov   -     cluchecks      :CVU checks not ready yet on node racnode1-0. This is expected after pod restart. Retrying in 30 seconds...
```

What came back by itself anyway: Clusterware, ASM, the listeners, the VIPs and SCAN, now on the new pod IPs, and the database (`AUTOMATIC`):

Output what this lab printed; nothing to run

```
=== racnode1-0 eth0: 10.244.1.9/24
=== racnode2-0 eth0: 10.244.2.7/24
…
ora.orclcdb.db
      1        ONLINE  OFFLINE      racnode1-0               STARTING
      2        ONLINE  ONLINE       racnode2-0
Open,HOME=/u01/app/oracle/product/19c/dbhome_1,STABLE
…
VIP IPv4 Address: 10.244.1.9
VIP IPv4 Address: 10.244.2.7
SCAN 1 IPv4 VIP: 10.244.1.9
SCAN 2 IPv4 VIP: 10.244.2.7
  racnode1-0-vip -> 10.244.1.9
  racnode2-0-vip -> 10.244.2.7
  racnode-scan -> 10.244.2.7 10.244.1.9
…
20:21:16  Instance ORCLCDB1 is running on node racnode1-0 Instance ORCLCDB2 is running on node racnode2-0
…
         1 ORCLPDB    MOUNTED
         2 ORCLPDB    READ WRITE
```

13.5 Open the PDB on both instances and save that state

`ORCLPDB` opened automatically only where its open state was saved. Saving it for both instances makes it open by itself after every restart:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
alter pluggable database ORCLPDB open instances=all;
alter pluggable database ORCLPDB save state instances=all;
set lines 200
col con_name for a10
col instance_name for a12
select inst_id, name, open_mode from gv$pdbs where name = 'ORCLPDB' order by 1;
select con_name, instance_name, state from dba_pdb_saved_states order by 2;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl start service -d ORCLCDB -s
orclpdbsvc -i ORCLCDB1 2>&1; srvctl status service -d ORCLCDB"
```

This lab's output:

Output what this lab printed; nothing to run

```
Pluggable database altered.

Pluggable database altered.

   INST_ID NAME       OPEN_MODE
---------- ---------- ----------
         1 ORCLPDB    READ WRITE
         2 ORCLPDB    READ WRITE

CON_NAME   INSTANCE_NAM STATE
---------- ------------ --------------
ORCLPDB    ORCLCDB1     OPEN
ORCLPDB    ORCLCDB2     OPEN

PRCR-1120 : Resources are already running.
CRS-5702: Resource 'ora.orclcdb.orclpdbsvc.svc' is already running on 'racnode1-0'
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
```

13.6 The guard, done right, and a restart that needs no help

What: each `envfile` becomes exactly the 20:15 backup plus the line `SKIP_DBCA=true`, with a newline added first if the backup lacks one. The block checks the result (41 keys, the last two lines `CPU_COUNT=3` and `SKIP_DBCA=true`) before patching. **It restarts both RAC pods:** about 2½ minutes later the operator rolls them, as in 13.4. That is the one restart the guard costs; after it the ConfigMaps don't change again.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/rac-op
for c in racnode1racdb-cmap racnode2racdb-cmap; do
  b=$(mktemp); f=$(mktemp)
  kubectl create --dry-run=client -f cm-$c-before-13.4.yaml -o jsonpath='{.data.envfile}' > $b
  { cat $b; [ -n "$(tail -c1 $b)" ] && echo; echo "SKIP_DBCA=true"; } > $f
  echo "--- $c: backup $(grep -c '=' $b) keys -> new $(grep -c '=' $f) keys; last lines:"; tail -2 $f |
cat -A
  if [ "$(grep -c '=' $f)" = "41" ] && [ "$(tail -2 $f | head -1)" = "CPU_COUNT=3" ] && [ "$(tail -1 $f)"
= "SKIP_DBCA=true" ]; then
    kubectl -n rac patch cm $c --type merge -p "$(python3 -c 'import json,sys; print(json.dumps({"data":
{"envfile":open(sys.argv[1]).read()}}))' $f)"
  else echo "STOP: $c not as expected, not patched"; fi
  rm -f $b $f
done
for i in $(seq 1 40); do
  echo "=== $(date +%T)  racdb: $(kubectl -n rac get racdatabase racdb -o jsonpath='{.status.state}')"
  kubectl -n rac get pods --no-headers -o custom-
columns='POD:.metadata.name,READY:.status.containerStatuses[0].ready,STARTED:.metadata.creationTimestamp,
IP:.status.podIP' | grep racnode
  sleep 20
done
```

(The backups `cm-racnode1racdb-cmap-before-13.4.yaml` and `cm-racnode2racdb-cmap-before-13.4.yaml` were taken at the start of 13.4 with `kubectl -n rac get cm <name> -o yaml`.)

This lab's output (watch lines cut):

Output what this lab printed; nothing to run

```
--- racnode1racdb-cmap: backup 40 keys -> new 41 keys; last lines:
CPU_COUNT=3$
SKIP_DBCA=true$
configmap/racnode1racdb-cmap patched
--- racnode2racdb-cmap: backup 40 keys -> new 41 keys; last lines:
CPU_COUNT=3$
SKIP_DBCA=true$
configmap/racnode2racdb-cmap patched
=== 20:27:10  racdb: AVAILABLE
racnode1-0   true   2026-10-30T00:17:01Z   10.244.1.9
racnode2-0   true   2026-10-30T00:17:01Z   10.244.2.7
…
=== 20:29:11  racdb: AVAILABLE
racnode1-0   false   2026-10-30T00:28:58Z   10.244.1.10
racnode2-0   false   2026-10-30T00:28:58Z   10.244.2.8
…
=== 20:32:13  racdb: AVAILABLE
racnode1-0   false   2026-10-30T00:28:58Z   10.244.1.10
racnode2-0   true    2026-10-30T00:28:58Z   10.244.2.8
…
=== 20:32:53  racdb: AVAILABLE
racnode1-0   true   2026-10-30T00:28:58Z   10.244.1.10
racnode2-0   true   2026-10-30T00:28:58Z   10.244.2.8
```

Check after the restart (read-only):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode1-0 racnode2-0; do echo "=== $p sees: $(kubectl -n rac exec $p -c $p -- grep -E
'^(SKIP_DBCA|CPU_COUNT|OP_TYPE)=' /etc/rac_env_vars/envfile | tr '\n' ' ')"; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'L=/tmp/orod/oracle_db_setup.log; echo
"statefile: $(cat /tmp/orod/.statefile)"; grep -n -E "restore_gi_files|updateosfiles|already
configured|All required CVU checks|SKIP_DBCA|create_db|dbca -silent|Total time for setup|ERROR|FATAL" $L
| tail -15'
for i in $(seq 1 20); do r=$(kubectl -n rac exec racnode1-0 -c racnode1-0 -- bash -c 'ps -eo
pid,etime,args | awk "\$3 ~ /python/ && /main\\.py/ && !/check/"'); [ -z "$r" ] && { echo "$(date +%T)
setup finished"; break; }; echo "$(date +%T) still running: $r"; sleep 30; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB; srvctl
status service -d ORCLCDB"
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
set lines 200 pages 50
col value for a90
col name for a10
select inst_id, name, open_mode from gv$pdbs where name = 'ORCLPDB' order by 1;
select inst_id, value from gv$parameter where name = 'local_listener' order by 1;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl config nodeapps | grep 'VIP IPv4';
srvctl config scan | grep 'IPv4 VIP'"
kubectl -n rac get sts racnode1 racnode2 -o custom-
columns='NAME:.metadata.name,CURRENT:.status.currentRevision,UPDATE:.status.updateRevision'
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
```

PowerShell on the Windows PC

```
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: both pods see `SKIP_DBCA=true`; the setup log shows the restore and Clusterware, *"Grid is already configured"*, the CVU checks passing, **no** `create_db` / `dbca -silent`, and a final `Total time for setup()`; both instances running, `ORCLPDB` `READ WRITE` on both, `local_listener` unchanged, the service on both; VIPs and SCAN on the new IPs; `CURRENT` = `UPDATE`; both instances from Windows.

This lab's output:

Output what this lab printed; nothing to run

```
=== racnode1-0 sees: OP_TYPE=setuprac CPU_COUNT=3 SKIP_DBCA=true
=== racnode2-0 sees: OP_TYPE=setuprac CPU_COUNT=3 SKIP_DBCA=true
statefile: provisioning
87:10/30/2026 12:29:01 AM     INFO:   oracommon   - populateracenvvars :SKIP_DBCA key is populated: true
…
199:10/30/2026 12:29:05 AM     INFO:   oracommon   - populateracenvvars :SKIP_DBCA key exists with value true
295:10/30/2026 12:29:05 AM    ERROR:  orasetupenv  -   setupsoftware    :oracle gi home directory is not
empty. skipping software unzipping...
334:10/30/2026 12:29:06 AM    ERROR:  orasetupenv  -   setupsoftware    :oracle db home directory is not
empty. skipping software unzipping...
3360:10/30/2026 12:29:35 AM     INFO:   oracommon   -   restoregifiles   :restore_gi_files invoked (Oracle
major version: 19)
3363:10/30/2026 12:29:35 AM     INFO:   oracommon   -   executeprocess   :Received Command :
/u01/app/19c/grid/crs/install/rootcrs.sh -updateosfiles
3413:10/30/2026 12:29:58 AM     INFO:  oramachine   -       setup        :Total time for setup() = [ 56.742 ]
seconds
3425:          Grid is already configured on this machine
3434:10/30/2026 12:30:13 AM     INFO:   oragiprov   -       setup        :Total time for setup() = [ 15.098 ]
seconds
20:34:20 still running:      52       05:19 python3 main.py
…
20:35:51 setup finished
3780:10/30/2026 12:33:51 AM     INFO:  oraracprov   -     cluchecks      :All required CVU checks passed
successfully on node racnode1-0
3840:10/30/2026 12:35:18 AM     INFO:  oraracprov   -     cluchecks      :All required CVU checks passed
successfully on node racnode2-0
3873:10/30/2026 12:35:26 AM     INFO:  oraracprov   -       setup        :Total time for setup() = [ 328.67 ]
seconds
Instance ORCLCDB1 is running on node racnode1-0
Instance ORCLCDB2 is running on node racnode2-0
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2

   INST_ID NAME       OPEN_MODE
---------- ---------- ----------
         1 ORCLPDB    READ WRITE
         2 ORCLPDB    READ WRITE

   INST_ID VALUE
---------- --------------------------------------------------------------------------------
         1 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
         2 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))

VIP IPv4 Address: 10.244.1.10
VIP IPv4 Address: 10.244.2.8
SCAN 1 IPv4 VIP: 10.244.1.10
SCAN 2 IPv4 VIP: 10.244.2.8
NAME       CURRENT               UPDATE
racnode1   racnode1-75fbb798ff   racnode1-75fbb798ff
```

```
racnode2   racnode2-c48dbfc47    racnode2-c48dbfc47
NAME         READY   STATUS    RESTARTS   AGE     IP            NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   0          4m46s   10.244.1.10   op-worker1   <none>           <none>
racnode2-0   1/1     Running   0          4m46s   10.244.2.8    op-worker2   <none>           <none>
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 …
10  ORCLCDB2  racnode2-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 4, 'ORCLCDB1': 6}
```

What 13.6 shows: a full restart of both RAC pods comes back without any manual step: the scripts restore Grid's files with `rootcrs.sh -updateosfiles`, Clusterware starts and moves the VIPs and SCAN to the new pod IPs, the database opens (`AUTOMATIC`), `ORCLPDB` opens on both instances (saved state), `local_listener` holds (spfile), and Windows reaches both instances through the Phase 12 forwards (the NodePorts follow the pods). The state file stays `provisioning`, because writing `completed` is part of the block the guard skips; the operator reports `AVAILABLE` regardless.

13.7 Rules for this lab from now on

RULE	WHY
Don't change the env ConfigMaps (`racnodeNracdb-cmap`) while the database is needed.	Any change makes the operator roll **both** RAC pods together, within minutes, with a 30 s stop: a hard restart of the whole cluster.
Changing `envVars` in the `RacDatabase` has no effect on this cluster.	For an `instanceDetails`-style spec they only go into the ConfigMaps when the database is first created. `SKIP_DBCA` can't be in the YAML from the start, since the first start must run DBCA; for a new build, add it to both ConfigMaps once the database exists, before any restart, as in 13.6.
Before a planned restart: stop the database, then Clusterware on each pod.	Kubernetes gives a pod 30 s, too short for RAC to stop cleanly. With the guard in place the pods come back by themselves (13.6).
After a restart, check the pod IPs.	Each new pod gets a new IP; Clusterware followed (VIPs and SCAN), and so did the names and NodePorts, but anything that uses a pod IP directly won't.
Undo the guard: remove the `SKIP_DBCA=true` line from both ConfigMaps (or restore `cm-…-before-13.4.yaml`), and `SKIP_DBCA` from `racdb.yaml`.	Needed only for operations that should run the database part of the setup again; it rolls both pods.

A full VM shutdown and start was tested in Phase 14: it works without a manual step if the database is stopped with `crsctl stop crs`; see its runbook (14.6).

PHASE 14

Full shutdown and cold start of the VMs

Result of this phase: the whole lab survives a shutdown of all three VMs and comes back with **no manual step**: Kubernetes, the RAC pods, Grid, Clusterware, the database, `ORCLPDB` on both instances, the operator's `AVAILABLE`, and the Phase 12 access from Windows. The phase ends with a start/stop runbook (14.6) to use whenever the PC is shut down.

What this phase found:

- **How you stop the database decides whether it starts by itself.** `srvctl stop database` sets the database resource's target to `OFFLINE`; Clusterware keeps that target across restarts, so after a cold start everything came back except the database, which needed `srvctl start database` (first cycle, 14.2–14.4). `crsctl stop crs` alone shuts each instance down with its stop option (`immediate`) but leaves the target `ONLINE`; after the second cold start the database opened by itself (14.5).
- **The guard from Phase 13 works after a cold start too.** On both pods the setup script restored Grid's files (`restore_gi_files`, `rootcrs.sh -updateosfiles`), found Grid configured, and skipped the database block (`SKIP_DBCA`): no DBCA.
- **Each VM boot gives each RAC pod a new container** (restart count +1) and new pod IP; the StatefulSets don't roll (current and update revision unchanged). Clusterware moves the VIPs and SCAN to the new IPs, and the Services' endpoints follow by themselves.
- **Timing:** about 1 minute from VM start to the Kubernetes API; Grid restored about 3–4 minutes after the VMs start; pods `Ready` after 6–9 minutes; with the target `ONLINE`, both instances open about 11 minutes after the VMs were started.
- **The operator lags behind.** After a start it shows `racdb` `PENDING` (and once the connect string `…/ORCLPDB`) until its next check, about every 2 minutes; then `AVAILABLE` with `orclpdbsvc` again.

What happened, in order (times US Eastern, 29 October 2026):

TIME	STEP	WHAT HAPPENED
20:54–20:58	14.1	baseline: all as after Phase 13; manual OCR backup; the 10/0 SCAN test explained by load balancing
20:59–21:01	14.2 a	clean stop, first way: `srvctl stop database`, then `crsctl stop crs` on both pods
21:02–21:03	14.2 b	VMs shut down from inside (workers, then op-controlplane), all off at 21:03:22
21:04–21:06	14.3 a	op-controlplane started, API ready after 53 s, workers started
21:06–21:11	14.3 b	new containers, Grid restored, Clusterware up, pods `Ready` at 21:10:30 and 21:11:01; database stays down (target `OFFLINE`)

TIME	STEP	WHAT HAPPENED
21:14–21:18	14.4	database started by hand (21:14:16–21:16:51); `racdb` `AVAILABLE` at 21:18:07
21:20–21:21	14.5 a	clean stop, second way: `crsctl stop crs` only (64 s)
21:22–21:24	14.5 b	VMs shut down, all off at 21:24:10
21:24–21:26	14.5 c	VMs started, API ready after 59 s
21:27–21:36	14.5 d	ORCLCDB2 open by itself at 21:33:40, ORCLCDB1 at 21:35:44; no manual step

14.1 Baseline before the shutdown

What: record everything a cold start has to bring back, so the result can be compared; take a manual OCR backup; run the SCAN test from Windows. Only reads, apart from the OCR backup.

a.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/rac-op
echo "=== nodes / operator / RAC pods"
kubectl get nodes -o wide
kubectl -n oracle-database-operator-system get pods
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
kubectl -n rac get sts racnode1 racnode2 -o custom-
columns='NAME:.metadata.name,CURRENT:.status.currentRevision,UPDATE:.status.updateRevision'
echo "=== guard in the env ConfigMaps"
for c in racnode1racdb-cmap racnode2racdb-cmap; do echo "$c: $(kubectl -n rac get cm $c -o
jsonpath='{.data.envfile}' | grep -cE '^SKIP_DBCA=true$') SKIP_DBCA line(s), $(kubectl -n rac get cm $c -
o jsonpath='{.data.envfile}' | grep -c '=') keys"; done
echo "=== autostart: CRS on each pod, database policy"
for p in racnode1-0 racnode2-0; do echo "$p: $(kubectl -n rac exec $p -c $p --
/u01/app/19c/grid/bin/crsctl config crs 2>&1 | tr '\n' ' ')"; done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl config database -d ORCLCDB | grep
-E 'Management policy|Start option|Stop option'; srvctl status database -d ORCLCDB; srvctl status service
-d ORCLCDB"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl config nodeapps | grep 'VIP IPv4';
srvctl config scan | grep 'IPv4 VIP'"
echo "=== PDB, saved state, local_listener"
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
set lines 200 pages 50
col value for a90
col name for a10
col con_name for a10
col instance_name for a12
col state for a10
select inst_id, name, open_mode from gv$pdbs where name = 'ORCLPDB' order by 1;
select con_name, instance_name, state from dba_pdb_saved_states order by 2;
select inst_id, value from gv$parameter where name = 'local_listener' order by 1;
EOF
echo "=== manual OCR backup"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- /u01/app/19c/grid/bin/ocrconfig -manualbackup
kubectl -n rac exec racnode1-0 -c racnode1-0 -- /u01/app/19c/grid/bin/ocrconfig -showbackup manual | tail
-3
```

Check: three nodes `Ready`, the operator `Running`; both pods `1/1`, CURRENT = UPDATE; `1 SKIP_DBCA line(s), 41 keys` for each ConfigMap; `CRS-4622 … autostart is enabled` on both pods; `Management policy: AUTOMATIC`; `ORCLPDB READ WRITE` on 1 and 2 with two saved-state rows; the two vip `local_listener` values; a new OCR backup line.

This lab's output:

Output what this lab printed; nothing to run

```
=== nodes / operator / RAC pods
NAME              STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                          CONTAINER-RUNTIME
op-controlplane   Ready    control-plane   2d21h   v1.36.5   10.0.1.20     <none>        Oracle Linux Server
9.8   5.15.0-325.221.6.6.el9uek.x86_64 (amd64)   cri-o://1.36.7
op-worker1        Ready    worker          2d21h   v1.36.5   10.0.1.21     <none>        Oracle Linux Server
9.8   5.15.0-325.221.6.6.el9uek.x86_64 (amd64)   cri-o://1.36.7
op-worker2        Ready    worker          2d21h   v1.36.5   10.0.1.22     <none>        Oracle Linux Server
9.8   5.15.0-325.221.6.6.el9uek.x86_64 (amd64)   cri-o://1.36.7
NAME                                                    READY   STATUS    RESTARTS   AGE
oracle-database-operator-controller-manager-54cf89dd45-f644n   1/1     Running   1          2d12h
NAME         READY   STATUS    RESTARTS   AGE   IP            NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   0          25m   10.244.1.10   op-worker1   <none>           <none>
racnode2-0   1/1     Running   0          25m   10.244.2.8    op-worker2   <none>           <none>
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
NAME       CURRENT               UPDATE
racnode1   racnode1-75fbb798ff   racnode1-75fbb798ff
racnode2   racnode2-c48dbfc47    racnode2-c48dbfc47
=== guard in the env ConfigMaps
racnode1racdb-cmap: 1 SKIP_DBCA line(s), 41 keys
racnode2racdb-cmap: 1 SKIP_DBCA line(s), 41 keys
=== autostart: CRS on each pod, database policy
racnode1-0: CRS-4622: Oracle High Availability Services autostart is enabled.
racnode2-0: CRS-4622: Oracle High Availability Services autostart is enabled.
Start options: open
Stop options: immediate
Management policy: AUTOMATIC
Instance ORCLCDB1 is running on node racnode1-0
Instance ORCLCDB2 is running on node racnode2-0
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
VIP IPv4 Address: 10.244.1.10
VIP IPv4 Address: 10.244.2.8
SCAN 1 IPv4 VIP: 10.244.1.10
SCAN 2 IPv4 VIP: 10.244.2.8
=== PDB, saved state, local_listener

   INST_ID NAME       OPEN_MODE
---------- ---------- ----------
         1 ORCLPDB    READ WRITE
         2 ORCLPDB    READ WRITE



CON_NAME   INSTANCE_NAM STATE
---------- ------------ ----------
ORCLPDB    ORCLCDB1     OPEN
ORCLPDB    ORCLCDB2     OPEN
```

```
   INST_ID VALUE
---------- --------------------------------------------------------------------------------
         1 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
         2 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))

=== manual OCR backup

racnode2-0     2026/10/30 00:54:50     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261030_005450.ocr.265.1247964891
1753837251

racnode1-0     2026/10/29 23:57:58     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261029_235758.ocr.263.1247961479
1753837251
racnode2-0     2026/10/30 00:54:50     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261030_005450.ocr.265.1247964891
1753837251

racnode1-0     2026/10/29 23:57:58     +VOTEDG:/racnode-c/OCRBACKUP/backup_20261029_235758.ocr.263.1247961479
1753837251
```

The OCR backup times are UTC: `00:54:50` is 20:54:50 Eastern. Clusterware wrote it on racnode2-0, whichever node holds the OCR master.

b. The VM layer and the SCAN test (type the password at the prompt):

Windows PowerShell

```
CONNECT FIRST  local, on your PC

$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
foreach ($p in 2220,2221,2222) { "=== port $p"; ssh -p $p root@127.0.0.1 'hostname -s; uptime -p;
systemctl is-enabled kubelet crio; grep HugePages_Total /proc/meminfo; free -g | grep Mem' }
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: only the three `op-*` VMs; `enabled enabled` on each node; 2048 hugepages on both workers; the test reaches the database.

This lab's output:

Output what this lab printed; nothing to run

```
"op-controlplane" {e4aec2c6-bbe6-439e-a4bd-e0743189316f}
"op-worker1" {8164c82b-4b03-49a4-b193-671b310c6962}
"op-worker2" {05c9cfea-2eba-4f10-8c54-9c1b2919b592}
=== port 2220
op-controlplane
up 2 days, 22 hours, 10 minutes
enabled
enabled
HugePages_Total:       0
Mem:               3           1           0           0           1           2
=== port 2221
op-worker1
up 2 days, 38 minutes
enabled
enabled
HugePages_Total:    2048
Mem:              21           8           3           1          10          12
=== port 2222
op-worker2
up 2 days, 38 minutes
enabled
enabled
HugePages_Total:    2048
Mem:              21           8           2           1          11          12
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB2  racnode2-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB2  racnode2-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB2  racnode2-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB2  racnode2-0  ORCLPDB
 9  ORCLCDB2  racnode2-0  ORCLPDB
10  ORCLCDB2  racnode2-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 10}
```

c. Why every connection went to ORCLCDB2 (read-only, then the test again):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl status scan_listener; srvctl status listener -listener LISTENER"
for p in racnode1-0 racnode2-0; do
  for l in LISTENER_SCAN1 LISTENER_SCAN2 LISTENER; do
    echo "=== $p $l: orclpdbsvc handlers"
    kubectl -n rac exec $p -c $p -- su - grid -c "lsnrctl services $l 2>/dev/null | sed -n '/Service \"orclpdbsvc/,/^Service /p' | grep -E 'Service|Instance|DEDICATED|REMOTE|ADDRESS|established'"
  done
done
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl config service -d ORCLCDB -s orclpdbsvc | grep -E 'Preferred|Available|Connection Load|Runtime Load|Service role'"
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba" <<'EOF'
set lines 200 pages 50
col value for a90
select inst_id, value from gv$parameter where name = 'remote_listener' order by 1;
select inst_id, count(*) sessions from gv$session where type = 'USER' group by inst_id order by 1;
EOF
kubectl -n rac get endpoints racnode-scan-lsnr racnode1-0-lsnr-ext racnode2-0-lsnr-ext
```

Windows PowerShell

CONNECT FIRST local, on your PC

```
Test-NetConnection 127.0.0.1 -Port 31522 | Select-Object TcpTestSucceeded
Test-NetConnection 127.0.0.1 -Port 31523 | Select-Object TcpTestSucceeded
python D:\rac-k8s-poc-operator\scan_test.py
```

Check: both SCAN listeners have a handler for each instance; both `remote_listener` values point at SCAN; the endpoints carry the current pod IPs; both ports `True`.

This lab's output (a SCAN listener runs on only one pod, so `exit code 1` for the other is expected):

Output what this lab printed; nothing to run

```
SCAN Listener LISTENER_SCAN1 is enabled
SCAN listener LISTENER_SCAN1 is running on node racnode1-0
SCAN Listener LISTENER_SCAN2 is enabled
SCAN listener LISTENER_SCAN2 is running on node racnode2-0
Listener LISTENER is enabled
Listener LISTENER is running on node(s): racnode1-0,racnode2-0
=== racnode1-0 LISTENER_SCAN1: orclpdbsvc handlers
Service "orclpdbsvc" has 2 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:6 refused:0 state:ready
         REMOTE SERVER
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:15 refused:0 state:ready
         REMOTE SERVER
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
=== racnode1-0 LISTENER_SCAN2: orclpdbsvc handlers
command terminated with exit code 1
=== racnode1-0 LISTENER: orclpdbsvc handlers
Service "orclpdbsvc" has 1 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:6 refused:0 state:ready
=== racnode2-0 LISTENER_SCAN1: orclpdbsvc handlers
command terminated with exit code 1
=== racnode2-0 LISTENER_SCAN2: orclpdbsvc handlers
Service "orclpdbsvc" has 2 instance(s).
  Instance "ORCLCDB1", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:0 refused:0 state:ready
         REMOTE SERVER
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:0 refused:0 state:ready
         REMOTE SERVER
         (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))
=== racnode2-0 LISTENER: orclpdbsvc handlers
Service "orclpdbsvc" has 1 instance(s).
  Instance "ORCLCDB2", status READY, has 1 handler(s) for this service...
      "DEDICATED" established:15 refused:0 state:ready
Service role: PRIMARY
Connection Load Balancing Goal: LONG
Runtime Load Balancing Goal: NONE
Preferred instances: ORCLCDB1,ORCLCDB2
Available instances:

   INST_ID VALUE
---------- ----------------------------------------------------------------------------------------
         1  racnode-scan:1521
         2  racnode-scan:1521
```

```
   INST_ID   SESSIONS
---------- ----------
         1          8
         2          6

Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                  ENDPOINTS           AGE
racnode-scan-lsnr     10.244.1.10:1521    2d
racnode1-0-lsnr-ext   10.244.1.10:31522   22h
racnode2-0-lsnr-ext   10.244.2.8:31523    22h

TcpTestSucceeded
----------------
            True

TcpTestSucceeded
----------------
            True

SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB2  racnode2-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB2  racnode2-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB1  racnode1-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB1  racnode1-0  ORCLPDB
10  ORCLCDB1  racnode1-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 6, 'ORCLCDB1': 4}
```

What it shows: nothing was wrong. The service's **connection load balancing goal is** `LONG`: SCAN sends each new connection to the instance with fewer sessions. ORCLCDB1 had more user sessions (8 against 6, background agents included), so SCAN kept choosing ORCLCDB2 until the counts evened out; the second run shows exactly that (five to ORCLCDB2, then mixed). A 10/0 result from this test therefore doesn't mean an instance is missing; the handlers in the SCAN listeners do.

One gap the output also shows (from Phase 12, not caused here): the operator's `racnode-scan-lsnr` Service has **one** endpoint, racnode1-0's. Windows reaches SCAN only through LISTENER_SCAN1 on racnode1-0, so with racnode1-0 down `127.0.0.1:31521` fails although ORCLCDB2 is up. Listed under open points.

14.2 Clean stop, first way: `srvctl stop database`, then Clusterware and the VMs

Interruption: the whole lab, until 14.4.

a. The database, then Clusterware (racnode2-0 first):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
date
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl stop database -d ORCLCDB; srvctl status database -d ORCLCDB"
for p in racnode2-0 racnode1-0; do
  echo "=== crsctl stop crs on $p"
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl stop crs 2>&1 | grep -E 'CRS-279[13]|CRS-4133|CRS-4687|CRS-4000|error|fail'
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl check crs
done
sleep 20
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
date
```

Check: both instances `not running`; `CRS-4133` then `CRS-4639` on each pod; the pods `0/1` but `Running` (no liveness probe, so Kubernetes doesn't restart them). Go on to b only if both CRS stops completed.

This lab's output:

Output what this lab printed; nothing to run

```
Thu Oct 29 08:59:35 PM EDT 2026
Instance ORCLCDB1 is not running on node racnode1-0
Instance ORCLCDB2 is not running on node racnode2-0
=== crsctl stop crs on racnode2-0
CRS-2791: Starting shutdown of Oracle High Availability Services-managed resources on 'racnode2-0'
CRS-2793: Shutdown of Oracle High Availability Services-managed resources on 'racnode2-0' has completed
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
=== crsctl stop crs on racnode1-0
CRS-2791: Starting shutdown of Oracle High Availability Services-managed resources on 'racnode1-0'
CRS-2793: Shutdown of Oracle High Availability Services-managed resources on 'racnode1-0' has completed
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
NAME                        READY   STATUS        RESTARTS   AGE   IP            NODE         NOMINATED NODE
READINESS GATES
disk-check-daemonset-62zp8  1/1     Terminating   0          33s   10.244.2.9    op-worker2   <none>
<none>
disk-check-daemonset-k2sp9  1/1     Terminating   0          33s   10.244.1.11   op-worker1   <none>
<none>
racnode1-0                  0/1     Running       0          32m   10.244.1.10   op-worker1   <none>
<none>
racnode2-0                  0/1     Running       0          32m   10.244.2.8    op-worker2   <none>
<none>
NAME    DBNAME    DBSTATE        ROLE           VERSION        PDB CONNECT STR   STATE
racdb   ORCLCDB   NOTAVAILABLE   NOTAVAILABLE   NOTAVAILABLE   NOTAVAILABLE      PENDING
Thu Oct 29 09:01:41 PM EDT 2026
```

`disk-check-daemonset` : the operator starts this short-lived disk check on each worker when it reconciles a database that isn't available; it removes it again by itself. `PENDING` is the operator's state for a database it can't reach.

b. Shut the VMs down from inside the guests, workers first, then op-controlplane. The block waits up to 5 minutes per VM and never forces a power-off:

Windows PowerShell

CONNECT FIRST local, on your PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
function Wait-Off($name) { $t0 = Get-Date; for ($i = 0; $i -lt 60; $i++) { Start-Sleep 5; if (-not ((&
$VB list runningvms) -match "`"$name")) { return "$name off after $([int]((Get-Date) - $t0).TotalSeconds)
s" } }; return "$name STILL RUNNING after 300 s - stop here, don't force" }
Get-Date
foreach ($p in 2222, 2221) { ssh -o ConnectTimeout=5 -p $p root@127.0.0.1 'hostname -s; systemctl
poweroff' }
Wait-Off "op-worker2"; Wait-Off "op-worker1"
ssh -o ConnectTimeout=5 -p 2220 root@127.0.0.1 'hostname -s; systemctl poweroff'
Wait-Off "op-controlplane"
"--- running VMs now:"; & $VB list runningvms
Get-Date
```

Check: three `… off after N s` lines and nothing under "running VMs now".

This lab's output:

Output what this lab printed; nothing to run

```
Thursday, October 29, 2026 9:01:59 PM

op-worker2
op-worker1
op-worker2 off after 41 s
op-worker1 off after 5 s
op-controlplane
op-controlplane off after 36 s
--- running VMs now:

Thursday, October 29, 2026 9:03:22 PM
```

Both workers shut down at the same time; `op-worker1 … 5 s` counts only from the moment worker2 was off.

14.3 Cold start

a. op-controlplane first, the workers once the API answers:

Windows PowerShell

CONNECT FIRST local, on your PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
Get-Date
& $VB startvm op-controlplane --type headless
$t0 = Get-Date; $ok = $false
for ($i = 0; $i -lt 60; $i++) { Start-Sleep 5; $r = ssh -o ConnectTimeout=3 -o BatchMode=yes -p 2220
root@127.0.0.1 'kubectl get --raw=/readyz 2>/dev/null' 2>$null; if ($r -eq 'ok') { $ok = $true; break } }
if ($ok) { "API ready after $([int]((Get-Date) - $t0).TotalSeconds) s" } else { "API NOT ready after 300
s - stop here" }
if ($ok) {
  foreach ($n in "op-worker1","op-worker2") { & $VB startvm $n --type headless }
  $t0 = Get-Date
  for ($i = 0; $i -lt 60; $i++) { Start-Sleep 10; $r = ssh -o ConnectTimeout=3 -p 2220 root@127.0.0.1
'kubectl get nodes --no-headers | grep -cw Ready'; if ($r -eq '3') { break } }
  "3 nodes Ready after $([int]((Get-Date) - $t0).TotalSeconds) s"
}
Get-Date
```

Check: `API ready after N s`, then both workers started.

This lab's output (the first version of the node count used `awk` inside a double-quoted PowerShell string; the escaping left a stray `\` for awk, so the count never worked; the block above has the fixed line, which returned at once when run on its own afterwards):

Output what this lab printed; nothing to run

```
Thursday, October 29, 2026 9:04:21 PM

Waiting for VM "op-controlplane" to power on...
VM "op-controlplane" has been successfully started.
API ready after 53 s
Waiting for VM "op-worker1" to power on...
VM "op-worker1" has been successfully started.
Waiting for VM "op-worker2" to power on...
VM "op-worker2" has been successfully started.
awk: cmd. line:1: \$2=="Ready"
awk: cmd. line:1: ^ backslash not last character on line
…
```

The node count is not a real check right after a start: in 14.5 c the fixed line reported `3 nodes Ready after 11 s`, before the workers could have booted. After the control plane restarts, Kubernetes keeps showing each node's last status until the node controller's grace period runs out. The runbook (14.6) waits for kubelet on the workers instead; the RAC watch below is the real check.

b. Watch RAC come back (every 30 s, up to 25 minutes, ends when both instances run):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
cd /root/rac-op; date
for i in $(seq 1 50); do
  pods=$(kubectl -n rac get pods racnode1-0 racnode2-0 --no-headers -o custom-
columns='N:.metadata.name,R:.status.containerStatuses[0].ready,C:.status.containerStatuses[0].restartCoun
t,S:.status.phase' 2>&1 | tr -s ' ' | tr '\n' ' ')
  st=$(kubectl -n rac get racdatabase racdb -o jsonpath='{.status.state}' 2>/dev/null)
  db=$(kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB"
2>&1 | tr '\n' ' ' | cut -c1-140)
  echo "$(date +%T)  [$pods] racdb=$st  $db"
  echo "$db" | grep -q 'ORCLCDB1 is running' && echo "$db" | grep -q 'ORCLCDB2 is running' && break
  sleep 30
done
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
kubectl -n rac get events --sort-by=.lastTimestamp | tail -20
```

This lab's output (stopped with Ctrl+C at 21:12, see below):

`Output` what this lab printed; nothing to run

```
Thu Oct 29 09:06:04 PM EDT 2026
21:06:05  [racnode1-0 false 0 Running racnode2-0 false 0 Running ] racdb=PENDING  error: unable to upgrade connection: error dialing backend: dial tcp 10.0.1.21:10250: connect: connection refused
21:06:35  [racnode1-0 false 1 Running racnode2-0 false 0 Running ] racdb=PENDING  error: unable to upgrade connection: container not found ("racnode1-0")
21:07:06  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  error: unable to upgrade connection: container not found ("racnode1-0")
21:07:38  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  ****Unable to retrieve Oracle Clusterware home. Start Oracle Clusterware stack and try again. command terminated with exit code 1
21:08:09  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  ****Unable to retrieve Oracle Clusterware home. Start Oracle Clusterware stack and try again. command terminated with exit code 1
21:08:56  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  PRKH-1010 : Unable to communicate with CRS services. PRKH-3003 : An attempt to communicate with the CSS daemon failed command terminated wit
21:09:27  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  PRCD-1027 : Failed to retrieve database ORCLCDB PRCR-1070 : Failed to check if resource ora.orclcdb.db is registered CRS-0184 : Cannot commu
21:09:59  [racnode1-0 false 1 Running racnode2-0 false 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:10:30  [racnode1-0 true 1 Running racnode2-0 false 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:11:01  [racnode1-0 true 1 Running racnode2-0 true 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:11:33  [racnode1-0 true 1 Running racnode2-0 true 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:12:04  [racnode1-0 true 1 Running racnode2-0 true 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:12:35  [racnode1-0 true 1 Running racnode2-0 true 1 Running ] racdb=PENDING  Instance ORCLCDB1 is not running on node racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
```

How to read the stages: `connection refused` on port 10250 = kubelet on the worker not up yet; `container not found` = kubelet is creating the new container (restart count 1); `srvctl: command not found` (seen in 14.5) = the setup script hasn't written the user environment yet; `Unable to retrieve Oracle Clusterware home` = Grid's files not restored yet; `PRKH-1010`, `CRS-0184` = Clusterware starting; `not running` with the pods `Ready` = Clusterware up, database not started. Here it stayed at the last stage, because `srvctl stop database` had set the database's target to `OFFLINE` (14.4 confirms). That was also the case in 13.3, where the database was started by hand for the same reason.

14.4 Confirm the cause, start the database, check everything

a.

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
echo "=== database resource target/state (expect TARGET=OFFLINE)"
kubectl -n rac exec racnode1-0 -c racnode1-0 -- /u01/app/19c/grid/bin/crsctl stat res ora.orclcdb.db -t
kubectl -n rac exec racnode1-0 -c racnode1-0 -- /u01/app/19c/grid/bin/crsctl stat res -t | grep -A3 -E
'^ora\.(asm|LISTENER|scan|DATA|FRA|VOTEDG)' | head -40
echo "=== setup log on both pods"
for p in racnode1-0 racnode2-0; do kubectl -n rac exec $p -c $p -- bash -c
'L=/tmp/orod/oracle_db_setup.log; echo "--- $(hostname) statefile: $(cat /tmp/orod/.statefile)"; grep -E
"SKIP_DBCA key exists|restore_gi_files invoked|updateosfiles|already configured|Total time for
setup|create_db|dbca -silent" $L | tail -8'; done
echo "=== start the database"; date
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl start database -d ORCLCDB; srvctl
status database -d ORCLCDB; srvctl status service -d ORCLCDB"; date
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
set lines 200 pages 50
col value for a90
col name for a10
select inst_id, name, open_mode from gv$pdbs where name = 'ORCLPDB' order by 1;
select inst_id, value from gv$parameter where name = 'local_listener' order by 1;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl config nodeapps | grep 'VIP IPv4';
srvctl config scan | grep 'IPv4 VIP'"
kubectl -n rac get sts racnode1 racnode2 -o custom-
columns='NAME:.metadata.name,CURRENT:.status.currentRevision,UPDATE:.status.updateRevision'
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
kubectl get endpointslices -n rac -o wide | grep -E 'scan|lsnr-ext'
kubectl -n rac get events --sort-by=.lastTimestamp | tail -15
```

Check: `ora.orclcdb.db` `OFFLINE OFFLINE` before the start, everything else `ONLINE`; the guard and `restore_gi_files` in the setup log, no `create_db` or `dbca`; after the start both instances and the service on both; `ORCLPDB` `READ WRITE` on 1 and 2; the two vip `local_listener` values; CURRENT = UPDATE.

This lab's output:

Output what this lab printed; nothing to run

```
=== database resource target/state (expect TARGET=OFFLINE)
--------------------------------------------------------------------------------
Name           Target  State        Server                   State details
--------------------------------------------------------------------------------
Cluster Resources
--------------------------------------------------------------------------------
ora.orclcdb.db
      1        OFFLINE OFFLINE                               Instance Shutdown,ST
                                                             ABLE
      2        OFFLINE OFFLINE                               Instance Shutdown,ST
                                                             ABLE
--------------------------------------------------------------------------------
ora.LISTENER.lsnr
               ONLINE  ONLINE       racnode1-0               STABLE
               ONLINE  ONLINE       racnode2-0               STABLE
ora.chad
--
ora.DATA.dg(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               STABLE
      2        ONLINE  ONLINE       racnode2-0               STABLE
ora.FRA.dg(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               STABLE
      2        ONLINE  ONLINE       racnode2-0               STABLE
ora.LISTENER_SCAN1.lsnr
      1        ONLINE  ONLINE       racnode1-0               STABLE
ora.LISTENER_SCAN2.lsnr
      1        ONLINE  ONLINE       racnode2-0               STABLE
ora.VOTEDG.dg(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               STABLE
      2        ONLINE  ONLINE       racnode2-0               STABLE
ora.asm(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               Started,STABLE
      2        ONLINE  ONLINE       racnode2-0               Started,STABLE
ora.asmnet1.asmnetwork(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               STABLE
      2        ONLINE  ONLINE       racnode2-0               STABLE
ora.asmnet2.asmnetwork(ora.asmgroup)
      1        ONLINE  ONLINE       racnode1-0               STABLE
      2        ONLINE  ONLINE       racnode2-0               STABLE
ora.cvu
--
ora.scan1.vip
      1        ONLINE  ONLINE       racnode1-0               STABLE
ora.scan2.vip
      1        ONLINE  ONLINE       racnode2-0               STABLE
--------------------------------------------------------------------------------
=== setup log on both pods
--- racnode1-0 statefile: provisioning
```

```
10/30/2026 01:07:23 AM     INFO:   oracommon   - populateracenvvars :SKIP_DBCA key exists with value true
10/30/2026 01:08:07 AM     INFO:   oracommon   -   restoregifiles   :restore_gi_files invoked (Oracle major
version: 19)
10/30/2026 01:08:07 AM     INFO:   oracommon   -   executeprocess   :Received Command :
/u01/app/19c/grid/crs/install/rootcrs.sh -updateosfiles
10/30/2026 01:09:03 AM     INFO:  oramachine   -       setup        :Total time for setup() = [ 103.557 ]
seconds
          Grid is already configured on this machine
10/30/2026 01:09:19 AM     INFO:   oragiprov   -       setup        :Total time for setup() = [ 16.214 ]
seconds
10/30/2026 01:13:17 AM     INFO:  oraracprov   -       setup        :Total time for setup() = [ 254.485 ]
seconds
--- racnode2-0 statefile: provisioning
10/30/2026 01:07:40 AM     INFO:   oracommon   - populateracenvvars :SKIP_DBCA key exists with value true
10/30/2026 01:08:27 AM     INFO:   oracommon   -   restoregifiles   :restore_gi_files invoked (Oracle major
version: 19)
10/30/2026 01:08:27 AM     INFO:   oracommon   -   executeprocess   :Received Command :
/u01/app/19c/grid/crs/install/rootcrs.sh -updateosfiles
10/30/2026 01:09:02 AM     INFO:  oramachine   -       setup        :Total time for setup() = [ 82.257 ]
seconds
=== start the database
Thu Oct 29 09:14:16 PM EDT 2026
Instance ORCLCDB1 is running on node racnode1-0
Instance ORCLCDB2 is running on node racnode2-0
Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
Thu Oct 29 09:16:51 PM EDT 2026

   INST_ID NAME       OPEN_MODE
---------- ---------- ----------
         1 ORCLPDB    READ WRITE
         2 ORCLPDB    READ WRITE



   INST_ID VALUE
---------- --------------------------------------------------------------------------------------------
         1 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)(PORT=31522))
         2 (ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)(PORT=31523))

VIP IPv4 Address: 10.244.1.2
VIP IPv4 Address: 10.244.2.2
SCAN 1 IPv4 VIP: 10.244.1.2
SCAN 2 IPv4 VIP: 10.244.2.2
NAME       CURRENT               UPDATE
racnode1   racnode1-75fbb798ff   racnode1-75fbb798ff
racnode2   racnode2-c48dbfc47    racnode2-c48dbfc47
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   1          47m   10.244.1.2   op-worker1   <none>           <none>
racnode2-0   1/1     Running   1          47m   10.244.2.2   op-worker2   <none>           <none>
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR             STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/ORCLPDB   PENDING
racnode-scan-69fqn          IPv4          <unset>   10.244.1.2,10.244.2.2   2d
racnode-scan-lsnr-qbpd2     IPv4          1521      10.244.1.2              2d
racnode1-0-lsnr-ext-tgml2   IPv4          31522     10.244.1.2              23h
```

```
racnode2-0-lsnr-ext-spfhc   IPv4          31523     10.244.2.2              23h
9m45s       Normal    Started                 pod/racnode1-0                  Container started
9m44s       Normal    Pulled                  pod/racnode1-0                  Container image
"localhost/oracle/database-rac:19.3.0-slim" already present on machine and can be accessed by the pod
9m44s       Normal    Started                 pod/racnode1-0                  Container started
9m44s       Normal    Created                 pod/racnode1-0                  Container created
9m38s       Warning   FailedCreatePodSandBox  pod/racnode2-0                  Failed to create pod sandbox:
rpc error: code = Unknown desc = failed to create pod network sandbox k8s_racnode2-0_rac_e8ed6d9c-…
(c175002c…): error adding pod rac_racnode2-0 to CNI network "multus-cni-network": plugin type="multus-shim"
name="multus-cni-network" failed (add): … error adding container to network "cbr0": plugin type="flannel"
failed (add): failed to load flannel 'subnet.env' file: open /run/flannel/subnet.env: no such file or
directory. Check the flannel pod log for this node....
9m24s       Normal    AddedInterface          pod/racnode2-0                  Add eth0 [10.244.2.2/24] from
cbr0
9m23s       Normal    AddedInterface          pod/racnode2-0                  Add eth1 [10.0.11.141/24]
from rac/macvlan-conf1
9m23s       Normal    Pulled                  pod/racnode2-0                  Container image
"localhost/oracle/database-rac:19.3.0-slim" already present on machine and can be accessed by the pod
9m23s       Normal    AddedInterface          pod/racnode2-0                  Add eth2 [10.0.12.141/24]
from rac/macvlan-conf2
9m22s       Normal    Created                 pod/racnode2-0                  Container created
9m22s       Normal    Started                 pod/racnode2-0                  Container started
9m21s       Normal    Pulled                  pod/racnode2-0                  Container image
"localhost/oracle/database-rac:19.3.0-slim" already present on machine and can be accessed by the pod
9m21s       Normal    Created                 pod/racnode2-0                  Container created
9m21s       Normal    Started                 pod/racnode2-0                  Container started
6m20s       Warning   Unhealthy               pod/racnode2-0                  Readiness probe failed: dial
tcp 10.244.2.2:1522: connect: connection refused
```

What it shows: Clusterware brought back everything whose target was `ONLINE` (ASM, the disk groups, the listeners, SCAN); only the database, target `OFFLINE`, stayed down. The guard worked on both pods (no `create_db`, no `dbca`); racnode2-0's grep ends at `oramachine` because its later steps logged nothing matching. The setup log's times are UTC (01:07 = 21:07 Eastern). The new pod IPs are 10.244.1.2 and 10.244.2.2; VIPs, SCAN VIPs and the Services' endpoints all follow. `FailedCreatePodSandBox` is start-up order on worker2: the pod tried to start before flannel had written `/run/flannel/subnet.env`; kubelet retried 14 s later. The readiness failure is the node listener port (1522) before Clusterware started it.

b. The operator's status, and Windows:

`op-controlplane`

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for i in $(seq 1 10); do echo "$(date +%T)  $(kubectl -n rac get racdatabase racdb --no-headers)";
kubectl -n rac get racdatabase racdb -o jsonpath='{.status.state}' | grep -q AVAILABLE && break; sleep
30; done
kubectl -n oracle-database-operator-system logs deploy/oracle-database-operator-controller-manager --
since=15m 2>/dev/null | grep -iE 'racdb|rac/' | grep -iE 'state|pending|available|error|pdb|connect' |
tail -15
```

Windows PowerShell

CONNECT FIRST local, on your PC

```
python D:\rac-k8s-poc-operator\scan_test.py
```

This lab's output (operator log times UTC; the Service lines repeat for all six Services, two shown):

Output what this lab printed; nothing to run

```
21:17:36  racdb   ORCLCDB   OPEN   PRIMARY   19.32.0.0.0   racnode-scan:1521/ORCLPDB   PENDING
21:18:07  racdb   ORCLCDB   OPEN   PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
2026-10-30T01:16:11Z    INFO    controllers.controllers.RacDatabase     Entering reconcile phase
{"controller": "racdatabase", "namespace": "rac", "name": "racdb", "phase": "PendingAndRecovery"}
2026-10-30T01:16:11Z    INFO    controllers.controllers.RacDatabase     Service reconciled to desired state
{"Instance.Namespace": "rac", "Instance.Name": "racdb", "dep.Name": "racnode1-0-vip", "Service.Namespace":
"rac", "Service.Name": "racnode1-0-vip"}
…
2026-10-30T01:18:04Z    INFO    controllers.controllers.RacDatabase     Entering reconcile phase
{"controller": "racdatabase", "namespace": "rac", "name": "racdb", "phase": "StateGuard"}
2026-10-30T01:18:04Z    INFO    controllers.controllers.RacDatabase     Entering reconcile phase
{"controller": "racdatabase", "namespace": "rac", "name": "racdb", "phase": "PendingAndRecovery"}
2026-10-30T01:18:04Z    INFO    controllers.controllers.RacDatabase     Service reconciled to desired state
{"Instance.Namespace": "rac", "Instance.Name": "racdb", "dep.Name": "racnode-scan", "Service.Namespace":
"rac", "Service.Name": "racnode-scan"}
SYSTEM password:
 1  ORCLCDB2  racnode2-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 3  ORCLCDB2  racnode2-0  ORCLPDB
 4  ORCLCDB1  racnode1-0  ORCLPDB
 5  ORCLCDB2  racnode2-0  ORCLPDB
 6  ORCLCDB2  racnode2-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB1  racnode1-0  ORCLPDB
10  ORCLCDB2  racnode2-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB2': 6, 'ORCLCDB1': 4}
```

What it shows: the operator's check at 21:16:11 ran while the database was still opening, so it kept `PENDING`; its next check, two minutes later, set `AVAILABLE` and the connect string back to `orclpdbsvc`. Windows reaches both instances. **First cycle result:** everything came back by itself except the database, because of how it was stopped.

14.5 Second cycle: stop with `crsctl stop crs` only

Why: with the database's target left `ONLINE`, Clusterware should open it after a cold start without a manual step. `crsctl stop crs` still shuts each instance down cleanly, with the database's stop option (`immediate`).

a. Stop (check the target first):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
date
kubectl -n rac exec racnode1-0 -c racnode1-0 -- /u01/app/19c/grid/bin/crsctl stat res ora.orclcdb.db -t |
grep -A2 '^ora.orclcdb.db'
for p in racnode2-0 racnode1-0; do
  echo "=== crsctl stop crs on $p"
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl stop crs 2>&1 | grep -E
'ora.orclcdb.db|CRS-279[13]|CRS-4133|CRS-4687|CRS-4000|error|fail'
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl check crs
done
date
```

Check: `ONLINE ONLINE` before the stop; on each pod `CRS-2673 … stop 'ora.orclcdb.db'` and `CRS-2677 … succeeded`, then `CRS-4133`.

This lab's output (`grep -A2` shows only instance 1's row, because its details wrap onto a second line):

Output — what this lab printed; nothing to run

```
Thu Oct 29 09:20:00 PM EDT 2026
ora.orclcdb.db
      1        ONLINE  ONLINE       racnode1-0               Open,HOME=/u01/app/o
                                                             racle/product/19c/db
=== crsctl stop crs on racnode2-0
CRS-2791: Starting shutdown of Oracle High Availability Services-managed resources on 'racnode2-0'
CRS-2673: Attempting to stop 'ora.orclcdb.db' on 'racnode2-0'
CRS-2677: Stop of 'ora.orclcdb.db' on 'racnode2-0' succeeded
CRS-2793: Shutdown of Oracle High Availability Services-managed resources on 'racnode2-0' has completed
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
=== crsctl stop crs on racnode1-0
CRS-2791: Starting shutdown of Oracle High Availability Services-managed resources on 'racnode1-0'
CRS-2673: Attempting to stop 'ora.orclcdb.db' on 'racnode1-0'
CRS-2677: Stop of 'ora.orclcdb.db' on 'racnode1-0' succeeded
CRS-2793: Shutdown of Oracle High Availability Services-managed resources on 'racnode1-0' has completed
CRS-4133: Oracle High Availability Services has been stopped.
CRS-4639: Could not contact Oracle High Availability Services
Thu Oct 29 09:21:04 PM EDT 2026
```

b. Shut the VMs down: the same block as 14.2 b.

This lab's output:

Output what this lab printed; nothing to run

```
Thursday, October 29, 2026 9:22:51 PM

op-worker2
op-worker1
op-worker2 off after 36 s
op-worker1 off after 5 s
op-controlplane
op-controlplane off after 36 s
--- running VMs now:

Thursday, October 29, 2026 9:24:10 PM
```

c. Start the VMs: the same block as 14.3 a.

This lab's output (`3 nodes Ready after 11 s` is the stale status described in 14.3, not the workers' real state):

Output what this lab printed; nothing to run

```
Thursday, October 29, 2026 9:24:30 PM

Waiting for VM "op-controlplane" to power on...
VM "op-controlplane" has been successfully started.
API ready after 59 s
Waiting for VM "op-worker1" to power on...
VM "op-worker1" has been successfully started.
Waiting for VM "op-worker2" to power on...
VM "op-worker2" has been successfully started.
3 nodes Ready after 11 s

Thursday, October 29, 2026 9:26:10 PM
```

d. Watch, without starting anything:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
date
for i in $(seq 1 50); do
  pods=$(kubectl -n rac get pods racnode1-0 racnode2-0 --no-headers -o custom-
columns='N:.metadata.name,R:.status.containerStatuses[0].ready,C:.status.containerStatuses[0].restartCoun
t' 2>&1 | tr -s ' ' | tr '\n' ' ')
  db=$(kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status database -d ORCLCDB"
2>&1 | tr '\n' ' ' | cut -c1-140)
  echo "$(date +%T)  [$pods] racdb=$(kubectl -n rac get racdatabase racdb -o jsonpath='{.status.state}'
2>/dev/null)  $db"
  echo "$db" | grep -q 'ORCLCDB1 is running' && echo "$db" | grep -q 'ORCLCDB2 is running' && break
  sleep 30
done
kubectl -n rac exec -i racnode1-0 -c racnode1-0 -- su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s
/ as sysdba" <<'EOF'
set lines 200 pages 50
col name for a10
select inst_id, name, open_mode from gv$pdbs where name = 'ORCLPDB' order by 1;
EOF
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - oracle -c "srvctl status service -d ORCLCDB"
sleep 150; kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
```

Check: the loop ends with both instances `is running`; `ORCLPDB` `READ WRITE` on 1 and 2; the service on both; `racdb` `AVAILABLE`.

This lab's output:

Output what this lab printed; nothing to run

```
Thu Oct 29 09:27:16 PM EDT 2026
21:27:18  [racnode1-0 false 1 racnode2-0 false 1 ] racdb=PENDING  error: unable to upgrade connection:
container not found ("racnode1-0")
21:27:50  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  -bash: line 1: srvctl: command not found
command terminated with exit code 127
21:28:23  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  ****Unable to retrieve Oracle Clusterware
home. Start Oracle Clusterware stack and try again. command terminated with exit code 1
21:28:57  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  PRKH-1010 : Unable to communicate with CRS
services. PRKH-3003 : An attempt to communicate with the CSS daemon failed command terminated wit
21:29:28  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  PRKH-1010 : Unable to communicate with CRS
services. PRKH-3003 : An attempt to communicate with the CSS daemon failed command terminated wit
21:30:00  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  PRCD-1027 : Failed to retrieve database
ORCLCDB PRCR-1070 : Failed to check if resource ora.orclcdb.db is registered CRS-0184 : Cannot commu
21:30:31  [racnode1-0 false 2 racnode2-0 false 2 ] racdb=PENDING  PRCD-1027 : Failed to retrieve database
ORCLCDB PRCR-1070 : Failed to check if resource ora.orclcdb.db is registered CRS-0184 : Cannot commu
21:31:02  [racnode1-0 false 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:31:34  [racnode1-0 false 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:32:05  [racnode1-0 false 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:32:36  [racnode1-0 false 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:33:07  [racnode1-0 false 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is not running on node racnode2-0
21:33:40  [racnode1-0 true 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is running on node racnode2-0
21:34:11  [racnode1-0 true 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is running on node racnode2-0
21:34:42  [racnode1-0 true 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is running on node racnode2-0
21:35:13  [racnode1-0 true 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is not running on node
racnode1-0 Instance ORCLCDB2 is running on node racnode2-0
21:35:44  [racnode1-0 true 2 racnode2-0 true 2 ] racdb=PENDING  Instance ORCLCDB1 is running on node
racnode1-0 Instance ORCLCDB2 is running on node racnode2-0

   INST_ID NAME       OPEN_MODE
---------- ---------- ----------
         1 ORCLPDB    READ WRITE
         2 ORCLPDB    READ WRITE

Service orclpdbsvc is running on instance(s) ORCLCDB1,ORCLCDB2
```

The final lines (after `sleep 150`) were read again separately:

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
kubectl -n rac get pods -o wide; kubectl -n rac get racdatabase racdb
kubectl -n rac exec racnode1-0 -c racnode1-0 -- su - grid -c "srvctl config nodeapps | grep 'VIP IPv4'; srvctl config scan | grep 'IPv4 VIP'"
kubectl -n rac get sts racnode1 racnode2 -o custom-columns='NAME:.metadata.name,CURRENT:.status.currentRevision,UPDATE:.status.updateRevision'
```

Output what this lab printed; nothing to run

```
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE         NOMINATED NODE   READINESS GATES
racnode1-0   1/1     Running   2          69m   10.244.1.5   op-worker1   <none>           <none>
racnode2-0   1/1     Running   2          69m   10.244.2.2   op-worker2   <none>           <none>
NAME    DBNAME    DBSTATE   ROLE      VERSION       PDB CONNECT STR                 STATE
racdb   ORCLCDB   OPEN      PRIMARY   19.32.0.0.0   racnode-scan:1521/orclpdbsvc   AVAILABLE
VIP IPv4 Address: 10.244.1.5
VIP IPv4 Address: 10.244.2.2
SCAN 1 IPv4 VIP: 10.244.1.5
SCAN 2 IPv4 VIP: 10.244.2.2
NAME       CURRENT               UPDATE
racnode1   racnode1-75fbb798ff   racnode1-75fbb798ff
racnode2   racnode2-c48dbfc47    racnode2-c48dbfc47
```

e. Windows, three runs (the first while only ORCLCDB2 was open, the second and third after both were):

Windows PowerShell

CONNECT FIRST local, on your PC

```
python D:\rac-k8s-poc-operator\scan_test.py
```

Output what this lab printed; nothing to run

```
connections per instance: {'ORCLCDB2': 10}
connections per instance: {'ORCLCDB1': 10}
 1  ORCLCDB1  racnode1-0  ORCLPDB
 2  ORCLCDB1  racnode1-0  ORCLPDB
 3  ORCLCDB1  racnode1-0  ORCLPDB
 4  ORCLCDB1  racnode1-0  ORCLPDB
 5  ORCLCDB1  racnode1-0  ORCLPDB
 6  ORCLCDB1  racnode1-0  ORCLPDB
 7  ORCLCDB2  racnode2-0  ORCLPDB
 8  ORCLCDB1  racnode1-0  ORCLPDB
 9  ORCLCDB2  racnode2-0  ORCLPDB
10  ORCLCDB1  racnode1-0  ORCLPDB
oracledb 26.0.1 thin mode: True
connections per instance: {'ORCLCDB1': 8, 'ORCLCDB2': 2}
```

What 14.5 shows: the database came back **without any manual step**: ORCLCDB2 at 21:33:40 and ORCLCDB1 at 21:35:44, about 11 minutes after the VMs were started and 16 minutes after the stop began. Each instance starts once its own node's Clusterware, ASM and disk groups are up; racnode1-0 was about 2½ minutes behind racnode2-0. The first Windows run reached the only open instance through SCAN; the next ones followed the `LONG` balancing (the instance with fewer sessions first). racnode1-0's pod IP changed again (10.244.1.5); the VIP and SCAN VIP followed.

14.6 Runbook: stopping and starting the lab

Use this whenever the PC is shut down or restarted. Each block was run as shown in 14.5, except the worker wait in step 3, which replaces the node count that gave a stale result (14.3).

Stop (about 3 minutes):

1. Stop Clusterware on both pods; the database stops with it and its target stays `ONLINE` (14.5 a):

op-controlplane

CONNECT FIRST `ssh -p 2220 root@127.0.0.1`

```
for p in racnode2-0 racnode1-0; do
  echo "=== crsctl stop crs on $p"
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl stop crs 2>&1 | grep -E
'ora.orclcdb.db|CRS-279[13]|CRS-4133|CRS-4687|CRS-4000|error|fail'
  kubectl -n rac exec $p -c $p -- /u01/app/19c/grid/bin/crsctl check crs
done
```

Check: `CRS-4133` on both pods. If not, don't shut down; find out why first.

2. Shut the VMs down from inside, workers first (14.2 b):

Windows PowerShell

CONNECT FIRST local, on your PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
function Wait-Off($name) { $t0 = Get-Date; for ($i = 0; $i -lt 60; $i++) { Start-Sleep 5; if (-not ((&
$VB list runningvms) -match "`"$name")) { return "$name off after $([int]((Get-Date) - $t0).TotalSeconds)
s" } }; return "$name STILL RUNNING after 300 s - stop here, don't force" }
foreach ($p in 2222, 2221) { ssh -o ConnectTimeout=5 -p $p root@127.0.0.1 'hostname -s; systemctl
poweroff' }
Wait-Off "op-worker2"; Wait-Off "op-worker1"
ssh -o ConnectTimeout=5 -p 2220 root@127.0.0.1 'hostname -s; systemctl poweroff'
Wait-Off "op-controlplane"
"--- running VMs now:"; & $VB list runningvms
```

Check: three `off` lines and no running VMs. Only then shut down or restart Windows.

Start (about 12 minutes until both instances are open):

3. op-controlplane, then the workers once the API answers; then wait until kubelet runs on both workers:

Windows PowerShell

CONNECT FIRST local, on your PC

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB list runningvms
& $VB startvm op-controlplane --type headless
$t0 = Get-Date; $ok = $false
for ($i = 0; $i -lt 60; $i++) { Start-Sleep 5; $r = ssh -o ConnectTimeout=3 -o BatchMode=yes -p 2220
root@127.0.0.1 'kubectl get --raw=/readyz 2>/dev/null' 2>$null; if ($r -eq 'ok') { $ok = $true; break } }
if ($ok) { "API ready after $([int]((Get-Date) - $t0).TotalSeconds) s" } else { "API NOT ready after 300
s - stop here" }
if ($ok) {
  foreach ($n in "op-worker1","op-worker2") { & $VB startvm $n --type headless }
  foreach ($p in 2221, 2222) { $t0 = Get-Date; for ($i = 0; $i -lt 60; $i++) { Start-Sleep 5; $r = ssh -o
ConnectTimeout=3 -o BatchMode=yes -p $p root@127.0.0.1 'systemctl is-active kubelet' 2>$null; if ($r -eq
'active') { break } }; "port ${p}: kubelet $r after $([int]((Get-Date) - $t0).TotalSeconds) s" }
}
```

Check: the first line lists no running VM from the manual lab (one lab at a time); `API ready`; `kubelet active` on both workers.

4. Watch the database open by itself: the 14.5 d block. **Check:** both instances `is running` (about 9–11 minutes after the workers start), `ORCLPDB READ WRITE` on 1 and 2, the service on both, `racdb AVAILABLE` (allow 2 minutes for the operator).
5. Windows: `python D:\rac-k8s-poc-operator\scan_test.py`. A 10/0 split is normal right after a start (`LONG` balancing); the test failing is not.

If the database stays down with the pods `Ready` and `srvctl status database` showing both instances `not running` for more than 5 minutes: check the target with `crsctl stat res ora.orclcdb.db -t` (14.4 a). `OFFLINE` means it was stopped with `srvctl`; start it with `srvctl start database -d ORCLCDB` as `oracle` on racnode1-0. Don't delete pods or change the ConfigMaps (Phase 13).

Not covered: a hard power-off (VirtualBox "Power Off", a Windows crash or power loss) with the database open. Clusterware should recover the instances, but that hasn't been tested here.

What was changed in Phase 14: nothing in the cluster's configuration. A manual OCR backup (`backup_20261030_005450.ocr`) was added; the database's target is `ONLINE` again (as before the phase).

Next steps

After Phase 6 the operator lab is at the same point as the manual lab before its Phase 7, with three deliberate differences the operator asks for: the node directories `racnode1` / `racnode2` on both workers (4f), the worker label `raccluster=raccluster01` (5.4), and the Multus networks in namespace `rac` (6.3). The rest of the worker set-up (kernel settings, SELinux module, `allowedUnsafeSysctls`, hugepages, clock source) was already in the golden image and the kubeadm configuration.

The phases below follow Oracle's Oracle Database Operator for Kubernetes (release v2.2.0, RAC controller `RacDatabase`, API `database.oracle.com/v4`). Phases 7 to 11 are above. Phase 11 is done: the database is open on both nodes (11.5), with the control file and redo fix in 11.6. Phase 12 (above) opened the database to

Windows: SCAN at `127.0.0.1:31521/orclpdbsvc` reaches both instances. Phase 13 made restarts of the RAC pods safe, and Phase 14 showed that the whole lab survives a shutdown and cold start of the VMs; its runbook (14.6) is the way to stop and start the lab.

PHASE	WHAT	FROM THE OPERATOR DOCS
12. Access from Windows (done)	extra node-listener ports, explicit `local_listener`, own NodePorts and Windows forwards, then a SCAN test from Python	`provisioning_oracle_rac_db_with_node_port.md`, `database_connection.md`
13. Safe restart of the RAC pods (done)	the `SKIP_DBCA` guard in the env ConfigMaps, PDB saved state	— (Oracle's setup scripts in the image)
14. Shutdown and cold start of the VMs (done)	clean stop with `crsctl stop crs`, VM shutdown and start, runbook	—

Open points:

- **Which image (settled).** The manual lab's image can't be reused: it was built from the OCNE doc's short Containerfile, and the operator drives the setup through the scripts in Oracle's own RAC image (it follows `/tmp/orod/oracle_db_setup.log`). Oracle doesn't publish the slim image on its registry, so Phase 8 builds it on each worker with `buildContainerImage.sh -v 19.3.0 -i -o '--build-arg SLIMMING=true'` (Oracle Linux 9 base).
- **Sizing (settled in 11.1).** 16 GiB pods with 4 GiB of hugepages fit once the operator runs one controller pod instead of three; the `RAC_WEBHOOK_DEV_MODE` way around the 16 GiB minimum isn't needed.
- **Platform.** The operator's README lists OKE, OpenShift and OLCNE 1.9+ as tested, and GKE, AKS, EKS, OKD and Minikube as supported. That table covers the whole operator; the RAC controller's own prerequisites are written only for OCNE. A plain kubeadm cluster with CRI-O isn't on either list, so expect lab-specific workarounds, as in the manual lab. One showed up so far: the ASM discovery string with by-id disk names (`CRS_ASM_DISCOVERY_STRING` in 11.2, recovery in 11.4).
- **Online redo logs land in FRA.** With no `DBREDO` disk group in the YAML, the setup script sets `DB_CREATE_ONLINE_LOG_DEST_1` to the recovery disk group, so the redo logs are only in `+FRA` (one member per group). That works for the lab; the usual layout keeps redo on DATA or its own disk group with a second member in FRA. The control file too ended up only in `+FRA` (11.5). 11.6 adds a second copy of each in `+DATA`. `db_recovery_file_dest_size` isn't set by the operator; DBCA chose 23895M.
- **Restarts (Phases 13 and 14).** A full restart of both RAC pods, and a shutdown and cold start of all three VMs, come back without help once the env ConfigMaps carry `SKIP_DBCA=true` and the database is stopped with `crsctl stop crs` (not `srvctl stop database`, which leaves its target `OFFLINE`). Any change to the env ConfigMaps makes the operator roll both pods together. A hard power-off with the database open hasn't been tested.
- **SCAN from Windows goes through one pod (found in 14.1).** The operator's `racnode-scan-lsnr` Service has one endpoint, racnode1-0's, so Windows only reaches LISTENER_SCAN1. With racnode1-0 down, `127.0.0.1:31521` fails although ORCLCDB2 is open. Not yet looked into: whether the operator can expose both SCAN listeners, or a NodePort per SCAN listener.
- **Access from outside is set up by hand (Phase 12).** The operator's own node-listener NodePort fields (`baseLsnrTargetPort`) only apply when the database is first created. A new build should set them in the YAML instead (not tested here). An instance-down test (`srvctl stop instance`) wasn't run; the pods have no liveness probe, so it wouldn't restart a pod.

- **VIPs are the pod IPs.** Flannel gives each pod one IP, so `racnode1-0-vip` and `racnode2-0-vip` resolve to the pods' own addresses. A VIP can't move to the other node when its pod dies; client failover relies on SCAN and the service running on the other instance.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.